

Propeller 2 Assembly Beginner Guide

K.L. Wanamaker

August 31, 2026

Contents

1	Getting Started with the Propeller 2	11
1.1	What is P2ASM	12
1.2	How P2ASM Actually Works	12
1.2.1	Cog vs Hub (Execution and Memory)	13
1.2.2	Registers vs Memory	13
1.2.3	Instruction Execution Flow	14
2	Getting started	16
2.1	How the P2 Executes Your Code	16
2.1.1	The Program Counter	17
2.1.2	Programs Do Not Stop Automatically	17
2.2	Comments	18
2.2.1	Tips for Writing Good Comments	18
2.3	The CON Section	19
2.3.1	Clock Frequency	19
2.3.2	Baud Rate	20
2.3.3	Baud Mode	20
2.4	Transmission Pin	21
2.4.1	Example	22
2.5	The DAT Section	22
2.5.1	Origin (ORG 0)	22
2.6	Real Clock (asmclk)	23
2.6.1	Example Timing	23
2.7	Disabling a SmartPin (DIRL)	23
2.7.1	Syntax	24
2.7.2	Example	24
2.7.3	Why It Is Used	24
2.8	Configuring a SmartPin (WRPIN)	25
2.8.1	Syntax	25
2.8.2	Example	26
2.9	SmartPin Speed (WXPIN)	26
2.9.1	Syntax	26
2.9.2	Example	27
2.10	Setting the Output (DIRH)	27
2.10.1	Conceptual Model	27

2.10.2	Syntax	28
2.10.3	Example	28
2.11	Sending Data (WYPIN)	28
2.11.1	Setup vs Execution	28
2.11.2	What wypin Does	28
2.11.3	Conceptual Model	29
2.11.4	Requirement	29
2.11.5	Example	29
2.12	Full Example: Serial Transmission Program	30
2.12.1	Clock and Baud Rate Setup	30
2.12.2	Transmission Pin Setup	30
2.12.3	Sending the Character	31
2.12.4	Program End	31
2.12.5	Summary	31
2.13	SmartPin Serial Setup Sequence	32
2.14	Program Structure Template	32
2.15	How It Works	32
2.15.1	Execution Starts at org 0	32
2.15.2	No Operating System	32
2.15.3	You Control Execution	33
2.15.4	Infinite Loops Are Required	33
2.16	Typical Program Flow	33
2.17	Exercise	33
2.18	Summary	34
3	Data Movement	35
3.1	What is a Variable	35
3.2	Declaring a Variable	35
3.3	Storing a Value in a Variable	36
3.4	Using a Variable	36
3.5	The MOV Instruction	36
3.5.1	Syntax	36
3.5.2	Example	36
3.6	Conceptual Model	37
3.7	Register-to-Register Transfer	37
3.8	Visualizing MOV	37
3.9	Hardware Output	37
3.10	Program: <code>print_character.spin2</code>	38
3.11	Exercise	38
3.12	Summary	39
4	Adding, Subtracting, and Multiplying	40
4.1	Why Math Matters	40
4.2	The ADD Instruction	40
4.2.1	Syntax	41

4.2.2	Example	41
4.2.3	ASCII and Number Display	41
4.2.4	Example Conversion	41
4.2.5	Important Limitation	42
4.2.6	Program: 0003_add.spin2	43
4.3	The SUB Instruction	44
4.3.1	Syntax	44
4.3.2	Examples	44
4.3.3	ASCII Conversion After Subtraction	44
4.3.4	Important Limitation	45
4.3.5	Program: 0004_sub.spin2	46
4.4	The MUL Instruction	47
4.4.1	Syntax	47
4.4.2	Examples	47
4.4.3	Program: 0005_mul.spin2	48
4.5	Summary	49
5	Bitwise Operations	50
5.1	The AND Instruction	50
5.1.1	Syntax	50
5.1.2	Program: 0006_and.spin2	52
5.2	The OR Instruction	52
5.2.1	Syntax	53
5.2.2	Program: 0007_or.spin2	54
5.3	The XOR Instruction	54
5.3.1	Syntax	55
5.3.2	Program: 0008_xor.spin2	56
5.4	The NOT Instruction	56
5.4.1	Syntax	57
5.4.2	Program: 0009_not.spin2	58
5.5	Exercise	58
5.6	Summary	59
6	Shifting	60
6.1	SHL — Shift Left	60
6.1.1	Syntax	60
6.1.2	Program: 0010_shl	61
6.2	SHR — Shift Right	61
6.2.1	Syntax	62
6.2.2	Program: 0011_shr.spin2	63
6.3	SAL — Shift Arithmetic Left	63
6.3.1	Syntax	64
6.3.2	Program: 0012_sal.spin2	65
6.4	SAR — Shift Arithmetic Right	65
6.4.1	Syntax	66

6.4.2	Program: 0013_sar.spin2	67
6.5	Exercise	67
6.6	Summary	68
7	Loops	69
7.1	JMP — Jump (Unconditional)	69
7.1.1	Syntax	69
7.1.2	Program: 0014_jmp.spin2	71
7.2	DJNZ — Decrement and Jump if Not Zero	72
7.2.1	Syntax	72
7.2.2	Program: 0015_djnz.spin2	73
7.3	TJNZ — Test and Jump if Not Zero	74
7.3.1	Syntax	74
7.3.2	Program: 0016_tjnz.spins2	75
7.4	TJZ — Test and Jump if Zero	76
7.4.1	Syntax	76
7.4.2	Program: 0017_tjz.spin2	77
7.5	Summary	78
8	Decision Making (Conditional Execution)	79
8.1	Decision-Making Overview	80
8.2	CMP — Compare	80
8.2.1	Syntax	80
8.3	IF_Z	81
8.3.1	Syntax	81
8.3.2	Program: 0018_if_z.spin2	82
8.4	IF_NZ	82
8.4.1	Syntax	83
8.4.2	Program: 0019_if_nz.spin2	84
8.5	CMP + IF_Z / IF_NZ	85
8.5.1	Syntax	85
8.5.2	Program: 0020_if_z_if_nz.spin2	86
8.6	CMP + IF_C	87
8.6.1	Syntax	87
8.6.2	Program: 0021_if_c.spin2	88
8.7	CMP + IF_NC	89
8.7.1	Syntax	89
8.7.2	Program: 0022_if_nc.spin2	90
8.8	CMP + IF_C / IF_NC	91
8.8.1	Syntax	91
8.8.2	Program: 0023_if_c_if_nc.spin2	92
8.9	CMP + IF_BE	93
8.9.1	Syntax	93
8.9.2	Program: 0024_if_be.spin2	94
8.10	CMP + IF_A	95

8.10.1	Syntax	95
8.10.2	Program: 0025_if_a.spin2	96
8.11	Exercise	97
8.12	Summary	97
9	Subroutines	98
9.1	JMP — Jump (Revisited)	98
9.1.1	Syntax	98
9.1.2	Program: 0026_jump.spin2	99
9.2	CALL	100
9.2.1	Syntax	100
9.2.2	Program: 0027_call.spin2	101
9.3	Global Labels and Local Labels	102
9.3.1	Syntax	102
9.3.2	Program: 0028_label.spin2	103
9.4	Exercise	104
9.5	Summary	104
10	Timing & Waits	106
10.1	WAITX Instruction	106
10.1.1	Syntax	107
10.1.2	Program: 0029_waitx.spin2	108
10.2	Instruction: NOP (No Operation)	108
10.2.1	Syntax	109
10.2.2	Program: 0030_nop.spin2	110
10.3	GETCT / WAITCT1 / ADDCT1 Instruction	110
10.3.1	Syntax	111
10.3.2	Program: 0031_waitct1.spin2	112
10.4	POLLCT1 Instruction (Non-blocking Timer Poll)	113
10.4.1	Syntax	113
10.4.2	Program: 0032_pollct11.spin2	114
10.5	WAITX vs NOP	115
10.5.1	Program: 0033_delay.spin2	116
10.6	Exercise	117
10.7	Summary	117
11	Hardware Functions	118
11.1	QMUL — Unsigned Multiplication Using CORDIC	118
11.1.1	Syntax	118
11.1.2	Program: 0034_qmul.spin2	120
11.2	QDIV — Unsigned Division Using CORDIC	121
11.2.1	Syntax	121
11.2.2	Program: 0035_qdiv.spin2	122
11.3	Exercise	123
11.4	Summary	123

12 Hub Memory Data Access	124
12.1 Variables in P2ASM	124
12.1.1 Example	124
12.1.2 Program: 0036_variables.spin2	125
12.2 RDBYTE	126
12.2.1 Syntax	126
12.2.2 Program: 0037_rdbyte.spin2	127
12.3 WRBYTE	127
12.3.1 Syntax	128
12.3.2 Program: 0038_wrbyte.spin2	129
12.4 RDBYTE Looping	130
12.4.1 Program: 0039_multi_rdbyte.spin2	131
12.5 RDBYTE for Strings	132
12.5.1 Program: 0040_rdbyte_strings.spin2	133
12.6 RDWORD	134
12.6.1 Syntax	134
12.6.2 Program: 0041_rdword.spin2	135
12.7 WRWORD	135
12.7.1 Syntax	136
12.7.2 Program: 0042_wrword.spin2	137
12.8 RDLONG	138
12.8.1 Syntax	138
12.8.2 Program: 0043_rdlong.spin2	139
12.9 WRLONG	139
12.9.1 Syntax	140
12.9.2 Program: 0044_wrlong.spin2	141
12.10 Exercise	142
12.11 Summary	142
13 How Numbers Work in P2 Assembly	143
13.1 Binary Representation	143
13.2 Signed Numbers and Two's Complement	143
13.3 Arithmetic Instructions	143
13.4 Memory Alignment	144
13.5 Printing Numbers in P2 Assembly	144
13.5.1 Program: 0045_print_number_sub.spin2	145
13.6 Printing Numbers Using QDIV.spin2	146
13.6.1 Program: 0046_print_number_qdiv.spin2	147
13.7 Exercises	148
13.8 Summary	148
14 Cog Control	150
14.1 What is a cog?	150
14.1.1 Syntax	150
14.1.2 Program: 0047_cog_control.spin2	152

14.1.3	Program: 0048_cog_addition.spin2	154
14.1.4	Program: 0049_cog_multicog.spin2	156
14.1.5	Program: 0050_cog_memoryflip.spin2	158
14.2	Cog Process Control	159
14.2.1	Syntax	159
14.2.2	Program: 0051_cog_lock.spin2	160
14.3	Exercise	161
14.4	Summary	162
15	String Manipulation	163
15.1	Print a String	163
15.2	String Length	163
15.2.1	Program: 0052_string_length.spin2	164
15.3	String Number Length	165
15.3.1	Program: 0053_string_number_length.spin2	166
15.4	String Compare	167
15.4.1	Program: 0054_string_compare.spin2	168
15.5	String Number Compare	169
15.5.1	Program: 0055_string_number_compare.spin2	170
15.6	String Copy	171
15.6.1	Program: 0056_string_copy.spin2	173
15.7	String Number Copy	174
15.7.1	Program: 0057_string_number_copy.spin2	175
15.8	String Concatenate	177
15.8.1	Program: 0058_string_concatenate.spin2	178
15.9	String Number Concatenate	180
15.9.1	Program: 0059_string_number_concatenate.spin2	181
15.10	String to Decimal	183
15.10.1	Program: 0060_string_to_decimal.spin2	184
15.11	Decimal to Character	185
15.11.1	Program: 0061_decimal_to_character.spin2	186
15.12	Decimal to String	187
15.12.1	Program: 0062_decimal_to_string.spin2	188
15.13	Decimal to string revisited	190
15.13.1	Program: 0063_decimal_to_string2.spin2	191
15.14	Exercise	193
15.15	Summary	193
16	SmartPin for UART)	195
16.1	About SmartPins	195
16.1.1	Syntax	195
16.2	Basic UART Echo with SmartPins	196
16.2.1	Configuring a SmartPin for Reception	197
16.2.2	Syntax	197
16.2.3	Program: 0064_echo.spin2	199

16.3	Single Digit Input and Check (0–9)	200
16.3.1	Program: 0065_get_digit.spin2	201
16.4	Multi-character input	203
16.4.1	Program: 0066_input_match.spin2	204
16.5	Line Editing with CRLF	206
16.5.1	Program: 0067_line_edit.spin2	207
16.6	Read Line	209
16.6.1	Program: 0068_read_line.spin2	210
16.7	Send String	212
16.7.1	Program: 0069_send_string.spin2	213
16.8	Tokenized Echo Command	216
16.8.1	Program: 0070_token_echo.spin2	217
16.9	Exercise	221
16.10	Summary	221
17	Introduction to SPI (Serial Peripheral Interface)	222
17.1	What is SPI?	222
17.1.1	Example	223
17.2	Reading the JEDEC ID from SPI Flash	224
17.2.1	Program: 0071_spi_display_jedec.spin2	225
17.3	Reading from SPI Flash	228
17.3.1	Program: 0072_spi_read_byte.spin2	230
17.4	Writing to SPI Flash	233
17.4.1	Program: 0073_spi_write_byte.spin2	234
17.5	Exercise	238
17.6	Summary	238
18	Practice Programs	239
A	Integrating P2ASM with High-Level Programs	241
A.1	Inline P2ASM	241
A.2	Example: Adding a Value	242
A.3	Example: Fast Multiplication by Shifting	243
A.4	Example: Processing Several Values	244
A.5	Example: Direct Pin Control	245
A.6	Using Local Variables	246
A.7	Persistent Values	247
A.8	Inline P2ASM vs. a Separate Cog	248
A.9	A Practical Design Pattern	249
A.10	Start Small	250
B	Propeller 2 Assembly Reference	252
B.1	Hub / Cog Setup & Program Control	252
B.2	Pin / SmartPin Instructions	253
B.3	Arithmetic Instructions	253

B.4	Bitwise / Shift Instructions	253
B.5	Flow Control	254
B.6	Compare & Conditional Execution	254
B.7	Timing / Counter Instructions	254
B.8	Data Movement / Memory Access	254

Chapter 1

Getting Started with the Propeller 2

We are beginning our journey with the Propeller P2 micro-controller. Unlike many micro-controllers, which usually have a single processor, the P2 is a multi core micro-controller. This means it contains 8 identical 32-bit processors, called “cogs,” that can all run code at the same time.

Having multiple processors makes the P2 very flexible and powerful because each processor can handle a different task independently. For example, one processor could control an LED display while another reads input from a sensor, all without slowing each other down.

All 8 processors are connected through a central hub, which acts as the brain that coordinates access to shared resources. The hub contains shared RAM that all processors can read from and write to, enabling fast data exchange between cogs. There is a built-in hardware math engine that accelerates complex operations such as trigonometry, vector math, and graphics calculations. The hub also contains housekeeping hardware that manages system timing, synchronization, and inter-processor communication to keep all cogs operating smoothly.

One of the most unique features of the P2 is its 64 smart Input/Output(I/O) pins. These pins are much more than simple digital inputs or outputs. Each pin can be programmed to perform complex analog or digital functions independently. The pins can generate waveforms, count pulses, communicating with devices, or even acting as a small ADC (analog-to-digital converter).

The P2 is designed so every processor can access every I/O pin, giving us maximum flexibility. The pins are powered at 3.3 volts in groups of four, which improves their analog performance.

Overall, the Propeller P2 is a versatile micro-controller capable of handling multiple tasks simultaneously, performing complex calculations efficiently, and controlling a wide range of input/output devices. For beginners, it’s exciting because it opens the door to experimenting with advanced electronics while still being approachable.

The Propeller 2 was developed by Parallax, Inc., a company known for creating innovative educational and hobbyist micro-controllers. Its design builds upon the success of the original Propeller chip, offering more memory, faster processors, and more flexible I/O capabilities. The P2 was first publicly announced around 2020, with development starting several years earlier to address the needs of modern embedded electronics.

1.1 What is P2ASM

As we continue our journey, we come across the question: how do we tell the P2 what to do? This is where P2ASM comes in.

P2ASM stands for Propeller 2 Assembler, and it is the programming language that speaks directly to the P2's processors, or “cogs.” Unlike high-level languages like C, Basic or Python, which need to be translated into machine instructions, P2ASM lets us write instructions that are very close to the hardware. Think of it like giving the P2 step-by-step directions in a language it naturally understands.

P2ASM allows us to control each processor individually, directly manage the smart I/O pins, and write highly efficient code with minimal overhead. This makes P2ASM ideal for tasks that require precise timing, low latency, and maximum performance on the Propeller 2.

P2ASM gives full control of the Propeller 2 hardware, letting programs run much faster than high-level languages and typically occupy less memory. Its precise timing makes it ideal for time-critical tasks, advanced I/O, and projects involving multiple processors. With its clear and structured commands, beginners can start with simple tasks and quickly progress to more complex projects.

This text is a practical guide to programming the Propeller 2 using P2ASM. It focuses on teaching full control of the hardware, writing efficient code, and achieving precise timing. It is designed to help you start with simple tasks and gradually progress to more complex projects, including multiple processors, smart I/O, and graphics.

This book is not a complete reference for every Propeller 2 instruction or hardware feature. It does not focus on high-level languages, but on low-level programming with P2ASM. It is also not intended to overwhelm beginners; concepts are introduced step by step with clear explanations and practical examples.

P2ASM (Propeller 2 Assembly Language) is included as part of the official Parallax Propeller 2 software tools. It is not a separate download. To get P2ASM software tools

1. Visit the Parallax Inc. website
2. Navigate to the Propeller 2 section
3. Download the current official development tools provided by Parallax (these include the assembler, compiler, and supporting utilities)
4. You can also download third-party development tools.

Tool setup steps and filenames may change over time. Always follow the instructions provided by Parallax or third-parties for installation and updates

1.2 How P2ASM Actually Works

Before writing instructions, it's important to understand what is happening when your code runs on the Propeller 2. P2ASM is very close to the hardware, which means you are not just writing commands— you are controlling how the processor behaves step by step.

1.2.1 Cog vs Hub (Execution and Memory)

Each of the 8 cogs is an independent processor with its own small, fast local memory. This is where instructions are executed. The hub is shared memory that all cogs can access. It is larger, but slower to access than cog memory.

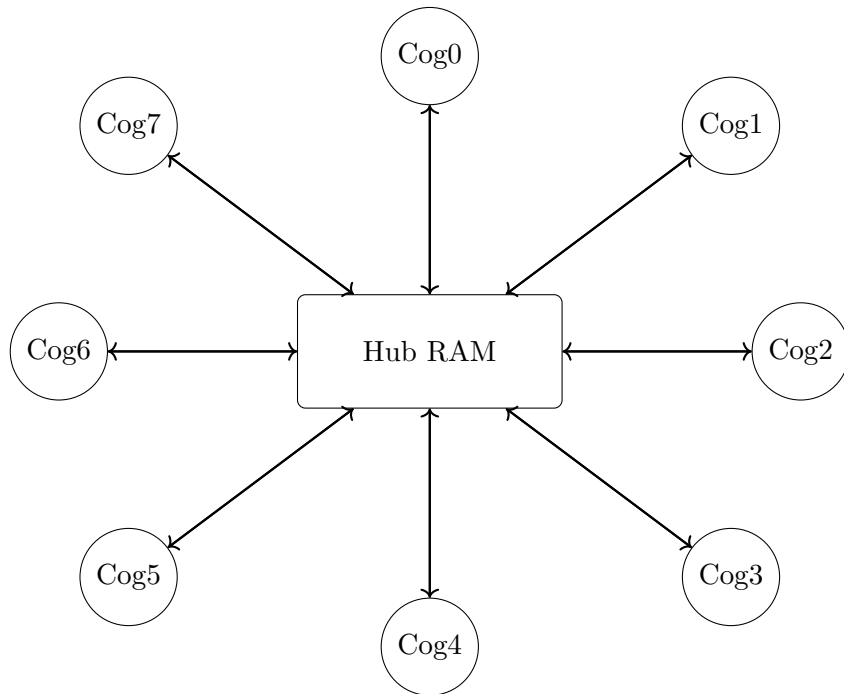


Figure 1.1: Propeller 2 Hub RAM shared memory architecture across 8 Cogs

This creates an important timing difference:

1. Cog memory (local) → fast, used for instructions and immediate data
2. Hub memory (shared) → slower, used for communication between cogs

Because of this, the P2 can run multiple tasks at once, but accessing shared data (hub) may take longer than working within a cog's local memory

1.2.2 Registers vs Memory

When your code runs, it primarily works with registers inside a cog.

- Registers are small, fast storage locations used during execution
- They hold values temporarily while instructions operate on them

Memory (hub or cog RAM) stores data more permanently, but registers are where most operations happen.

A typical flow looks like this:

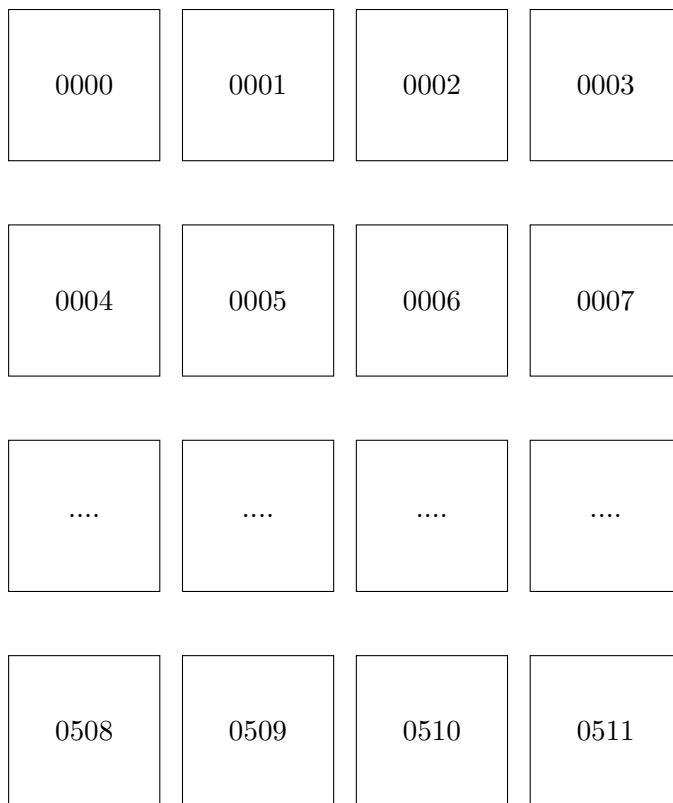


Figure 1.2: Memory layout showing sequential address blocks

1. Load a value from memory into a register
2. Perform operations on the register
3. Store the result back into memory (if needed)

This is why instructions like `mov`, `add`, and `cmp` work directly with registers—they are the working space of the processor.

1.2.3 Instruction Execution Flow

P2ASM runs instructions in sequence, one at a time, inside a cog. The general flow is:

1. Fetch the next instruction
2. Decode it
3. Execute it
4. Move to the next instruction

Unless you explicitly change the flow (with jumps or conditional execution), the program simply runs line by line. This is very different from high-level languages, where many details are hidden. In P2ASM, you are directly controlling execution

Having established an introduction to the Propeller 2 and its development environment, we now proceed from system setup into the fundamental mechanisms by which data is introduced and prepared for processing within the architecture.

Chapter Summary

- The Propeller 2 has 8 independent processors called cogs
- All cogs share memory through the hub
- Cog memory is fast; hub memory is shared but slower
- Registers are where instructions operate directly
- Instructions execute sequentially unless redirected

Chapter 2

Getting started

2.1 How the P2 Executes Your Code

Before working with pins, timing, or serial communication, it's important to understand what the Propeller 2 is actually doing when your program runs.

Each cog executes instructions one at a time, in order, from its local memory. There is no operating system, no background scheduler, and nothing running behind the scenes. The cog simply follows instructions step by step.

Execution Process

1. The cog starts at address 0 in its memory (set by `org 0`)
2. It reads the first instruction
3. It executes that instruction
4. It moves to the next instruction
5. This continues one instruction at a time

At any moment, the cog is only doing one thing: executing the current instruction.

Example

```
mov r0, #5
add r0, #1
mov r1, r0
```

What Happens Internally

- First instruction: r0 becomes 5

- Second instruction: r0 becomes 6
- Third instruction: the value in r0 is copied into r1

There are no hidden steps. Each instruction directly changes the state of the processor.

2.1.1 The Program Counter

Inside the cog is a special register called the program counter.

It keeps track of which instruction is currently being executed.

You can think of it like a finger reading a list:

- It points to the current instruction
- After each instruction, it moves to the next one
- If a jump instruction is used, it moves to a different location instead

2.1.2 Programs Do Not Stop Automatically

The Propeller 2 does not automatically stop your program.

If execution reaches the end of your code, the cog will continue into whatever memory comes next, which can lead to unpredictable behavior.

Because of this, most programs end with an infinite loop:

<code>jmp #</code>

This instruction tells the cog to jump to its current location and remain there forever.

Execution Model

You can think of a cog as a simple machine that:

- reads one instruction
- executes it
- moves to the next
- repeats forever

Everything you write in P2ASM directly controls this process.

Now that we understand how instructions execute, we can begin building a real program.

2.2 Comments

Comments are notes written in your code that the assembler ignores when generating machine code.

They are meant for humans, helping explain:

- what the code does
- why certain instructions are used
- how different sections work

Using comments effectively makes your code easier to understand, maintain, and debug.

Syntax

```
' This is a comment
```

In P2ASM, comments begin with a single quote ('). Everything after it on that line is ignored by the assembler.

Example

```
mov r0, #5      ' Load 5 into r0
add r0, #1      ' Increment r0
```

Comments allow you to document your logic, hardware connections, and intent without affecting execution, turning your code into a clear and readable guide instead of just a list of instructions.

2.2.1 Tips for Writing Good Comments

- Keep comments short and clear, focusing on **why** the code does something rather than just what it does.
- Always update comments if the code changes.
- Outdated comments can be misleading.
- For complex calculations or bitwise operations, include step-by-step explanations.
- Good comments make code easier to understand and maintain over time.

Example

```

mov A, #5          ' Put the number 5 into register A
add B, A           ' Add the value in A to register B
wypin A, #62       ' Send the value in A to I/O pin 62

' This section initializes the TX pin for serial output
wrpin ##(P_ASYNC_TX | P_OE), #TRANSMISSION_PIN

mov number1, #2    ' Assign 2 to number1

```

2.3 The CON Section

The CON section in P2ASM is used to define constants—values that do not change while your program is running.

Constants make your code easier to read and maintain by allowing you to assign meaningful names to important values such as:

- `_CLKFREQ` — system clock frequency
- `BAUD_RATE` — serial communication speed
- `TRANSMISSION_PIN` — I/O pin number

Unlike variables, constants are fixed and defined before the program begins execution.

Using descriptive names instead of raw numbers improves readability and makes it easier to update values in one place without modifying multiple lines of code.

Example

```

con
_CLKFREQ = 180_000_000    ' System clock in Hz
BAUD_RATE = 2_000_000     ' Serial speed in bits per second
TRANSMISSION_PIN = 62     ' Pin used for transmitting serial data

```

2.3.1 Clock Frequency

Before communicating with other devices, we need a reliable sense of time.

The system clock (`_CLKFREQ`) acts like the heartbeat of the Propeller 2 chip. It determines when instructions execute, when pins toggle, and how time intervals are measured.

This internal clock drives instruction execution, SmartPin timing, and all time-sensitive hardware, ensuring that every operation occurs with precise and predictable timing.

Example

```
_CLKFREQ = 180_000_000
```

At a system clock frequency of 180 MHz:

- Each clock cycle is approximately 5.56 ns
- Most instructions execute in 2 cycles (11.11 ns per instruction)

This extremely fast timing allows precise control over hardware and enables high-speed real-time operation.

2.3.2 Baud Rate

The baud rate determines how fast data is transmitted over a serial connection and is measured in bits per second (bps).

Choosing the correct baud rate is essential for reliable communication. If the transmitter and receiver are not configured to the same baud rate, data may be corrupted or misinterpreted.

Example

```
BAUD_RATE = 2_000_000
```

This sets the serial communication speed to 2,000,000 bits per second (2 Mbps).

2.3.3 Baud Mode

When sending serial data with a SmartPin, the Propeller 2 requires two key pieces of information:

- How long each bit should last
- How many bits are sent in each frame

These are combined into a single configuration value called `BAUD_MODE`, which defines both timing and frame size.

Bit Timing

First, we calculate how long each bit lasts by dividing the system clock frequency by the desired baud rate:

```
BIT_PERIOD = (_CLKFREQ / BAUD_RATE)
```

For example:

- $180,000,000 \div 2,000,000 = 90$

This uses integer division, so the result is a whole number.

Each bit therefore lasts 90 clock cycles. You can think of this as a timer: every 90 clock cycles, the SmartPin moves to the next bit.

Frame Size

Next, we define the frame size.

A typical serial frame uses 8 bits, but the SmartPin expects this value as:

- frame size (frame size - 1)

So:

$$8 - 1 = 7$$

Combining the Values

Finally, both values are combined into a single configuration word:

$$\text{BAUD_MODE} = (\text{BIT_PERIOD} \ll 16) \mid (8 - 1)$$

This creates a 32-bit value where:

- The upper 16 bits store the bit timing
- The lower bits store the frame size

You can think of `BAUD_MODE` as a complete SmartPin configuration:

- It defines how fast bits are transmitted
- It defines how many bits are sent per frame

This value is then written to the SmartPin using `wxpin`, which applies the timing configuration to the hardware.

2.4 Transmission Pin

The transmission pin is the Propeller 2 I/O pin that physically sends data.

For this example, we use pin 62.

This pin must be properly configured using `wrpin`, `wxpin`, and `dirh` so it knows its role, timing, and direction. This ensures that data is transmitted correctly.

2.4.1 Example

```
TRANSMISSION_PIN = 62
```

2.5 The DAT Section

The DAT section contains the actual program instructions that the Propeller 2 executes.

Anything that runs—such as configuring pins, sending data, or controlling hardware—goes here.

You can think of DAT as the engine of your program, providing the step-by-step instructions the Propeller 2 follows.

Constants defined in the CON section are often used in DAT to make code cleaner and easier to read.

Without the DAT section, the Propeller 2 has no instructions to execute.

Example

```
dat
org 0

asmclk

dir1 #TRANSMISSION_PIN
wrpin ##(P_ASYNC_TX | P_OE), #TRANSMISSION_PIN
wxpin ##BAUD_MODE, #TRANSMISSION_PIN
dirh #TRANSMISSION_PIN

wypin #1, #TRANSMISSION_PIN
wypin #"0", #TRANSMISSION_PIN
```

2.5.1 Origin (ORG 0)

In Propeller 2 assembly (P2ASM), **ORG 0** tells the assembler that the program should begin at address 0 in a cog's local memory. Each cog in the Propeller 2 has its own 512-long (2 KB) memory space. When a cog starts executing, it begins at location 0 by default. The **ORG** directive defines where instructions are placed in cog RAM. By writing **ORG 0**, you ensure that your first instruction is assembled exactly where execution begins. Without this, code could be placed at a different memory location, which may cause incorrect execution if the program counter does not align with the instruction layout. In short, **ORG 0** aligns your program's starting point with the cog's execution entry point.

2.6 Real Clock (asmclk)

While `_CLKFREQ` defines how fast the system runs, `asmclk` tells the assembler to use the actual system clock frequency for timing calculations. This affects instructions such as delays and SmartPin configuration, where precise timing is critical. Without `asmclk`, timing-based instructions may not match the real hardware clock exactly.

By enabling it, you ensure that:

- Delays are accurate
- Pulse widths are precise
- Signal timing matches real hardware behavior

2.6.1 Example Timing

At a system clock frequency of 180 MHz:

- One clock cycle = 5.56 ns
- Most instructions execute in 2 cycles (11.11 ns per instruction)

`asmclk` ensures that these real timing values are used when the assembler calculates delays and SmartPin timing.

Use Cases

This is essential for:

- Serial communication
- PWM generation
- Video output timing
- SmartPin bit timing

2.7 Disabling a SmartPin (DIRL)

The `dir1` instruction turns off a pin's output driver, placing it into a high-impedance (Hi-Z) state.

In this state, the pin no longer drives any signal and is effectively disconnected from the circuit.

This creates a safe, inactive condition that is useful before reconfiguring a SmartPin.

When `dir1` is executed:

- The output driver is disabled

- The pin stops actively driving high or low signals
- The pin enters a high-impedance (Hi-Z) state

This ensures that no unintended or partial signals are transmitted during setup.

2.7.1 Syntax

```
dir1 #pin
```

Parameters

pin — the I/O pin to set as input (low direction)

- Specifies the physical pin number to modify
- Example: #TRANSMISSION_PIN
- This pin is configured as an input by clearing its direction bit

2.7.2 Example

```
dir1 #TRANSMISSION_PIN
```

2.7.3 Why It Is Used

In real-world applications, pins are often reused for different modes such as:

- Transmit (TX)
- Receive (RX)
- PWM output

Disabling the pin first prevents:

- Signal glitches
- Electrical noise
- Incorrect or partial outputs during reconfiguration

`dir1` ensures the pin is safely disabled before being configured for a new function.

2.8 Configuring a SmartPin (WRPIN)

The `wrpin` instruction sets the role of a Propeller 2 pin.

It determines whether the pin will:

- transmit signals
- receive signals
- or remain idle

By defining the pin's behavior, `wrpin` prepares it for further configuration such as timing or direction.

Without `wrpin`, the pin has no defined function and cannot operate correctly.

`wrpin` can configure special behaviors such as:

- Serial transmission (TX)
- Serial reception (RX)
- Timing-based output such as PWM

It is the first step in setting up any SmartPin operation.

Think of a pin as a worker:

- `P_ASYNC_TX` tells the worker to send data to the outside world
- `P_OE` gives permission to drive the output line
- `wrpin` assigns the job to the worker

This is why `wrpin` is always used before timing or output configuration.

2.8.1 Syntax

```
wrpin ##mode, pin
```

Parameters

mode — the Smart Pin configuration value

- Defines the operating mode of the pin (e.g., input, output, serial transmit, serial receive)
- Encodes settings such as direction, logic behavior, and Smart Pin function
- Often built using predefined constants (eg' `P_ASYNC_TX`, `P_ASYNC_RX`)

pin — the I/O pin to configure

- Specifies which physical pin receives the configuration
- Example: `#TRANSMISSION_PIN`

2.8.2 Example

```
wrpin ##(P_ASYNC_TX | P_OE), #TRANSMISSION_PIN
```

- `P_ASYNC_TX` — configures the pin for UART-style serial transmission
- `P_OE` — enables output driver control
- `|` — combines multiple configuration flags into one value
- `#TRANSMISSION_PIN` — selects the physical I/O pin

2.9 SmartPin Speed (WXPIN)

The `wxpin` instruction sets how fast a SmartPin sends or receives signals by defining its timing.

This timing control is essential for communication protocols such as UART, where precise bit timing determines whether data is transmitted correctly.

Without `wxpin`, the pin has no defined signal speed, which can result in garbled data or communication errors.

Think of a SmartPin like a telegraph operator sending dots and dashes.

`wxpin` determines how long each dot or dash lasts—it controls timing, not the data itself. Accurate timing is critical because:

- Both devices must agree on signal speed
- Incorrect timing leads to corrupted data
- Stable communication depends on precise bit duration

`wxpin` is typically used after `wrpin`, because:

- `wrpin` assigns the pin's role (TX, RX, PWM)
- `wxpin` defines how that role operates over time

`wxpin` defines the timing behavior of a SmartPin, ensuring that data is transmitted at the correct speed for reliable communication.

2.9.1 Syntax

```
wxpin ##value, pin
```

Parameters

value — the timing configuration for the pin

- Usually calculated from the system clock (`_CLKFREQ`) and baud rate (`BAUD_RATE`)
- In serial communication, this value is typically `BAUD_MODE`, which includes both timing and frame size
- It defines how long each bit is held during transmission

pin — the I/O pin to apply the timing to

- Example: `#TRANSMISSION_PIN`
- This selects the physical pin used for serial output

2.9.2 Example

```
wxpin ##BAUD_MODE, #TRANSMISSION_PIN
```

- `BAUD_MODE` — combined timing and frame configuration value
- `#TRANSMISSION_PIN` — the physical I/O pin being configured

2.10 Setting the Output (DIRH)

The `dirh` instruction tells the Propeller 2 that a specific pin should function as an output. This allows the pin to drive a high or low voltage to the outside world.

Without `dirh`, the pin remains in a high-impedance (input) state and cannot drive the signal line.

Even if `wrpin` and `wxpin` are configured, no data will be transmitted until `dirh` enables the output driver.

In short, `dirh` enables the pin's output driver, allowing it to actively drive the line high or low. This makes the pin a signal source rather than a listener.

2.10.1 Conceptual Model

Think of each pin as a tiny faucet:

- `dirh` is turning the handle to let water flow out
- Without it, the faucet stays closed and the pin can only listen

`dirh` controls direction, not timing or data.

It is required whenever a pin must drive something such as:

- UART TX lines
- LEDs
- SmartPin-based outputs

Even with correct configuration and timing, no signal will be output unless `dirh` is set.

2.10.2 Syntax

```
dirh #pin
```

Parameters

pin — the I/O pin to configure as an output

- Example: `#TRANSMISSION_PIN`
- This selects the physical pin that will drive signals

2.10.3 Example

```
dirh #TRANSMISSION_PIN
```

2.11 Sending Data (WYPIN)

The `wypin` instruction is what actually sends data or a signal through a SmartPin.

It is the execution step in the SmartPin configuration process.

2.11.1 Setup vs Execution

Think of SmartPin configuration in two phases:

Setup phase:

- `wrpin` assigns the pin's role
- `wxpin` sets timing and parameters
- `dirh` enables the output driver

Execution phase:

- `wypin` sends the actual data

None of the setup instructions produce output on their own—they only prepare the pin.

2.11.2 What `wypin` Does

When `wypin` is executed, the Propeller 2:

- takes the provided value
- passes it to the SmartPin hardware
- triggers the configured behavior (TX, PWM, SPI, etc.)

This may result in:

- transmitting a byte (serial communication)
- toggling a signal line
- generating a waveform

2.11.3 Conceptual Model

Think of the SmartPin as a worker:

- `wrpin`, `wxpin`, and `dirh` prepare the worker
- `wypin` hands the worker the actual data to process

This is the moment the operation actually happens.

2.11.4 Requirement

`wypin` must always come after the SmartPin has been fully configured.

Without proper setup, no meaningful output will occur.

Syntax

```
wypin #value, pin
```

Parameters

value — the data or signal to send

- Can be a byte, character, or control value
- Example: `"A"` or `#1`

pin — the I/O pin used for transmission

- Example: `#TRANSMISSION_PIN`

2.11.5 Example

```
wypin "H", #TRANSMISSION_PIN
```

2.12 Full Example: Serial Transmission Program

Listing 2.1: Propeller 2 Serial Transmission Example

```
'tx.spin2
CON
    _CLKFREQ          = 180_000_000
    BAUD_RATE         = 2_000_000
    TRANSMISSION_PIN  = 62
    BIT_PERIOD        = (_CLKFREQ / BAUD_RATE)
    BAUD_MODE         = (BIT_PERIOD << 16) | (8 - 1)

DAT
    org 0
    asmclk

    dirl    #TRANSMISSION_PIN
    wrpin   ##(P_ASYNC_TX | P_OE), #TRANSMISSION_PIN
    wxpin   ##BAUD_MODE, #TRANSMISSION_PIN
    dirh    #TRANSMISSION_PIN
    wypin   #1, #TRANSMISSION_PIN

    wypin   #"0", #TRANSMISSION_PIN

    endprog
    jmp    #
```

This is a simple example of sending a character from a Propeller 2 program to a terminal using a single pin.

The process demonstrates the full SmartPin setup and transmission sequence.

2.12.1 Clock and Baud Rate Setup

The program begins by defining timing parameters:

- System clock: `_CLKFREQ = 180,000,000`
- Serial speed: `BAUD_RATE = 2,000,000`

These values are used to calculate the bit period, which determines how long each bit is transmitted.

2.12.2 Transmission Pin Setup

Pin 62 is selected as the transmission pin:

```
TRANSMISSION_PIN = 62
```

The pin is then configured step by step:

- `dir1 #TRANSMISSION_PIN` — disables output (sets pin to input/Hi-Z)
- `wrpin ##(P_ASYNC_TX | P_OE), #TRANSMISSION_PIN` — sets asynchronous transmit mode and enables output capability
- `wxpin ##BAUD_MODE, #TRANSMISSION_PIN` — loads baud rate and timing configuration
- `dirh #TRANSMISSION_PIN` — enables the output driver
- `wypin #1, #TRANSMISSION_PIN` — sets the idle state (line high) before transmission

2.12.3 Sending the Character

The `#` syntax tells the assembler to use the ASCII value of a character.

For example:

```
wypin #"0", #TRANSMISSION_PIN
```

This sends the ASCII character `"0"` (decimal 48), not the numeric value 0. The SmartPin hardware automatically:

- formats the data into a serial frame
- applies correct timing
- generates the waveform on the output pin

As a result, the terminal displays the character `"0"`.

2.12.4 Program End

- `endprog` is a label used to mark the end of the program
- `jmp #` creates an infinite loop by jumping to itself

```
endprog  
jmp #
```

2.12.5 Summary

This example demonstrates the complete sequence required to send a character from the Propeller 2 to a terminal using a single SmartPin.

2.13 SmartPin Serial Setup Sequence

Serial transmission using a SmartPin must always follow this order:

- `dir1` — disable output (pin enters high-impedance/input state)
- `wrpin` — set the pin's role and function
- `wxpin` — configure timing (BAUD_MODE)
- `dirh` — enable output driver
- `wypin` — transmit data

This sequence is required for correct SmartPin operation.

2.14 Program Structure Template

Every P2ASM program follows a consistent structure:

```
CON

DAT

org 0
asmclk

' setup code

main_loop
' main program code

jmp #main_loop
```

2.15 How It Works

2.15.1 Execution Starts at `org 0`

The `org 0` directive tells the assembler that execution begins at address 0 in cog memory. When the cog starts, it immediately begins executing from this location.

2.15.2 No Operating System

The Propeller 2 executes your code directly. There is:

- No operating system

- No scheduler
- No `main()` function

2.15.3 You Control Execution

Because nothing manages program flow, you must explicitly define what happens next at all times.

2.15.4 Infinite Loops Are Required

Most programs end with a loop such as:

```
jmp #main_loop
```

Without this, execution may continue into unintended memory, causing unpredictable behavior.

2.16 Typical Program Flow

1. Initialization (configure pins, registers, peripherals)
2. Enter main loop
3. Repeated execution of tasks

A P2ASM program can be thought of as:

Start at address 0, initialize hardware, then run a loop forever.

Once this structure is understood, all P2ASM programs become significantly easier to read and write.

2.17 Exercise

Use `wypin` to send the characters "A", "B", and "C" and observe the output on a serial monitor. Write a program that transmits the following characters in sequence:

- "A"
- "B"
- "C"

Observe how each character is transmitted as its ASCII value over the serial connection. Try modifying the program by:

- Changing the characters being sent
- Sending numbers such as "0" through "9"
- Experimenting with different ASCII characters

Observe how the output changes on the serial monitor each time.

2.18 Summary

With the foundational environment in place, we now examine how data is transferred between registers, hub memory, and execution units, forming the basis for all subsequent computation.

Chapter Summary

- Programs start at `org 0` and run sequentially
- The program counter tracks the current instruction
- Programs do not stop automatically (must use loops)
- The `CON` section defines constants for readability
- SmartPins require setup: `wrpin` → `wxpin` → `dirh` → `wypin`
- Timing depends on `CLKFREQ` and `asmclk`

Chapter 3

Data Movement

Moving data is essential because the Propeller 2 must store values and send them to hardware components.

Before the Propeller can perform any meaningful operation, values must be placed in the correct location. This process is called **data movement**.

You can think of it as transferring objects between boxes: the source holds the original value, and the destination receives it. Depending on the instruction, the source may keep its value or be overwritten.

This concept underpins almost every program on the Propeller 2.

3.1 What is a Variable

A variable is a named location in memory that stores a value.

You can think of it like a mailbox:

- It has a name for easy reference
- It stores a value in memory
- It can hold numbers, characters, or other data

Variables make it easier to organize and manipulate data during program execution.

3.2 Declaring a Variable

Variables are declared using the **res** directive.

```
mydata res 1
```

This reserves one memory location for **mydata**.

Initially, this memory may contain an undefined value, so it should be assigned before use.

3.3 Storing a Value in a Variable

The `mov` instruction stores values in variables.

```
mov mydata, #"A"
```

This stores the ASCII value of "A" (decimal 65) into `mydata`.

After execution, the variable holds the value and can be reused later.

3.4 Using a Variable

Once a variable contains a value, it can be used in instructions such as `wypin`.

```
wypin mydata, #TRANSMISSION_PIN
```

This sends the value stored in `mydata` through the specified pin.

Variables allow programs to be flexible instead of hardcoding values directly.

3.5 The MOV Instruction

The `mov` instruction is the most basic data movement operation.

It copies a value from a source into a destination.

3.5.1 Syntax

```
mov destination, {#}source
```

- **destination** — where the value is stored
- **source** — where the value comes from
- **#** — indicates a literal constant

If `#` is used, the value is treated as a constant. If omitted, the value is treated as a register or variable.

3.5.2 Example

```
mov mybox, #"A"
```

This places the letter "A" into `mybox`.

3.6 Conceptual Model

Think of memory as a row of mailboxes:

- Each mailbox stores a value
- `mov A, B` copies the value from B into A
- The source remains unchanged

3.7 Register-to-Register Transfer

```
mov B, A
```

This copies the value in A into B, leaving A unchanged.

3.8 Visualizing MOV

Instruction	Before	After
MOV A, #8	A = 0	A = 8
MOV B, A	B = 0	B = 8
MOV OUTA, #1	OUTA = 0	OUTA = 1

3.9 Hardware Output

Values can also be moved into hardware registers such as output pins.

By writing to output registers, the Propeller 2 can control external hardware such as LEDs, motors, and other devices.

This allows software to directly interact with the physical world.

3.10 Program: print_character.spin2

Listing 3.1: Program: print character

```
'print_character.spin2
CON
    _CLKFREQ          = 180_000_000
    BAUD_RATE         = 2_000_000
    TRANSMISSION_PIN  = 62
    BIT_PERIOD        = (_CLKFREQ / BAUD_RATE)
    BAUD_MODE         = (BIT_PERIOD << 16) | (8 - 1)

DAT
    org 0
    asmclk

    dirl    #TRANSMISSION_PIN
    wrpin   ##(P_ASYNC_TX | P_OE), #TRANSMISSION_PIN
    wxpin   ##BAUD_MODE, #TRANSMISSION_PIN
    dirh    #TRANSMISSION_PIN
    wypin   #1, #TRANSMISSION_PIN

    mov character, #"A"
    wypin character, #TRANSMISSION_PIN

    endprog
    jmp    $$

    character res 1
```

When this program runs, the letter "A" appears on the terminal screen.

This happens because the program first stores the ASCII value for "A" in the variable `character`, then sends that value through the transmission pin using `wypin`.

The Propeller 2 handles all timing and signal generation internally, so the character is transmitted correctly without requiring manual control of the electrical signal.

3.11 Exercise

Load the letter "B" into register A, then copy the value from A into register B. Verify that both registers contain the same value.

- Use `mov A, #"B"` to load the value
- Use `mov B, A` to copy the value
- Use `wypin` to observe the result

3.12 Summary

Building upon the principles of data movement, we now explore the arithmetic operations that define core computational capability within the Propeller 2 instruction set.

Chapter Summary

- Variables are stored in registers or memory
- `mov` copies data between registers or values
- Most operations happen in registers
- Data must be moved before it can be used
- Hardware output depends on register values

Chapter 4

Adding, Subtracting, and Multiplying

The Propeller 2 microcontroller can perform a wide range of mathematical operations, from simple addition and subtraction to more advanced calculations.

These operations are performed directly on registers or memory locations, giving you precise control over how values are manipulated, stored, and used in your program.

This low-level approach allows for efficient and predictable computation, which is especially useful in embedded systems programming.

4.1 Why Math Matters

Math is essential in almost every program. It is used to:

- Count loops
- Control timing
- Move objects on a screen
- Convert numbers for display or output

Even simple tasks like blinking an LED require math to calculate delays between on and off states.

Math allows programs to make decisions, measure time, and manipulate data meaningfully.

4.2 The ADD Instruction

The `add` instruction adds two values together.

It takes:

- a destination register
- a source value (register or literal)

The result is stored in the destination register.

4.2.1 Syntax

```
add destination, {#}source
```

- **destination** — where the result is stored
- **source** — value to add
- **#** — indicates a literal constant

4.2.2 Example

```
add A, #5
```

This adds 5 to the value already stored in register A.

```
add B, A
```

This adds the value in register A to register B.

4.2.3 ASCII and Number Display

Computers use ASCII codes to represent characters.

For example:

- '0' = 48
- '1' = 49
- '9' = 57

To display a number, it must be converted into its ASCII equivalent before sending it to a terminal.

4.2.4 Example Conversion

If register A contains the value 4:

```
add A, #48
```

Now A contains 52, which corresponds to the character '4' in ASCII.

When sent to a terminal, it displays as the character 4.

4.2.5 Important Limitation

This method works for single-digit numbers.

To display multi-digit numbers, you must:

- split the number into digits
- convert each digit to ASCII
- send each digit separately

The CPU does not understand characters like '4'—only numbers.

ASCII provides a mapping between numbers and characters so that values can be displayed as readable text.

4.2.6 Program: 0003_add.spin2

Listing 4.1: Program: addition

```
'add.spin2
CON
    _CLKFREQ          = 180_000_000
    BAUD_RATE         = 2_000_000
    TRANSMISSION_PIN  = 62
    BIT_PERIOD        = (_CLKFREQ / BAUD_RATE)
    BAUD_MODE         = (BIT_PERIOD << 16) | (8 - 1)

DAT
    org 0
    asmclk

    dirl    #TRANSMISSION_PIN
    wrpin   ##(P_ASYNC_TX | P_OE), #TRANSMISSION_PIN
    wxpin   ##BAUD_MODE, #TRANSMISSION_PIN
    dirh    #TRANSMISSION_PIN
    wypin   #1, #TRANSMISSION_PIN

    mov number1, #2
    mov number2, #2

    add number2, number1
    add number2, #"0"

    wypin number2, #TRANSMISSION_PIN

    endprog
    jmp    $$

    number1 res 1
    number2 res 1
```

When this program runs, the terminal will display the character "4".

First:

```
mov number1, #2
mov number2, #2
```

This stores the value 2 in both registers.

Next:

```
add number2, number1
```

This adds the value in `number1` to `number2`, so `number2` now contains 4.

Then:

```
add number2, #"0"
```

This converts the number 4 into its ASCII representation by adding 48, resulting in the character '4'.

Finally:

```
wypin number2, #TRANSMISSION_PIN
```

This sends the ASCII value to the terminal, displaying the character "4".

4.3 The SUB Instruction

The `sub` instruction subtracts one value from another.

The value from the source is subtracted from the destination, and the result is stored back in the destination register.

4.3.1 Syntax

```
sub destination, {#}source
```

- **destination** — where the result is stored
- **source** — value to subtract
- **#** — indicates a literal constant

4.3.2 Examples

```
sub A, #2
```

This subtracts 2 from register A.

```
sub B, A
```

This subtracts the value in A from B.

4.3.3 ASCII Conversion After Subtraction

As with addition, subtraction results can be converted into displayable characters using ASCII encoding.

The ASCII codes for digits '0' to '9' range from 48 to 57.

To display a single-digit number, add 48 to the result:

```
add A , #48
```

For example:

If A = 6:

```
sub A , #2      ' A becomes 4
add A , #48     ' A becomes ASCII 52 ('4')
```

When sent to a terminal, this displays the character '4'.

4.3.4 Important Limitation

This method only works for single-digit numbers.

To display multi-digit results, you must:

- split the number into digits
- convert each digit to ASCII separately
- transmit each digit one at a time

4.3.5 Program: 0004_sub.spin2

Listing 4.2: Program:0004_sub.spin2

```
'sub.spin2
CON
    _CLKFREQ          = 180_000_000
    BAUD_RATE         = 2_000_000
    TRANSMISSION_PIN  = 62
    BIT_PERIOD        = (_CLKFREQ / BAUD_RATE)
    BAUD_MODE         = (BIT_PERIOD << 16) | (8 - 1)

DAT
    org 0
    asmclk

    dir1    #TRANSMISSION_PIN
    wrpin   ##(P_ASYNC_TX | P_OE), #TRANSMISSION_PIN
    wxpin   ##BAUD_MODE, #TRANSMISSION_PIN
    dirh    #TRANSMISSION_PIN
    wypin   #1, #TRANSMISSION_PIN

    mov number1, #5
    mov number2, #3

    sub number1, number2
    add number1, #"0"

    wypin number1, #TRANSMISSION_PIN

    endprog
    jmp  $$

    number1 res 1
    number2 res 1
```

When this program runs, the terminal will display the character "2".

First:

```
mov number1, #5
mov number2, #3
```

This stores the values 5 and 3 in the respective registers.

Next:

```
sub number1, number2
```

This subtracts `number2` from `number1`, leaving `number1` with the value 2.

Then:

```
add number1, #"0"
```

This converts the numeric value into its ASCII representation by adding 48, producing the character '2'.

Finally:

```
wypin number1, #TRANSMISSION_PIN
```

This sends the ASCII value through the transmission pin, and the terminal displays the character "2".

4.4 The MUL Instruction

The `mul` instruction multiplies two values. The value in the destination register is multiplied by the source, and the result is stored back in the destination register, updating it with the new product.

This allows you to perform precise multiplication directly on the values your program is working with.

`mul` is used because it efficiently multiplies two numbers in a single operation. It works with both literal values and register values, making it versatile for many tasks such as scaling values, performing arithmetic calculations, or preparing data for output.

This provides fast integer multiplication, making your programs more efficient and predictable.

4.4.1 Syntax

```
mul destination, {#}source
```

- **destination** — where the result is stored
- **source** — value to multiply
- **#** — indicates a literal constant (number, hex, ASCII)

4.4.2 Examples

```
mul A, #2
```

This multiplies the value in A by 2 and stores the result back in A.

```
mul B, A
```

This multiplies B by the value in A and stores the result in B.

4.4.3 Program: 0005_mul.spin2

Listing 4.3: Program: 0005_mul.spin2

```
'mul.spin2
CON
    _CLKFREQ          = 180_000_000
    BAUD_RATE         = 2_000_000
    TRANSMISSION_PIN  = 62
    BIT_PERIOD        = (_CLKFREQ / BAUD_RATE)
    BAUD_MODE          = (BIT_PERIOD << 16) | (8 - 1)

DAT
    org 0
    asmclk

    dir1    #TRANSMISSION_PIN
    wrpin   ##(P_ASYNC_TX | P_OE), #TRANSMISSION_PIN
    wxpin   ##BAUD_MODE, #TRANSMISSION_PIN
    dirh    #TRANSMISSION_PIN
    wypin   #1, #TRANSMISSION_PIN

    mov number1, #2
    mov number2, #3

    mul number2, number1
    add number2, #"0"
    wypin number2, #TRANSMISSION_PIN

    endprog
    jmp    #$.

number1 res 1
number2 res 1
```

When this program runs, the terminal will display the character "6".

First:

```
mov number1, #2
mov number2, #3
```

This stores the values 2 and 3 in the respective registers.

Next:

```
mul number2, number1
```

This multiplies `number2` by `number1`, so `number2` now holds 6.

Then:


```
add number2, #"0"
```

This converts the numeric value into its ASCII representation by adding 48, producing the character '6'.

Finally:

```
wypin number2, #TRANSMISSION_PIN
```

This sends the ASCII value through the transmission pin, and the terminal displays the character "6".

Exercise

In this exercise, you will practice manipulating variables using basic arithmetic instructions.

- Increment a variable by 1 using the `add` instruction.
- Decrease a variable and check whether it goes below zero, allowing you to handle underflow or trigger a conditional action.
- Double a value using multiplication, demonstrating how to scale a number efficiently in your program.

4.5 Summary

Beyond arithmetic computation, we now consider bitwise operations, which provide direct manipulation of binary data for control, masking, and hardware-level interfacing.

Chapter Summary

- Arithmetic operates directly on registers
- `add`, `sub`, and `mul` modify register values
- Results overwrite the destination register
- ASCII conversion is needed for readable output
- Math operations are fast and low-level

Chapter 5

Bitwise Operations

Bitwise operations manipulate individual bits within a value. A single byte can represent eight separate on/off states, such as LEDs, where each bit controls one LED.

Because bitwise operations work directly on the hardware level, they are extremely efficient. They allow your program to make decisions and control devices without extra memory usage or complex calculations.

A common concept in bitwise operations is the **bitmask**. A bitmask is a pattern of bits used to select, set, or clear specific bits within a value.

5.1 The AND Instruction

The **and** instruction is used to check or isolate specific bits without affecting the others.

It works by combining a value with a bitmask. Any bit that is 0 in the mask is cleared, while bits that are 1 in the mask are preserved. This makes **and** useful for reading hardware states, checking flags, and working with multiple signals stored in a single value.

For example:

```
00000101
AND 00000100
=    00000100
```

In this case, only bit 2 is set in the mask, so the result shows that bit 2 was active while all other bits are cleared.

The **and** instruction evaluates each bit individually. A result bit is set to 1 only when both corresponding bits are 1. Otherwise, the result bit becomes 0.

This makes **and** an essential tool in embedded programming, especially when working with hardware registers and status flags.

5.1.1 Syntax

```
and destination, {#}source
```

- **destination** — where the result is stored
- **source** — value or mask used for the operation
- **#** — indicates a literal constant (number, hex, ASCII)

5.1.2 Program: 0006_and.spin2

Listing 5.1: Program: 0006_and.spin2

```
'and.spin2
CON
    _CLKFREQ          = 180_000_000
    BAUD_RATE         = 2_000_000
    TRANSMISSION_PIN  = 62
    BIT_PERIOD        = (_CLKFREQ / BAUD_RATE)
    BAUD_MODE         = (BIT_PERIOD << 16) | (8 - 1)

DAT
    org 0
    asmclk

    dirl    #TRANSMISSION_PIN
    wrpin   ##(P_ASYNC_TX | P_OE), #TRANSMISSION_PIN
    wxpin   ##BAUD_MODE, #TRANSMISSION_PIN
    dirh    #TRANSMISSION_PIN
    wypin   #1, #TRANSMISSION_PIN

    mov number1, #13
    and number1, #5

    add number1, #"0"
    wypin number1, #TRANSMISSION_PIN

    endprog
    jmp    #$.

    number1 res 1
```

The variable `number1` is first set to 13, which in binary is 00001101. The `and` instruction then performs a bitwise comparison with the value 5 (00000101), resulting in 00000101, which is decimal 5.

This value is then converted to an ASCII character by adding "0", producing 53, which corresponds to the character '5'.

The program sends this value to the terminal, so the expected output displayed on the screen is the single character 5.

This demonstrates how `and` can be used for bit masking to isolate specific bits in a value.

5.2 The OR Instruction

The `or` instruction performs a bitwise OR between two values. Each bit in the result is set to 1 if either or both of the corresponding bits in the source or destination are 1. The result

is stored back in the destination register.

`or` is commonly used to set specific bits without affecting others. This is useful for turning on LEDs, enabling flags, or configuring hardware states while preserving unrelated bits.

For example, to turn on a specific bit such as bit 2 (for an LED), you can use a bitwise OR with a mask:

00000001
OR 00000100
= 00000101

In this case, the original value remains active while bit 2 is also turned on.

This shows how `or` can safely set specific bits without affecting others, making it ideal for hardware control and managing multiple flags packed into a single byte or word.

5.2.1 Syntax

<code>or destination, {#}source</code>
--

- **destination** — where the result is stored
- **source** — value or mask used for the operation
- **#** — indicates a literal constant (number, hex, ASCII)

5.2.2 Program: 0007_or.spin2

Listing 5.2: Program: 0007_or.spin2

```

'or.spin2
CON
    _CLKFREQ          = 180_000_000
    BAUD_RATE         = 2_000_000
    TRANSMISSION_PIN  = 62
    BIT_PERIOD        = (_CLKFREQ / BAUD_RATE)
    BAUD_MODE         = (BIT_PERIOD << 16) | (8 - 1)

DAT
    org 0
    asmclk

    dirl    #TRANSMISSION_PIN
    wrpin   ##(P_ASYNC_TX | P_OE), #TRANSMISSION_PIN
    wxpin   ##BAUD_MODE, #TRANSMISSION_PIN
    dirh    #TRANSMISSION_PIN
    wypin   #1, #TRANSMISSION_PIN

    mov number1, #4
    or  number1, #3
    add number1, #"0"
    wypin number1, #TRANSMISSION_PIN

    endprog
    jmp  $$

    number1 res 1

```

The program first sets `number1` to 4 (00000100 in binary). The `or` instruction then performs a bitwise OR with 3 (00000011), resulting in 00000111, which is decimal 7.

This value is then converted to an ASCII character by adding "0", giving 55, which corresponds to the character '7'.

The program sends this value to the terminal, so the expected output displayed on the screen is the single character 7.

This demonstrates how `or` can be used to turn on multiple bits at once without affecting the other bits in the original value.

5.3 The XOR Instruction

The `xor` instruction performs a bitwise exclusive OR between two values. Each result bit is set to 1 only if the corresponding bits in the source and destination are different; if they are the same, the result is 0. The result is stored back in the destination register.

`xor` is commonly used to toggle specific bits, compare flags, or invert bits without affecting others.

It is especially useful for flipping bits in a register—changing 1 to 0 or 0 to 1—without altering unrelated bits. By XOR'ing with a mask that has 1s in the positions you want to toggle, only those bits are inverted.

For example:

```
00000001
XOR 00000100
=    00000101
```

Here, bit 2 is toggled while all other bits remain unchanged.

If the same operation is repeated again, bit 2 will flip back off, demonstrating how `xor` can efficiently toggle specific bits.

5.3.1 Syntax

```
xor destination, {#}source
```

- **destination** — where the result is stored
- **source** — value or mask used for the operation
- **#** — indicates a literal constant (number, hex, ASCII)

5.3.2 Program: 0008_xor.spin2

Listing 5.3: Program: 0008_xor.spin2

```
'xor.spin2
CON
    _CLKFREQ          = 180_000_000
    BAUD_RATE         = 2_000_000
    TRANSMISSION_PIN  = 62
    BIT_PERIOD        = (_CLKFREQ / BAUD_RATE)
    BAUD_MODE         = (BIT_PERIOD << 16) | (8 - 1)

DAT
    org 0
    asmclk

    dirl    #TRANSMISSION_PIN
    wrpin   ##(P_ASYNC_TX | P_OE), #TRANSMISSION_PIN
    wxpin   ##BAUD_MODE, #TRANSMISSION_PIN
    dirh    #TRANSMISSION_PIN
    wypin   #1, #TRANSMISSION_PIN

    mov number1, #6
    xor number1, #3

    add number1, #"0"
    wypin number1, #TRANSMISSION_PIN

    endprog
    jmp    #$.

    number1 res 1
```

We set `number1` to 6, which in binary is 00000110. The `xor` instruction then performs a bitwise exclusive OR with 3 (00000011), resulting in 00000101, which is decimal 5.

This value is converted to an ASCII character by adding "0", producing 53, which corresponds to the character '5'.

The program sends this value to the terminal, so the expected output displayed on the screen is the single character 5.

This example demonstrates how `xor` can be used to toggle specific bits in a value while leaving the other bits unchanged.

5.4 The NOT Instruction

The `not` instruction performs a bitwise inversion of a value, flipping every bit in the destination register so that 1s become 0s and 0s become 1s. The result is stored back in the same

register.

This operation is useful for creating the binary opposite of a number, inverting flags, generating masks, or toggling all bits at once. It is a powerful tool in low-level and embedded programming.

For example:

```
00000001
NOT 00000001
=   11111110
```

If a register holds the binary value 00000001 (decimal 1), applying **not** flips every bit, resulting in 11111110, which is decimal 254 in an 8-bit system.

This demonstrates how **not** inverts all bits in a value, producing the binary opposite of the original number.

5.4.1 Syntax

```
not destination
```

- **destination** — where the result is stored

5.4.2 Program: 0009_not.spin2

Listing 5.4: Program: 0009_not.spin2

```
'not.spin2
CON
    _CLKFREQ          = 180_000_000
    BAUD_RATE         = 2_000_000
    TRANSMISSION_PIN  = 62
    BIT_PERIOD        = (_CLKFREQ / BAUD_RATE)
    BAUD_MODE         = (BIT_PERIOD << 16) | (8 - 1)

DAT
    org 0
    asmclk

    dir1    #TRANSMISSION_PIN
    wrpin   ##(P_ASYNC_TX | P_OE), #TRANSMISSION_PIN
    wxpin   ##BAUD_MODE, #TRANSMISSION_PIN
    dirh    #TRANSMISSION_PIN
    wypin   #1, #TRANSMISSION_PIN

    mov number1, #3
    not number1

    and number1, #%1111

    add number1, #"0"
    wypin number1, #TRANSMISSION_PIN

    endprog
    jmp  $$

    number1 res 1
```

We set `number1` to 3, which in binary is 00000011. The `not` instruction inverts all bits, producing 11111100 in an 8-bit context.

To focus on only the lowest 4 bits, the program then performs a bitwise `and` with 00001111, resulting in 00001100, which is decimal 12.

This value is then converted to an ASCII character by adding "0", giving 60. ASCII code 60 corresponds to the character '<' rather than the digits "12", showing why multi-digit values require special handling when converting to ASCII.

5.5 Exercise

In this exercise, you will practice manipulating individual bits in a register.

- Use `and` with a mask to keep only bit 2 of `number1`, clearing all other bits.
- Use `or` with a mask to turn on bit 0 without affecting the other bits.
- Use `xor` with a mask to toggle bit 3, flipping it while leaving the rest unchanged.
- Apply `not` to invert all bits of `number1`, creating the binary opposite of its current value.

5.6 Summary

Extending bitwise manipulation, we now examine shift operations, which enable efficient data scaling, alignment, and binary transformation within the processor.

Chapter Summary

- Bitwise operations manipulate individual bits
- `and`, `or`, `xor`, and `not` control binary patterns
- Bit masking allows selective data modification
- These operations are essential for hardware control
- Bitwise logic is fundamental to low-level programming

Chapter 6

Shifting

Shifting instructions allow you to move the bits in a value left or right. This is useful for efficiently multiplying or dividing by powers of 2, manipulating individual flags, and working with packed binary data.

By moving bits, you can quickly scale values, position specific bits, and perform low-level operations that are essential in embedded programming.

6.1 SHL — Shift Left

The `shl` (shift left) instruction moves the bits in a value to the left. Each shift effectively multiplies the value by 2.

It takes a destination register and a source value (either a register or a literal), which specifies how many positions to shift. New bits entering from the right are always set to 0, and the result is stored back into the destination register.

`shl` is commonly used for fast multiplication by powers of 2, bit manipulation, and positioning data within a register. It provides a fast and efficient alternative to standard multiplication in low-level programming.

6.1.1 Syntax

<code>shl destination, {#}source</code>

- **destination** — where the result is stored
- **source** — number of bit positions to shift
- **#** — indicates a literal constant (number, hex, ASCII)

6.1.2 Program: 0010_shl

Listing 6.1: Program: 0010_shl.spin2

```
'shl.spin2
CON
    _CLKFREQ          = 180_000_000
    BAUD_RATE         = 2_000_000
    TRANSMISSION_PIN  = 62
    BIT_PERIOD        = (_CLKFREQ / BAUD_RATE)
    BAUD_MODE          = (BIT_PERIOD << 16) | (8 - 1)

DAT
    org 0
    asmclk

    dirl    #TRANSMISSION_PIN
    wrpin   ##(P_ASYNC_TX | P_OE), #TRANSMISSION_PIN
    wxpin   ##BAUD_MODE, #TRANSMISSION_PIN
    dirh    #TRANSMISSION_PIN
    wypin   #1, #TRANSMISSION_PIN

    mov     number1, #2
    shl     number1, #2
    add     number1, #"0"
    wypin   number1, #TRANSMISSION_PIN

    endprog
    jmp     $$

    number1 res 1
```

We set `number1` to 2. The `shl` instruction then shifts this value left by 2 positions, effectively multiplying it by 4, resulting in 8.

Adding "0" converts this numeric value to its ASCII code, producing 56, which corresponds to the character '8'.

Finally, the program sends this value to the terminal via the TX pin, so the expected output displayed on the screen is the single character '8'.

This example demonstrates how shifting can be used for fast multiplication and combined with ASCII conversion for terminal output.

6.2 SHR — Shift Right

The `shr` (shift right) instruction moves the bits of a value to the right, effectively dividing it by 2 for each position shifted.

It takes a destination register and a source value (either a register or a literal), which

specifies how many positions to shift. New bits are filled on the left according to shift rules, and the result is stored back into the destination register.

`shr` is commonly used for fast division by powers of 2, extracting specific bits, and manipulating packed binary data in low-level programming.

6.2.1 Syntax

<code>shr destination, {#}source</code>

- **destination** — where the result is stored
- **source** — number of bit positions to shift
- **#** — indicates a literal constant (number, hex, ASCII)

6.2.2 Program: 0011_shr.spin2

Listing 6.2: Program: 0011_shr.spin2

```
'shr.spin2
CON
    _CLKFREQ          = 180_000_000
    BAUD_RATE         = 2_000_000
    TRANSMISSION_PIN  = 62
    BIT_PERIOD        = (_CLKFREQ / BAUD_RATE)
    BAUD_MODE          = (BIT_PERIOD << 16) | (8 - 1)

DAT
    org 0
    asmclk

    dirl    #TRANSMISSION_PIN
    wrpin   ##(P_ASYNC_TX | P_OE), #TRANSMISSION_PIN
    wxpin   ##BAUD_MODE, #TRANSMISSION_PIN
    dirh    #TRANSMISSION_PIN
    wypin   #1, #TRANSMISSION_PIN

    mov     number1, #16
    shr     number1, #1
    add     number1, #"0"
    wypin   number1, #TRANSMISSION_PIN

    endprog
    jmp     $$

    number1 res 1
```

Here, we set `number1` to 16. The `shr` instruction then shifts this value right by 1 position, effectively dividing it by 2, resulting in 8.

Adding "0" converts this numeric value to its ASCII code, producing 56, which corresponds to the character '8'.

Finally, the program sends this value to the terminal via the TX pin, so the expected output displayed on the screen is the single character '8'.

This example demonstrates how `shr` can be used for fast division by powers of 2 and combined with ASCII conversion for terminal output.

6.3 SAL — Shift Arithmetic Left

The `sal` (shift arithmetic left) instruction shifts the bits of a value to the left, effectively multiplying it by 2 for each position shifted while preserving the sign of signed numbers.

It takes a destination register and a source value (either a register or a literal) specifying

how many positions to shift. The result is stored back in the destination register.

sal is useful for efficient multiplication of signed numbers, correct handling of signed values, and low-level bit manipulation. It is commonly used in real-time calculations, graphics, animation, and signal processing.

6.3.1 Syntax

<pre>sal destination, {#}source</pre>

- **destination** — where the result is stored
- **source** — number of bit positions to shift
- **#** — indicates a literal constant (number, hex, ASCII)

6.3.2 Program: 0012_sal.spin2

Listing 6.3: Program: 0012_sal.spin2

```
'sal.spin2
CON
    _CLKFREQ          = 180_000_000
    BAUD_RATE         = 2_000_000
    TRANSMISSION_PIN  = 62
    BIT_PERIOD        = (_CLKFREQ / BAUD_RATE)
    BAUD_MODE         = (BIT_PERIOD << 16) | (8 - 1)

DAT
    org 0
    asmclk

    dirl    #TRANSMISSION_PIN
    wrpin   ##(P_ASYNC_TX | P_OE), #TRANSMISSION_PIN
    wxpin   ##BAUD_MODE, #TRANSMISSION_PIN
    dirh    #TRANSMISSION_PIN
    wypin   #1, #TRANSMISSION_PIN

    mov     number1, #4
    sal     number1, #1
    add     number1, #"0"
    wypin   number1, #TRANSMISSION_PIN

    endprog
    jmp    #$.

    number1 res 1
```

The variable `number1` is set to 4. The `sal` instruction then shifts this value left by 1 position, effectively multiplying it by 2, resulting in 8.

Adding "0" converts this numeric value to its ASCII code, producing 56, which corresponds to the character '8'.

Finally, the program sends this value to the terminal via the TX pin, so the expected output displayed on the screen is the single character '8'.

This demonstrates how `sal` can be used for fast multiplication while preserving signed values and combining it with ASCII conversion for display.

6.4 SAR — Shift Arithmetic Right

The `sar` (shift arithmetic right) instruction shifts the bits of a value to the right, effectively dividing it by 2 for each position while preserving the sign of signed numbers through sign extension of the leftmost bit.

It takes a destination register and a source value (either a register or a literal) specifying how many positions to shift. The result is stored back in the destination register.

sar is useful for fast division of signed numbers, bit manipulation, and low-level arithmetic operations. It is commonly used in real-time applications such as games, signal processing, and animations.

6.4.1 Syntax

<pre>sar destination, {#}source</pre>

- **destination** — where the result is stored
- **source** — number of bit positions to shift
- **#** — indicates a literal constant (number, hex, ASCII)

6.4.2 Program: 0013_sar.spin2

Listing 6.4: Program: 0013_sar.spin2

```
'sar.spin2
CON
    _CLKFREQ      = 180_000_000
    BAUD_RATE     = 2_000_000
    TRANSMISSION_PIN = 62
    BIT_PERIOD    = (_CLKFREQ / BAUD_RATE)
    BAUD_MODE     = (BIT_PERIOD << 16) | (8 - 1)

DAT
    org 0
    asmclk

    dirl    #TRANSMISSION_PIN
    wrpin   ##(P_ASYNC_TX | P_OE), #TRANSMISSION_PIN
    wxpin   ##BAUD_MODE, #TRANSMISSION_PIN
    dirh    #TRANSMISSION_PIN
    wypin   #1, #TRANSMISSION_PIN

    mov     number1, #4
    sar     number1, #1
    add     number1, #"0"
    wypin   number1, #TRANSMISSION_PIN

    endprog
    jmp     $$

    number1 res 1
```

If the value 4 is shifted right by 1 using `sar`, the result is 2. Adding "0" converts this numeric value to its ASCII code, producing the character '2'.

Sending this value over the TX pin will display '2' on a connected terminal, demonstrating how `sar` can be used for fast division by powers of 2 while preserving the sign for signed values and combining it with ASCII conversion for terminal output.

6.5 Exercise

In this exercise, you will practice using shift instructions to manipulate values.

- First, use `shl` to shift the value in `number1` one position to the left, doubling its current value.
- Next, use `shr` to shift `number1` two positions to the right, effectively dividing it by four.

- Then, apply `sal` to shift the signed value in `number1` three positions to the left, multiplying it by eight while preserving its sign.
- Finally, use `sar` to shift the signed value in `number1` one position to the right, halving it while keeping the sign intact.

6.6 Summary

Having established basic instruction-level operations, we now introduce repetition structures that enable iterative execution of instruction sequences.

Chapter Summary

- Shifts move bits left or right within a register
- `shl` and `shr` enable fast multiply/divide operations
- `sal` and `sar` support signed values
- Shifting changes both value and bit position
- Useful for performance and bit-level control

Chapter 7

Loops

Loops allow the CPU to repeatedly execute a block of code without rewriting instructions. Unlike high-level languages, loops in P2ASM are built using conditional tests and jump instructions, often paired with counters stored in registers.

They are essential for tasks such as blinking LEDs, reading multiple inputs, or processing arrays in memory, where repetition at the hardware level would otherwise be inefficient and error-prone.

Loops are a fundamental tool in P2ASM, allowing the CPU to execute a section of code multiple times without repeating instructions. They provide precise control over timing and iteration by using counters, conditions, or flags, making them flexible for tasks such as controlling hardware, reading inputs, and processing data in memory.

By combining these instructions with registers as counters, you can create flexible loops that either repeat a section of code a precise number of times or continue running until a specific condition is met. This approach gives P2ASM programmers precise control over program flow, making loops essential for tasks ranging from hardware control to data processing.

7.1 JMP — Jump (Unconditional)

The `jmp` instruction causes an unconditional jump to a specified location in the program. Execution immediately continues from that target location.

This makes `jmp` essential for tasks such as skipping code, repeating sections in loops, redirecting program flow, or handling errors.

7.1.1 Syntax

<code>jmp #address</code>

- **address** — the label or memory location to jump to
- **#** — indicates a literal address or constant

- `jmp` always transfers execution unconditionally

7.1.2 Program: 0014_jump.spin2

Listing 7.1: Program: 0014_jump.spin2

```

'jump.spin2
CON
    _CLKFREQ          = 180_000_000
    BAUD_RATE         = 2_000_000
    TRANSMISSION_PIN  = 62
    BIT_PERIOD        = (_CLKFREQ / BAUD_RATE)
    BAUD_MODE          = (BIT_PERIOD << 16) | (8 - 1)

DAT
    org 0
    asmclk

    dir1    #TRANSMISSION_PIN
    wrpin   ##(P_ASYNC_TX | P_OE), #TRANSMISSION_PIN
    wxpin   ##BAUD_MODE, #TRANSMISSION_PIN
    dirh    #TRANSMISSION_PIN
    wypin   #1, #TRANSMISSION_PIN

    jmp     #larger_value

smaller_value
    mov     number1, #2

larger_value
    mov     number1, #4
    add     number1, #"0"
    wypin   number1, #TRANSMISSION_PIN

endprog
    jmp    #$.

number1 res 1

```

A label `smaller_value` is defined but immediately jumps to `larger_value` using `jmp #larger_value`. This means the code under `smaller_value` is skipped, and execution continues at `larger_value`.

At `larger_value`, the program sets `number1` to 4, converts it to its ASCII equivalent by adding "0", and then transmits it using `wypin` on the `TRANSMISSION_PIN`.

As a result, the value 4 is sent, and the instructions under `smaller_value` never execute.

7.2 DJNZ — Decrement and Jump if Not Zero

The `djnz` (decrement and jump if not zero) instruction decreases the value in a register by one and checks the result.

If the value is not zero, execution jumps to the specified target, repeating the loop. If the value reaches zero, execution continues to the next instruction, ending the loop.

By combining decrementing and conditional branching into a single instruction, `djnz` enables compact and efficient count-controlled loops. It is commonly used for timing loops, repetitive I/O, and buffer or memory processing in embedded systems where speed and code size matter.

7.2.1 Syntax

<code>djnz register, {#}target</code>

- **register** — value to be decremented (must be a register or variable)
- **target** — label or address to jump to if result is not zero
- **#** — used only for literal numeric addresses (not labels)

7.2.2 Program: 0015_djnz.spin2

Listing 7.2: Program: 0015_djnz.spin2

```

'djnz.spin2
CON
    _CLKFREQ          = 180_000_000
    BAUD_RATE         = 2_000_000
    TRANSMISSION_PIN  = 62
    BIT_PERIOD        = (_CLKFREQ / BAUD_RATE)
    BAUD_MODE          = (BIT_PERIOD << 16) | (8 - 1)

DAT
    org 0
    asmclk

    dirl    #TRANSMISSION_PIN
    wrpin   ##(P_ASYNC_TX | P_OE), #TRANSMISSION_PIN
    wxpin   ##BAUD_MODE, #TRANSMISSION_PIN
    dirh    #TRANSMISSION_PIN
    wypin   #1, #TRANSMISSION_PIN

    mov     number1, #5

count_loop
    djnz    number1, #process_number
    jmp     #done

process_number
    mov     ascii_number, number1
    add     ascii_number, #"0"
    jmp     #count_loop

done
    wypin   ascii_number, #TRANSMISSION_PIN

endprog
    jmp    $$

number1 res 1
ascii_number res 1

```

In the first step, `number1` is initialized to 5, and the program enters the `count_loop`. The `djnz` instruction decrements `number1` by 1 and checks if it is zero.

If `number1` is not zero, execution jumps to `process_number`, where the current value of `number1` is copied to `ascii_number` and converted to its ASCII equivalent by adding "0".

The program then jumps back to `count_loop`. This process repeats until `number1` reaches

zero.

Finally, at the `done` label, the program sends the last stored value of `ascii_number` via `wypin` on the `TRANSMISSION_PIN`. As a result, only the final number before zero (1 in this case) is transmitted to the terminal, while all previous values are decremented but not printed.

7.3 TJNZ — Test and Jump if Not Zero

The `tjnz` (test and jump if not zero) instruction checks the value of a register and conditionally alters program flow.

If the register is not zero, execution jumps to the specified label. If it is zero, execution continues with the next instruction.

Because `tjnz` tests a register without modifying it, it is ideal for loops, counters, flags, and conditional branching. It simplifies code by removing the need for separate compare operations.

By combining the test and jump in a single instruction, `tjnz` makes loop control, status checks, and event polling more efficient and readable.

7.3.1 Syntax

<code>tjnz register, {#}target</code>

- **register** — value to be tested (must be a register or variable)
- **target** — label or address to jump to if the value is not zero
- **#** — used only for literal numeric addresses (not labels)

7.3.2 Program: 0016_tjnz.spins2

Listing 7.3: Program: 0016_tjnz.spin2

```

'tjnz.spin2
CON
    _CLKFREQ          = 180_000_000
    BAUD_RATE         = 2_000_000
    TRANSMISSION_PIN  = 62
    BIT_PERIOD        = (_CLKFREQ / BAUD_RATE)
    BAUD_MODE          = (BIT_PERIOD << 16) | (8 - 1)

DAT
    org 0
    asmclk

    dirl    #TRANSMISSION_PIN
    wrpin   ##(P_ASYNC_TX | P_OE), #TRANSMISSION_PIN
    wxpin   ##BAUD_MODE, #TRANSMISSION_PIN
    dirh    #TRANSMISSION_PIN
    wypin   #1, #TRANSMISSION_PIN

    mov     number1, #5

count_loop
    tjnz    number1, #process_number
    jmp     #done

process_number
    mov     ascii_number, number1
    add     ascii_number, #"0"
    sub     number1, #1
    jmp     #count_loop

done
    wypin   ascii_number, #TRANSMISSION_PIN

endprog
    jmp    #$

number1 res 1
ascii_number res 1

```

We initialize `number1` to 5 as the counter. The program enters `count_loop`, where `tjnz` tests if `number1` is non-zero.

If it is non-zero, execution jumps to `process_number`; if it is zero, execution jumps to `done` and the loop ends.

In `process_number`, the current value of `number1` is copied into `ascii_number`, converted to its ASCII code by adding "0", and then `number1` is decremented by 1.

Execution then jumps back to `count_loop`. Because `wypin` is only at the `done` label, only the last value of `ascii_number` (1 in this case) is transmitted via the transmission pin, not the intermediate values in the countdown.

7.4 TJZ — Test and Jump if Zero

The `tjz` (test and jump if zero) instruction checks whether a register contains zero.

If the register is zero, execution jumps to the specified label; otherwise, the program continues with the next instruction.

Because `tjz` tests a register without modifying it, it is useful for conditional branching such as wait loops, status monitoring, and detecting when counters reach zero.

It is commonly used in embedded control, polling routines, and event-driven logic. `tjz` complements `tjnz` by providing the opposite condition, giving programmers flexible control over program flow.

7.4.1 Syntax

<code>tjz register, {#}target</code>

- **register** — value to be tested (must be a register or variable)
- **target** — label or address to jump to if the value is zero
- **#** — used only for literal numeric addresses (not labels)

7.4.2 Program: 0017_tjz.spin2

Listing 7.4: Program: 0017_tjz.spin2

```

'tjz.spin2
CON
    _CLKFREQ          = 180_000_000
    BAUD_RATE         = 2_000_000
    TRANSMISSION_PIN  = 62
    BIT_PERIOD        = (_CLKFREQ / BAUD_RATE)
    BAUD_MODE          = (BIT_PERIOD << 16) | (8 - 1)

DAT
    org 0
    asmclk

    dir1    #TRANSMISSION_PIN
    wrpin   ##(P_ASYNC_TX | P_OE), #TRANSMISSION_PIN
    wxpin   ##BAUD_MODE, #TRANSMISSION_PIN
    dirh    #TRANSMISSION_PIN
    wypin   #1, #TRANSMISSION_PIN

    mov     number1, #5

    process_number
        sub     number1, #1
        tjz     number1, #done
        mov     ascii_flag, number1
        add     ascii_flag, #"0"
        jmp     #process_number

    done
        wypin   ascii_flag, #TRANSMISSION_PIN

    endprog
        jmp     #$.

    number1 res 1
    ascii_flag res 1

```

We initialize `number1` to 5 as the counter. The program enters `process_number`, where `number1` is first decremented by 1.

The `tjz` instruction checks if `number1` is zero: if so, execution jumps to `done`; if not, it continues execution.

The current value of `number1` is copied into `ascii_flag` and converted to its ASCII code by adding "0". Execution then jumps back to `process_number`.

Because the `wypin` instruction is only at the `done` label, only the last value of `ascii_flag`

before the loop ends (1 in this case) is transmitted via the transmission pin, not the intermediate values in the countdown.

Exercise

Practice using P2ASM jump instructions by creating loops and conditional branches with different behaviors.

- Start by using `jmp` to create a loop that sets `number1` to 1, jumps back to the loop label three times, and then sets `number1` to 5 and prints it as ASCII.
- Next, initialize a counter to 4 and use `djnz` to decrement it in a loop, printing the counter value as ASCII each iteration until it reaches zero.
- Then, set a counter to 3 and use `tjnz` to repeat a block that increments `number1` by 2, printing its value as ASCII each time until the counter reaches zero.
- Finally, initialize a flag to 5 and use `tjz` to wait in a loop while decrementing the flag, and once the loop finishes, set `number1` to 9 and print it as ASCII.

7.5 Summary

With repetition mechanisms in place, we now extend program behavior through conditional execution, allowing instructions to respond dynamically to changing data states.

Chapter Summary

- Loops repeat code using jump instructions
- `jmp` creates unconditional loops
- `djnz`, `tjnz`, and `tjz` control loop conditions
- Loops rely on register values
- Execution flow is manually controlled

Chapter 8

Decision Making (Conditional Execution)

Flags are single-bit indicators that describe the result of an operation. Instead of storing values, they store conditions—they tell you what happened during the last instruction.

The two most important flags in this context are the Zero Flag (Z) and the Carry Flag (C). These flags are updated automatically by instructions such as `cmp` and arithmetic operations, allowing the program to make decisions based on previous results without storing additional values.

Zero Flag (Z)

The Zero Flag is set whenever the result of an operation equals zero.

- $Z = 1$ means the result is zero
- $Z = 0$ means the result is non-zero

This flag is commonly used for equality checks, loop termination, and detecting when a value reaches zero.

Carry Flag (C)

The Carry Flag reflects results where a calculation exceeds register limits or requires a borrow during subtraction.

It is especially important for unsigned comparisons and multi-precision arithmetic.

The Carry flag is commonly used to determine:

- whether one value is less than another (unsigned)
- whether a borrow or overflow occurred
- loop or boundary conditions

8.1 Decision-Making Overview

Decision-making instructions allow a program to react to different situations instead of executing sequentially.

They enable the program to:

- check if a value is zero or non-zero
- compare two values
- react to overflow or carry conditions
- execute instructions only when specific conditions are met

This provides conditional logic similar to if-else statements in higher-level languages.

Condition	Instruction	Meaning
Equal (==)	<code>cmp a, b wz if_z</code>	Execute if $a = b$
Not equal (!=)	<code>cmp a, b wz if_nz</code>	Execute if $a \neq b$
Less than (<)	<code>cmp a, b wc if_c</code>	Execute if $a < b$ (unsigned)
Greater than (>)	<code>cmp a, b wcz if_a</code>	Execute if $a > b$ (unsigned)
Less or equal (<=)	<code>cmp a, b wcz if_be</code>	Execute if $a \leq b$
Greater or equal (>=)	<code>cmp a, b wc if_nc</code>	Execute if $a \geq b$

8.2 CMP — Compare

The `cmp` instruction compares two values without modifying either operand.

Internally, it performs a subtraction but only updates condition flags rather than storing the result.

8.2.1 Syntax

```
cmp destination, {#}source wz
cmp destination, {#}source wc
cmp destination, {#}source wcz
```

- **destination** — value being compared
- **source** — value to compare against
- **#** — literal constant (number, hex, ASCII)
- **wz** — updates Zero flag
- **wc** — updates Carry flag
- **wcz** — updates both flags

8.3 IF_Z

The `if_z` instruction conditionally executes the next instruction only if the Zero flag (Z) is set.

If $Z = 1$, the instruction runs. If $Z = 0$, it is skipped.

This allows conditional execution without jumps.

8.3.1 Syntax

<code>if_z instruction</code>

- **instruction** — any valid P2ASM instruction
- Executes only when Z flag is set
- Commonly used after `cmp` or arithmetic operations

8.3.2 Program: 0018_if_z.spin2

Listing 8.1: Program: 0018_if_z.spin2

```

'if_z.spin2
CON
    _CLKFREQ          = 180_000_000
    BAUD_RATE         = 2_000_000
    TRANSMISSION_PIN  = 62
    BIT_PERIOD        = (_CLKFREQ / BAUD_RATE)
    BAUD_MODE          = (BIT_PERIOD << 16) | (8 - 1)

DAT
    org 0
    asmclk

    dir1    #TRANSMISSION_PIN
    wrpin   ##(P_ASYNC_TX | P_OE), #TRANSMISSION_PIN
    wxpin   ##BAUD_MODE, #TRANSMISSION_PIN
    dirh    #TRANSMISSION_PIN
    wypin   #1, #TRANSMISSION_PIN

    mov     isEqual, #0
    mov     number1, #2
    mov     number2, #2

    cmp     number1, number2 wz
    if_z    mov     isEqual, #"1"

    wypin   isEqual, #TRANSMISSION_PIN

    endprog
    jmp     $$

    number1 res 1
    number2 res 1
    isEqual res 1

```

Since number1 and number2 are both 2, CMP sets the Zero flag (Z) to 1. The IF_Z instruction then executes the next line, moving "1" into isEqual, which is sent to the transmission pin. This outputs 1. If the numbers were not equal, Z would be 0, IF_Z would not run, and nothing would be sent to the pin.

8.4 IF_NZ

The IF_NZ instruction runs the next instruction only if the Zero (Z) flag is clear ($Z = 0$). The Z flag is usually set by a previous instruction like CMP or an arithmetic operation.

This allows the program to execute code only when two values are not equal, making `IF_NZ` useful for decision-making and branching in cases where values differ.

`IF_NZ` is useful because it runs instructions only when a condition is not met, such as when two values are different. This avoids unnecessary jumps for simple checks, works together with `CMP` to create “if not equal” logic, and makes assembly programs easier to read by allowing inline conditional execution.

8.4.1 Syntax

`if_nz instruction`

- `instruction` = the instruction to execute
- The instruction can be any standard P2ASM instruction, including shifts, arithmetic, bitwise, or movement instructions
- Use `#` if it is a literal constant (number, hex, ASCII)
- Omit `#` if it is a variable or register
- Only runs the instruction if the Zero flag (WZ) is not set

8.4.2 Program: 0019_if_nz.spin2

Listing 8.2: Program: 0019_if_nz.spin2

```

'if_nz.spin2
CON
    _CLKFREQ          = 180_000_000
    BAUD_RATE         = 2_000_000
    TRANSMISSION_PIN  = 62
    BIT_PERIOD        = (_CLKFREQ / BAUD_RATE)
    BAUD_MODE          = (BIT_PERIOD << 16) | (8 - 1)

DAT
    org 0
    asmclk

    dirl    #TRANSMISSION_PIN
    wrpin   ##(P_ASYNC_TX | P_OE), #TRANSMISSION_PIN
    wxpin   ##BAUD_MODE, #TRANSMISSION_PIN
    dirh    #TRANSMISSION_PIN
    wypin   #1, #TRANSMISSION_PIN

    mov     isNotEqual, #1
    mov     number1, #2
    mov     number2, #3

    cmp     number1, number2 wz
    if_nz   mov     isNotEqual, #"0"

    wypin   isNotEqual, #TRANSMISSION_PIN

    endprog
    jmp     $$

    number1 res 1
    number2 res 1
    isNotEqual res 1

```

The program initializes `isNotEqual` to 1, then assigns `number1` the value 2 and `number2` the value 3. It compares the two numbers using `cmp`, setting the zero flag if they are equal. Since the values are not equal, the `if_nz` condition is satisfied and `isNotEqual` is updated to the ASCII character "0". This value is then transmitted through the configured Smart-Pin using `wypin`, after which the program enters an infinite loop. The variables `number1`, `number2`, and `isNotEqual` are reserved in memory for runtime use.

8.5 **CMP + IF_Z / IF_NZ**

While number1 is 2 and number2 is 3, CMP sets the Zero flag (Z) to 0. The IF_NZ instruction then executes the next line, moving "0" into isNotEqual, which is sent to the transmission pin, resulting in an output of 0. If the numbers had been equal, Z would be 1, IF_NZ would skip the instruction, leaving isNotEqual at its initial value of 1, which would then be sent to the pin, resulting in an output of 1.

By combining CMP with IF_Z and IF_NZ, your program can conditionally execute instructions based on equality without needing jumps or loops for simple checks.

Using CMP with IF_Z and IF_NZ makes decision-making in P2ASM simple and efficient. It avoids unnecessary jumps when you only need to run a single instruction conditionally, allows inline "if-else" style behavior, and is ideal for small equality checks, flags, and quick decision routines.

8.5.1 Syntax

```
cmp destination, {#} source wz
    if_z instruction
    if_nz instruction
```

8.5.2 Program: 0020_if_z_if_nz.spin2

Listing 8.3: Program: 0020_if_z_if_nz.spin2

```

' if_z_if_nz.spin2
CON
    _CLKFREQ          = 180_000_000
    BAUD_RATE         = 2_000_000
    TRANSMISSION_PIN  = 62
    BIT_PERIOD        = (_CLKFREQ / BAUD_RATE)
    BAUD_MODE          = (BIT_PERIOD << 16) | (8 - 1)

DAT
    org 0
    asmclk

    dirl    #TRANSMISSION_PIN
    wrpin   ##(P_ASYNC_TX | P_OE), #TRANSMISSION_PIN
    wxpin   ##BAUD_MODE, #TRANSMISSION_PIN
    dirh    #TRANSMISSION_PIN
    wypin   #1, #TRANSMISSION_PIN

    mov     number1, #9
    mov     number2, #7

    cmp     number1, number2 wz
    if_z    mov     isEqual, #"1"
    if_nz   mov     isEqual, #"0"

    wypin   isEqual, #TRANSMISSION_PIN

endprog
    jmp    #$.

number1 res 1
number2 res 1
isEqual res 1

```

We assign `number1` the value 9 and `number2` the value 7, then compares them using `cmp`, setting the zero flag if the values are equal. Since the numbers are not equal, the `if_z` condition executes and `isEqual` is set to the ASCII character "1", while the `if_nz` condition is skipped. The resulting value in `isEqual` is then transmitted through the SmartPin using `wypin`. After transmission, the program enters an infinite loop. The variables `number1`, `number2`, and `isEqual` are reserved in memory for runtime use.

8.6 **CMP + IF_C**

For this program, number1 is 9 and number2 is 7. Since the values are not equal, CMP sets the Zero flag (Z) to 0. The IF_Z instruction is skipped because Z = 0, while IF_NZ executes, moving "0" into isEqual, which is then sent to the transmission pin, resulting in an output of 0. Only one of the conditional instructions runs depending on the Z flag, demonstrating inline conditional execution based on a comparison.

The IF_C instruction executes the next instruction only when the Carry flag (C = 1), making it useful for unsigned "less than" comparisons. When combined with CMP wc, it allows your program to conditionally execute instructions based on the relative size of two values without using jumps.

This approach keeps the compared registers unchanged and provides an efficient way to implement decision-making, sorting routines, and loops where it is necessary to test whether one value is smaller than another.

8.6.1 **Syntax**

`if_c instruction`

- `instruction` = the instruction to execute
- The instruction can be any standard P2ASM instruction, including shifts, arithmetic, bitwise, or movement instructions
- Use `#` if it is a literal constant (number, hex, ASCII)
- Omit `#` if it is a variable or register
- Only runs if the Carry flag (WC) is set

8.6.2 Program: 0021_if_c.spin2

Listing 8.4: Program: 0021_if_c.spin2

```

' if_c.spin2
CON
    _CLKFREQ          = 180_000_000
    BAUD_RATE         = 2_000_000
    TRANSMISSION_PIN  = 62
    BIT_PERIOD        = (_CLKFREQ / BAUD_RATE)
    BAUD_MODE          = (BIT_PERIOD << 16) | (8 - 1)

DAT
    org 0
    asmclk

    dir1    #TRANSMISSION_PIN
    wrpin   ##(P_ASYNC_TX | P_OE), #TRANSMISSION_PIN
    wxpin   ##BAUD_MODE, #TRANSMISSION_PIN
    dirh    #TRANSMISSION_PIN
    wypin   #1, #TRANSMISSION_PIN

    mov     isLessThan, #0
    mov     number1, #1
    mov     number2, #2

    cmp     number1, number2 wc
    if_c    mov     isLessThan, #"1"

    wypin   isLessThan, #TRANSMISSION_PIN

endprog
    jmp    $$

number1     res 1
number2     res 1
isLessThan  res 1

```

Here the program initializes `isLessThan` to 0, then assigns `number1` the value 1 and `number2` the value 2. It compares the two values using `cmp`, which sets the carry flag when `number1` is less than `number2`. Since $1 < 2$, the `if_c` condition is satisfied and `isLessThan` is updated to the ASCII character "1". This value is then transmitted through the Smart-Pin using `wypin`, after which the program enters an infinite loop. The variables `number1`, `number2`, and `isLessThan` are reserved in memory for runtime use.

8.7 **CMP + IF_NC**

Here, number1 is 1 and number2 is 2. Since number1 is less than number2, **CMP** we sets the Carry flag (C) to 1. The **IF_C** instruction then executes, moving "1" into `isLessThan`, which is sent to the transmission pin, resulting in an output of 1. If number1 were greater than or equal to number2, C would be 0, the **IF_C** instruction would be skipped, leaving `isLessThan` at its initial value of 0, which would then be sent to the pin, resulting in an output of 0.

The **IF_NC** instruction executes the next instruction only when $C = 0$, making it useful for unsigned "greater than or equal" comparisons. By combining **CMP** we with **IF_NC**, your program can conditionally execute instructions based on relative size without using jumps.

Using **CMP** with **IF_NC** provides an efficient way to perform unsigned greater-than-or-equal comparisons in P2ASM. It allows instructions to execute conditionally without adding extra jumps, keeps the compared registers unchanged, and is especially useful in decision-making, sorting routines, and loops where you need to test whether one value is greater than or equal to another.

8.7.1 **Syntax**

`if_nc instruction`

- `instruction` = the instruction to execute
- The instruction can be any standard P2ASM instruction, including shifts, arithmetic, bitwise, or movement instructions
- Use `#` if it is a literal constant (number, hex, ASCII)
- Omit `#` if it is a variable or register
- Only runs if the Carry flag (WC) is not set

8.7.2 Program: 0022_if_nc.spin2

Listing 8.5: Program: 0022_if_nc.spin2

```

' if_nc.spin2
CON
    _CLKFREQ          = 180_000_000
    BAUD_RATE         = 2_000_000
    TRANSMISSION_PIN  = 62
    BIT_PERIOD        = (_CLKFREQ / BAUD_RATE)
    BAUD_MODE         = (BIT_PERIOD << 16) | (8 - 1)

DAT
    org 0
    asmclk

    dirl    #TRANSMISSION_PIN
    wrpin   ##(P_ASYNC_TX | P_OE), #TRANSMISSION_PIN
    wxpin   ##BAUD_MODE, #TRANSMISSION_PIN
    dirh    #TRANSMISSION_PIN
    wypin   #1, #TRANSMISSION_PIN

    mov isGreaterOrEqual, #0
    mov     number1, #6
    mov     number2, #2

    cmp     number1, number2 wc
    if_nc   mov     isGreaterOrEqual, #"1"

    wypin   isGreaterOrEqual, #TRANSMISSION_PIN

endprog
    jmp     $$

number1          res 1
number2          res 1
isGreaterOrEqual res 1

```

The program begins by setting `isGreaterOrEqual` to 0, then assigns `number1` the value 6 and `number2` the value 2. It compares the two values using `cmp`, which sets the carry flag based on the unsigned comparison result. Since `number1` is greater than or equal to `number2`, the `if_nc` condition is satisfied and `isGreaterOrEqual` is updated to the ASCII character "1". This value is then transmitted through the SmartPin using `wypin`, after which the program enters an infinite loop. The variables `number1`, `number2`, and `isGreaterOrEqual` are reserved in memory for runtime use.

8.8 **CMP + IF_C / IF_NC**

In this IF_NC example, number1 is 6 and number2 is 2. Since number1 is greater than or equal to number2, CMP wc sets the Carry flag (C) to 0. The IF_NC instruction then executes, moving "1" into isGreaterOrEqual, which is sent to the transmission pin, resulting in an output of 1. If number1 were less than number2, C would be 1, the IF_NC instruction would be skipped, leaving isGreaterOrEqual at its initial value of 0, which would then be sent to the pin, resulting in an output of 0.

The IF_C instruction executes the next instruction only when the Carry flag (C = 1), while IF_NC executes the next instruction only when C = 0. Together, these instructions enable unsigned comparisons with inline conditional execution, allowing your program to respond differently depending on whether a value is smaller or larger/equal, all without using jumps.

By combining CMP with IF_C and IF_NC, conditional execution becomes simple and efficient. Instructions can run only when needed, making this approach ideal for decision-making tasks such as sorting, checking thresholds, or controlling outputs. It also keeps the original register values unchanged while using flags to guide program flow.

8.8.1 **Syntax**

```
cmp destination, {#}source wc
    if_c instruction
    if_nc instruction
```

8.8.2 Program: 0023_if_c_if_nc.spin2

Listing 8.6: Program: 0023_if_c_if_nc.spin2

```

' if_c_if_nc.spin2
CON
    _CLKFREQ          = 180_000_000
    BAUD_RATE         = 2_000_000
    TRANSMISSION_PIN  = 62
    BIT_PERIOD        = (_CLKFREQ / BAUD_RATE)
    BAUD_MODE          = (BIT_PERIOD << 16) | (8 - 1)

DAT
    org 0
    asmclk

    dirl    #TRANSMISSION_PIN
    wrpin   ##(P_ASYNC_TX | P_OE), #TRANSMISSION_PIN
    wxpin   ##BAUD_MODE, #TRANSMISSION_PIN
    dirh    #TRANSMISSION_PIN
    wypin   #1, #TRANSMISSION_PIN

    mov     number1, #4
    mov     number2, #8

    cmp     number1, number2 wc
    if_c    mov     isLessThan, #"1"
    if_nc   mov     isLessThan, #"0"

    wypin   isLessThan, #TRANSMISSION_PIN

    endprog
    jmp     $$

    number1    res 1
    number2    res 1
    isLessThan res 1

```

In this program, `number1` is 4 and `number2` is 8. Since 4 is less than 8, `cmp wc` sets the Carry flag (C) to 1. The `if_c` instruction executes, moving "1" into `isLessThan`, which is sent to the transmission pin, resulting in an output of 1.

The `if_nc` instruction is skipped because $C \neq 0$. If `number1` were greater than or equal to `number2`, C would be 0, `if_c` would be skipped, and `if_nc` would execute instead, setting `isLessThan` to 0, which would then be sent to the pin, resulting in an output of 0.

8.9 **CMP + IF_BE**

The `if_be` instruction executes the next instruction only when the first value is less than or equal to the second, which occurs when the Carry ($C = 1$) or Zero ($Z = 1$) flag is set.

When combined with `cmp`, it enables efficient unsigned “ $\leq$ ” comparisons and inline conditional execution without extra jumps. This approach allows instructions to run conditionally for tasks such as threshold checks, sorting routines, or enforcing limits, while keeping the original registers unchanged and using the Carry and Zero flags to guide program flow.

8.9.1 **Syntax**

- `if_be instruction`
- `instruction` — the instruction to execute
 - The instruction can be any standard P2ASM instruction, including shifts, arithmetic, and bitwise operations.
 - Use `#` if it is a literal constant (number, hex, ASCII).
 - Omit `#` if it is a variable or register.
 - Executes only if the first value is less than or equal to the second ($C = 1$ or $Z = 1$).

8.9.2 Program: 0024_if_be.spin2

Listing 8.7: Program: 0024_if_be.spin2

```

' if_be.spin2
CON
    _CLKFREQ          = 180_000_000
    BAUD_RATE         = 2_000_000
    TRANSMISSION_PIN  = 62
    BIT_PERIOD        = (_CLKFREQ / BAUD_RATE)
    BAUD_MODE          = (BIT_PERIOD << 16) | (8 - 1)

DAT
    org 0
    asmclk

    dirl    #TRANSMISSION_PIN
    wrpin   ##(P_ASYNC_TX | P_OE), #TRANSMISSION_PIN
    wxpin   ##BAUD_MODE, #TRANSMISSION_PIN
    dirh    #TRANSMISSION_PIN
    wypin   #1, #TRANSMISSION_PIN

    mov isLessOrEqual, #0
    mov     number1, #5
    mov     number2, #5

    cmp     number1, number2 wcz
    if_be   mov     isLessOrEqual, #"1"

    wypin   isLessOrEqual, #TRANSMISSION_PIN

endprog
    jmp     #$

number1      res 1
number2      res 1
isLessOrEqual res 1

```

Here, `number1` is 5 and `number2` is 5. Since `number1` is less than or equal to `number2`, `cmp wcz` sets the Carry flag (C) to 0 and the Zero flag (Z) to 1. The `if_be` instruction executes, moving "1" into `isLessOrEqual`, which is sent to the transmission pin, resulting in an output of 1.

If `number1` were greater than `number2`, `C = 0` and `Z = 0`, the `if_be` instruction would be skipped, leaving `isLessOrEqual` at its initial value of 0, which would then be sent to the pin, resulting in an output of 0.

8.10 **CMP + IF_A**

The combination of `cmp` with the `if_a` condition performs unsigned greater-than comparisons and executes the next instruction only when the first operand is strictly greater than the second.

The comparison is evaluated using status flags produced by `cmp wcz`:

- Carry flag $C = 0$ (no borrow)
- Zero flag $Z = 0$ (values are not equal)

This condition is represented by `if_a` (above), indicating that the first operand is greater than the second in unsigned arithmetic.

This enables branch-free conditional execution, allowing instructions to be selectively executed in-place without altering program flow. As a result, `cmp` supports compact implementations of threshold testing, sorting decisions, and boundary enforcement. Because `cmp` only updates status flags via `wcz` and does not modify the source operands, it enables fast comparisons while preserving register state.

8.10.1 **Syntax**

- `cmp destination, {#}source wcz`
- `if_a <instruction>`
- `destination` — the register or variable to compare
- `source` — the value to compare with
 - Use `#` if it is a literal constant (number, hex, ASCII).
 - Omit `#` if it is a variable or register.

8.10.2 Program: 0025_if_a.spin2

Listing 8.8: Program: 0025_if_a.spin2

```

' if_a.spin2
CON
    _CLKFREQ          = 180_000_000
    BAUD_RATE         = 2_000_000
    TRANSMISSION_PIN  = 62
    BIT_PERIOD        = (_CLKFREQ / BAUD_RATE)
    BAUD_MODE          = (BIT_PERIOD << 16) | (8 - 1)

DAT
    org 0
    asmclk

    dirl    #TRANSMISSION_PIN
    wrpin   ##(P_ASYNC_TX | P_OE), #TRANSMISSION_PIN
    wxpin   ##BAUD_MODE, #TRANSMISSION_PIN
    dirh    #TRANSMISSION_PIN
    wypin   #1, #TRANSMISSION_PIN

    mov     isGreater, #0
    mov     number1, #8
    mov     number2, #4

    cmp     number1, number2 wcz
    if_a    mov isGreater, #"1"

    wypin   isGreater, #TRANSMISSION_PIN

endprog
    jmp    #$

number1    res 1
number2    res 1
isGreater  res 1

```

In this `if_*` code, `number1` is 8 and `number2` is 4. Since `number1` is greater than `number2`, `cmp wcz` sets the Carry flag ($C = 0$) and the Zero flag ($Z = 0$).

The `if_a` instruction executes because both $C = 0$ and $Z = 0$, moving "1" into `isGreater`, which is then sent to the transmission pin, resulting in an output of 1.

If `number1` were less than or equal to `number2`, either $C = 1$ or $Z = 1$, so the `if_a` instruction would be skipped, leaving `isGreater` at its initial value of 0, which would then be sent to the pin, resulting in an output of 0.

8.11 Exercise

Combine different P2ASM conditionals to observe their effects.

- Compare values using `cmp wcz` and use `if_be` and `if_a` to handle less-than-or-equal and greater-than cases, sending the results to a transmission pin.
- Experiment with `if_z`, `if_nz`, `if_c`, and `if_nc` in sequence to observe which instructions execute based on the flags.
- Verify the outputs on a terminal or display to confirm how each condition behaves.

This exercise helps demonstrate how multiple conditionals can control program flow efficiently without relying on explicit jump instructions.

8.12 Summary

As program complexity increases, we introduce subroutines as a means of structuring reusable instruction sequences and improving program modularity.

Chapter Summary

- Decisions are based on flags (Z and C)
- `cmp` sets flags for comparisons
- `if_z`, `if_nz`, `if_c`, and others control execution
- Conditions determine whether instructions run
- Enables branching without high-level constructs

Chapter 9

Subroutines

Subroutines allow you to organize your program into smaller, logical sections. This enables code reuse for tasks that repeat or require encapsulation. P2ASM supports procedures using `call`, `ret`, `jmp`, and local labels, giving you a structured way to manage program flow.

In P2ASM, `jmp` performs an unconditional jump to a label or address. `call` jumps to a location in code and saves the return address, allowing the program to return after execution. `ret` returns control to the instruction immediately following the last `call`. Local labels, prefixed with a dot (`.`), are visible only within the subroutine and are used for loops or conditional branches inside the subroutine.

Subroutines reduce code duplication, making programs easier to read, maintain, and debug. They enable structured programming even in assembly, allowing you to break complex tasks into manageable parts. Local labels prevent naming collisions between different procedures, keeping each section of code self-contained and organized.

9.1 JMP — Jump (Revisited)

The `jmp` instruction causes an unconditional transfer of control to a specified memory location, immediately setting the program counter to that address and skipping any instructions in between. Unlike `call`, it does not save a return address.

`jmp` is a fundamental tool in P2ASM for controlling program flow, enabling loops, branching, subroutines, error handling, or skipping sections of code. Its behavior is simple and predictable, as it always transfers execution to the target without checking flags or conditions.

9.1.1 Syntax

- `jmp #address`
- address — the label, memory location, or constant to jump to
 - Use `#` if it is a literal constant (number, hex, ASCII).
 - Omit `#` if it is a variable or register.
 - `jmp` always unconditionally transfers execution to the target address.

9.1.2 Program: 0026_jump.spin2

Listing 9.1: Program: 0026_jump.spin2

```
'jump.spin2
CON
    _CLKFREQ          = 180_000_000
    BAUD_RATE         = 2_000_000
    TRANSMISSION_PIN  = 62
    BIT_PERIOD        = (_CLKFREQ / BAUD_RATE)
    BAUD_MODE         = (BIT_PERIOD << 16) | (8 - 1)

DAT
    org 0
    asmclk

    dirl    #TRANSMISSION_PIN
    wrpin   ##(P_ASYNC_TX | P_OE), #TRANSMISSION_PIN
    wxpin   ##BAUD_MODE, #TRANSMISSION_PIN
    dirh    #TRANSMISSION_PIN
    wypin   #1, #TRANSMISSION_PIN

    mov     number1, #6

    jmp     #SetAndPrint

    mov     number1, #3    ' This will be skipped

SetAndPrint
    mov     number1, #5
    add     number1, #"0"
    wypin   number1, #TRANSMISSION_PIN

endprog
    jmp    #$.

number1 res 1
```

Execution begins with `mov number1, #6`, but the following instruction `mov number1, #3` is skipped because `jmp #SetAndPrint` immediately redirects execution to the `SetAndPrint` label.

At `SetAndPrint`, `number1` is set to 5, converted to its ASCII code by adding "0", and sent out via `wypin` on `TRANSMISSION_PIN`. The program then reaches `endprog` and terminates.

This demonstrates how `jmp` can be used to skip instructions and jump directly to a specific section of code.

9.2 CALL

The `call` instruction in P2ASM jumps to a subroutine while saving the return address, allowing the program to resume execution immediately after the `call` once the subroutine finishes.

Unlike `jmp`, which simply redirects execution, `call` enables structured programming by allowing code reuse without duplication, improving organization and maintainability.

When executed, the processor first saves the address of the instruction following the `call`, then jumps to the specified subroutine label. The subroutine code runs normally, and a `ret` instruction returns execution to the saved address.

This mechanism supports modular, reusable subroutines and structured programming patterns such as loops, conditional processing, and modular I/O handling, making programs easier to read, maintain, and debug.

9.2.1 Syntax

- `call #address`
- `ret`
- address — the label, memory location, or constant to jump to
 - `ret` must be used inside the subroutine to return control.
 - Use `#` if it is a literal constant (number, hex, ASCII).
 - Omit `#` if it is a variable or register.
 - `call` saves the return address before transferring execution.

9.2.2 Program: 0027_call.spin2

Listing 9.2: Program: 0027_call.spin2

```

'call.spin2
CON
    _CLKFREQ          = 180_000_000
    BAUD_RATE         = 2_000_000
    TRANSMISSION_PIN  = 62
    BIT_PERIOD        = (_CLKFREQ / BAUD_RATE)
    BAUD_MODE         = (BIT_PERIOD << 16) | (8 - 1)

DAT
    org 0
    asmclk

    dir1    #TRANSMISSION_PIN
    wrpin   ##(P_ASYNC_TX | P_OE), #TRANSMISSION_PIN
    wxpin   ##BAUD_MODE, #TRANSMISSION_PIN
    dirh    #TRANSMISSION_PIN
    wypin   #1, #TRANSMISSION_PIN

    mov     number1, #0

    call    #SetAndPrint

    jmp     #endprog

SetAndPrint
    mov     number1, #5
    add     number1, #"0"
    wypin   number1, #TRANSMISSION_PIN

    ret

endprog
    jmp     #$

number1 res 1

```

This program demonstrates the use of `call` to invoke a subroutine.

Execution starts with `mov number1, #0`, then `call #SetAndPrint` jumps to the `SetAndPrint` subroutine while saving the return address.

Inside the subroutine, `number1` is set to 5, converted to its ASCII code by adding "0", and sent out via `wypin` on `TRANSMISSION_PIN`.

After the subroutine finishes, `ret` returns execution to the instruction immediately following the `call`, which is `jmp #endprog`, terminating the program.

9.3 Global Labels and Local Labels

A label is a symbolic name representing a specific location in your code, allowing you to jump or call that location without remembering its numeric address. Labels improve code readability by giving descriptive names to program locations, making it easier to follow program flow and understand program structure. They also help organize code and maintain clarity in more complex programs.

Global Labels

Global labels define entry points for subroutines or main program sections. They can be targeted by instructions like `jmp` or `call`, allowing execution to jump to a subroutine or a specific part of the program.

When a `call` is used, execution jumps to the subroutine while saving the return address, and `ret` returns control to the instruction immediately following the original call. Global labels provide a way to structure your program and reuse code efficiently.

Local Labels

Local labels, typically prefixed with a dot (`.`), are scoped only to the nearest preceding global label. They are commonly used for loops or internal branching within subroutines and can be safely reused in different subroutines, preventing naming conflicts.

Local labels enable branching or looping within subroutines, supporting modular and organized code while keeping program flow clear and maintainable.

9.3.1 Syntax

- Global label — program-wide label
- Local label — section-only label (belongs to nearest global label above it)

9.3.2 Program: 0028_label.spin2

Listing 9.3: Program: 0028_label.spin2

```

'label.spin2
CON
    _CLKFREQ          = 180_000_000
    BAUD_RATE         = 2_000_000
    TRANSMISSION_PIN  = 62
    BIT_PERIOD        = (_CLKFREQ / BAUD_RATE)
    BAUD_MODE          = (BIT_PERIOD << 16) | (8 - 1)
    DELAY              = 180_000_000 / 6

DAT
    org 0
    asmclk

    dirl    #TRANSMISSION_PIN
    wrpin   ##(P_ASYNC_TX | P_OE), #TRANSMISSION_PIN
    wxpin   ##BAUD_MODE, #TRANSMISSION_PIN
    dirh    #TRANSMISSION_PIN
    wypin   #1, #TRANSMISSION_PIN

    call #PrintEven
    call #PrintOdd
    jmp #endprog

PrintEven
    mov number1, ##0
    .loop
        mov ascii_out, number1
        add ascii_out, #"0"
        wypin ascii_out, #TRANSMISSION_PIN
        call #flush
        add number1, ##2
        cmp number1, ##10 wc
        if_ge jmp #.endLoop
        jmp #.loop
    .endLoop
    ret

PrintOdd
    mov number1, ##1
    .loop
        mov ascii_out, number1
        add ascii_out, #"0"
        wypin ascii_out, #TRANSMISSION_PIN
        call #flush

```

```

        add number1, ##2
        cmp number1, ##11 wcz
        if_nc jmp #.endLoop
        jmp #.loop
    .endLoop
    ret

flush
    mov count, ##DELAY
    .delay
    sub count, #1 wz
    if_nz jmp #.delay
    ret

endprog
    jmp  $$

number1    res 1
ascii_out  res 1
count      res 1

```

We print, in this program, even and odd numbers sequentially as ASCII characters to the `TRANSMISSION_PIN`, with a short delay between each output. First, the `PrintEven` subroutine runs, initializing `number1` to 0 and incrementing by 2 each loop iteration until it reaches 10, producing the sequence 0, 2, 4, 6, 8. Next, the `PrintOdd` subroutine runs, starting `number1` at 1 and incrementing by 2 until it reaches 11, producing 1, 3, 5, 7, 9. Each value is converted to ASCII by adding "0" before sending it via `WYPIN`. The `flush` subroutine introduces a brief delay between transmissions. After both subroutines finish, the program ends, producing the full output sequence: 0 2 4 6 8 1 3 5 7 9.

9.4 Exercise

First, use `JMP` to skip a line that sets `number1` to 2 and instead set `number1` to 5, then print it as ASCII. Next, create a subroutine that sets `number1` to 7 and prints it, using `CALL` and `RET` to return to the main program. Within a subroutine, practice a local label loop that runs three times, incrementing `number1` by 1 and printing each value as ASCII.

Finally, create multiple subroutines: one that prints even numbers from 0 to 4 and another that prints odd numbers from 1 to 5 sequentially. Call each subroutine from the main program to reinforce structured, modular coding.

9.5 Summary

Having established structured program flow, we now consider temporal control mechanisms that allow synchronization between instruction execution and external events.

Chapter Summary

- Subroutines allow code reuse
- `call` jumps to a routine and returns
- Labels define code locations
- Execution can move between sections
- Helps organize and simplify programs

Chapter 10

Timing & Waits

Timing control in P2ASM is essential for synchronizing tasks, generating delays, and coordinating I/O operations. Programs can pause execution for a specific number of clock cycles or wait until an event occurs. Common techniques include using instructions for very short delays, creating subtraction loops with a counter register for longer or variable delays, and employing hardware timers or counters when precise or extended timing is required. A simple delay can be implemented by loading a register with a count value and decrementing it until it reaches zero, with the `CMP` instruction and `wcz` or `wc` flags used to test when the counter has finished.

Delays are typically measured in clock cycles, which can be converted to seconds or microseconds using the system clock frequency (`_CLKFREQ`). Timing control ensures proper synchronization for serial or I/O communication, provides consistent spacing between repeated signals such as LED blinking, and helps prevent race conditions in multitasking or sensor-reading loops. It is also essential for generating precise pulse widths, timeouts, or other time-sensitive control signals. By accurately managing delays, programs can maintain reliable operation and predictable behavior in hardware interactions.

10.1 WAITX Instruction

The `WAITX` instruction pauses the executing COG for a precise number of system clock cycles. `WAITX` provides exact timing without manually decrementing counters, making it simpler and more reliable. It is ideal for tasks that require precise timing, such as serial communication, pulse generation, and sensor reading.

`WAITX` accepts a delay value specifying the number of system clock cycles to pause execution. Once the delay completes, the COG immediately resumes execution. The delay value can range from 0 to 4,294,967,295, allowing both very short and very long pauses. Short delays from 0 to 511 cycles can be specified using the `#value` form, while longer delays require the `##value` form, which uses an extra instruction word to encode a full 32-bit number.

With `_CLKFREQ = 180,000,000`, each system clock cycle lasts approximately 5.56 nanoseconds. Using the full 32-bit range of `WAITX`, the maximum delay of 4,294,967,295 cycles corresponds to roughly 23.86 seconds, demonstrating the instruction's ability to provide both

very short and very long precise delays.

10.1.1 Syntax

WAITX {# or ##}value

- #value = waits 0–511 clock cycles
- ##value = waits 0–4,294,967,295 clock cycles

10.1.2 Program: 0029_waitx.spin2

Listing 10.1: Program: 0029_waitx.spin2

```
'waitx.spin2
CON
    _CLKFREQ          = 180_000_000
    BAUD_RATE         = 2_000_000
    TRANSMISSION_PIN  = 62
    BIT_PERIOD        = (_CLKFREQ / BAUD_RATE)
    BAUD_MODE         = (BIT_PERIOD << 16) | (8 - 1)
    DELAY             = 180_000_000

DAT
    org 0
    asmclk

    dirl    #TRANSMISSION_PIN
    wrpin   ##(P_ASYNC_TX | P_OE), #TRANSMISSION_PIN
    wxpin   ##BAUD_MODE, #TRANSMISSION_PIN
    dirh    #TRANSMISSION_PIN
    wypin   #1, #TRANSMISSION_PIN

    waitx   ##DELAY

    mov     character, #"A"
    wypin   character, #TRANSMISSION_PIN

    endprog
    jmp     #$

    character res 1
```

This program demonstrates a precise delay using WAITX. Execution begins with WAITX ##DELAY, which halts the COG for the specified number of system clock cycles (in this case, corresponding to a 1-second delay). After the pause, `mov character, #"A"` loads the ASCII code for the letter A into the variable `character`, and `wypin` transmits it via the TRANSMISSION_PIN. The program then ends with `endprog`.

10.2 Instruction: NOP (No Operation)

NOP stands for “No Operation.” It executes without performing any functional task but still consumes one system clock cycle, allowing execution to move to the next instruction. Each NOP cycle is deterministic, with a duration of $1/_CLKFREQ$ (for example, at 180 MHz, one NOP takes approximately 5.56 nanoseconds). Multiple NOPs can be executed in sequence to create slightly longer delays, though using many NOPs for extended timing becomes imprac-

tical—longer delays are better handled with **WAITX** or loop counters.

The **NOP** instruction is useful for introducing precise, single-cycle timing gaps between instructions and synchronizing events in timing-sensitive hardware tasks, such as bit-banging or pulse generation. Its simplicity and predictability make it ideal for minimal delays without adding overhead beyond the one clock cycle it consumes.

10.2.1 Syntax

`nop`

- Single-cycle delay

10.2.2 Program: 0030_nop.spin2

Listing 10.2: Program: 0030_nop.spin2

```
'nop.spin2
CON
    _CLKFREQ          = 180_000_000
    BAUD_RATE         = 2_000_000
    TRANSMISSION_PIN  = 62
    BIT_PERIOD        = (_CLKFREQ / BAUD_RATE)
    BAUD_MODE          = (BIT_PERIOD << 16) | (8 - 1)

DAT
    org 0
    asmclk

    dir1    #TRANSMISSION_PIN
    wrpin   ##(P_ASYNC_TX | P_OE), #TRANSMISSION_PIN
    wxpin   ##BAUD_MODE, #TRANSMISSION_PIN
    dirh    #TRANSMISSION_PIN
    wypin   #1, #TRANSMISSION_PIN

    nop

    mov character, #"A"
    wypin character, #TRANSMISSION_PIN

    endprog
    jmp    $$

    character res 1
```

The program starts with a single-cycle NOP, which performs no operation but consumes one system clock cycle. This creates an imperceptibly short delay before the next instruction. Immediately after, `MOV character, #"A"` loads the ASCII code for A into the `character` variable, and `WYPIN` transmits it through the `TRANSMISSION_PIN`. The program then ends with `endprog`. On a terminal, the letter A appears instantly, but the NOP demonstrates how a one-cycle delay can be inserted for precise timing control in a sequence of instructions.

10.3 GETCT / WAITCT1 / ADDCT1 Instruction

The `GETCT` instruction reads the current value of a counter/timer into a register, allowing your program to track the timer's progress without blocking execution. It is commonly used in combination with `ADDCT1` and `WAITCT1` to implement precise, non-blocking or scheduled timing operations.

The `ADDCT1` instruction adds a specified value to counter/timer 1 (CT1), allowing you to

schedule future events relative to the current counter value. It is typically used with **GETCT** and **WAITCT1** to create precise delays or periodic timing without manually recalculating absolute times.

The **WAITCT1** instruction pauses program execution until counter/timer 1 (CT1) reaches a specified value. As a blocking instruction, it stops the program until the counter event occurs, providing precise timing for periodic events and hardware-controlled delays.

10.3.1 Syntax

GETCT *target_register*, CT1

- *target_register* = register that receives the counter value
- CT1 = counter/timer 1
- Copies the current counter value into the target register
- Does not block or pause execution
- The counter continues running
- Commonly used with **WAITX** or time comparisons
- The value is a 32-bit system counter

ADDCT1 *value*

- *value* = number of clock cycles to add to counter/timer 1 (CT1)
- Used for scheduling future events based on the current counter

WAITCT1

- Waits until the value in the CT1 compare register is reached
- Automatically blocks the COG until timeout
- No loop or flag polling required

10.3.2 Program: 0031_waitct1.spin2

Listing 10.3: Program: 0031_waitct1.spin2

```

'waitct1.spin2
CON
    _CLKFREQ          = 180_000_000
    BAUD_RATE         = 2_000_000
    TRANSMISSION_PIN  = 62
    BIT_PERIOD        = (_CLKFREQ / BAUD_RATE)
    BAUD_MODE         = (BIT_PERIOD << 16) | (8 - 1)

DAT
    org 0
    asmclk

    dirl    #TRANSMISSION_PIN
    wrpin   ##(P_ASYNC_TX | P_OE), #TRANSMISSION_PIN
    wxpin   ##BAUD_MODE, #TRANSMISSION_PIN
    dirh    #TRANSMISSION_PIN
    wypin   #1, #TRANSMISSION_PIN

    mov     counter, #0
    mov     delay, ##_CLKFREQ
    mov     time_duration, #5

    sleep
    getct   timeout
    .loop
        addct1 timeout, delay
        waitct1
        add     counter, #1
        cmp     counter, time_duration wz
        if_nz   jmp     #.loop
    .end

    wypin   #"#", #TRANSMISSION_PIN

endprog
    jmp     $$

timeout      res 1
counter      res 1
delay        res 1
time_duration res 1

```

We initialize a seconds counter to 0, set a delay to `_CLKFREQ` (one second worth of clock cycles), and set `time_duration` to 5. The program reads the current CT value into `timeout`,

then repeatedly schedules the next timeout by adding `delay` to `timeout` and executes `WAITCT1` to block the COG until that scheduled moment. After each wait, the program increments `counter` and repeats until `counter` equals `time_duration`. When the loop finishes (after five precise one-second waits, ~5 seconds total), the program sends a single # on `TRANSMISSION_PIN`. The waits are hardware-timed and block the COG, ensuring exact one-second increments.

10.4 POLLCT1 Instruction (Non-blocking Timer Poll)

The `POLLCT1` instruction checks whether Counter Timer 1 (CT1) has reached its target value without stopping program execution. Unlike `WAITCT1`, which blocks until the timer expires, `POLLCT1` runs immediately and updates the Carry (C) flag to indicate whether the timer has passed its target. This non-blocking behavior allows the program to make conditional decisions based on timer status, enabling responsive and flexible control.

`POLLCT1` is useful for non-blocking delays, letting a program perform other tasks while monitoring a timer. It provides fine-grained control over timing loops and is especially valuable for multitasking, responsive event handling, or implementing timeouts without halting program flow.

10.4.1 Syntax

`POLLCT1`

- Typically used after scheduling a target with `ADDCT1`
- Often combined with conditional instructions like `IF_C` or `IF_NC` to loop until the timer expires

10.4.2 Program: 0032_pollct11.spin2

Listing 10.4: Program: 0032_pollct11.spin2

```
'pollct1.spin2
CON
    _CLKFREQ          = 180_000_000
    BAUD_RATE         = 2_000_000
    TRANSMISSION_PIN  = 62
    BIT_PERIOD        = (_CLKFREQ / BAUD_RATE)
    BAUD_MODE          = (BIT_PERIOD << 16) | (8 - 1)

DAT
    org 0
    asmclk

    dir1    #TRANSMISSION_PIN
    wrpin   ##(P_ASYNC_TX | P_OE), #TRANSMISSION_PIN
    wxpin   ##BAUD_MODE, #TRANSMISSION_PIN
    dirh    #TRANSMISSION_PIN
    wypin   #1, #TRANSMISSION_PIN

    mov counter, #0
    mov delay, ##_CLKFREQ
    mov time_duration, #5

    sleep
        getct timeout
    .loop
        addct1 timeout, delay
    .wait
        pollct1
        if_c jmp #.wait
        add counter, #1
        cmp counter, time_duration wz
        if_nz jmp #.loop
    .end

    wypin #"#", #TRANSMISSION_PIN

endprog
    jmp $$

timeout          res 1
counter          res 1
delay            res 1
time_duration    res 1
```

We measure a five-second interval using non-blocking timer checks. The program begins by initializing a counter to 0 and a one-second delay value, then reads the current CT value into `timeout`. In the main loop, it schedules a future timeout using `ADDCT1`, based on the current CT value. The inner `.wait` loop repeatedly polls the timer with `POLLCT1` until the scheduled moment is reached, without blocking the COG, and sets the C flag when the timeout occurs. After each timeout, the counter increments. This process repeats until the counter reaches five, corresponding to five precise one-second intervals. When the loop finishes, the program outputs a # character on the `TRANSMISSION_PIN`, signaling that the full timed interval has completed.

10.5 WAITX vs NOP

Both `NOP` and `WAITX` can be used to introduce delays, but they differ significantly in efficiency and practicality. A `NOP` instruction consumes exactly two clock cycles while performing no useful work, making it suitable only for extremely short, single-cycle timing adjustments. In contrast, `WAITX` can stall for an arbitrary number of cycles, allowing multi-cycle delays to be handled cleanly and efficiently without wasting instruction space. Choosing the appropriate instruction for the situation is key to writing fast, memory-efficient, and maintainable code.

`NOP` is useful for very small timing adjustments, such as cycle tweaks or padding within tightly tuned instruction sequences. For longer or more precise delays, `WAITX` is the better choice, providing cycle-accurate pauses without bloating the program with repeated instructions. Long sequences of `NOPs` are impractical for meaningful delays, as even millisecond-scale pauses can require hundreds of thousands of instructions, wasting time, space, and stressing the COG's LUT cache and hub execution. They also make code harder to read and obscure intent. In contrast, `WAITX` allows flexible, variable-length, and precisely controlled delays, keeping code efficient, readable, and easy to maintain.

10.5.1 Program: 0033_delay.spin2

Listing 10.5: Program: 0033_delay.spin2

```

'delay.spin2
CON
    _CLKFREQ          = 180_000_000
    BAUD_RATE         = 2_000_000
    TRANSMISSION_PIN  = 62
    BIT_PERIOD        = (_CLKFREQ / BAUD_RATE)
    BAUD_MODE          = (BIT_PERIOD << 16) | (8 - 1)

DAT
    org 0
    asmclk

    dirl    #TRANSMISSION_PIN
    wrpin   ##(P_ASYNC_TX | P_OE), #TRANSMISSION_PIN
    wxpin   ##BAUD_MODE, #TRANSMISSION_PIN
    dirh    #TRANSMISSION_PIN
    wypin   #1, #TRANSMISSION_PIN

    nop
    nop
    nop

    mov character, #"A"
    wypin character, #TRANSMISSION_PIN

    waitx   ##_CLKFREQ

    mov character, #"B"
    wypin character, #TRANSMISSION_PIN

    endprog
    jmp     $$

    character res 1

```

Let's show two delay methods and their visible effect: the three consecutive NOP instructions introduce a tiny, three-cycle pause before transmitting 'A', which is effectively instantaneous to a human observer (~ 50 ns at 180 MHz). The program then places the ASCII character "A" into `character` and outputs it on the transmission pin. Next, `WAITX ##_CLKFREQ` performs a multi-cycle stall equal to one second ($\approx$ `_CLKFREQ` cycles at 180 MHz), starting from the current COG clock, after which the program stores "B" into `character` and transmits it, producing "B" exactly one second after the `WAITX` instruction begins. The three NOPs simply provide a minimal timing offset, and the line `character res 1` reserves

the single-byte storage used for both transmissions.

10.6 Exercise

Practice timing and delays in P2ASM with this exercise. Start by inserting three consecutive NOP instructions, then set a character to "X" and print it. Next, use `WAITX #50` for a short pause before setting a character to "Y" and printing it, followed by a longer delay using `WAITX ##_CLKFREQ*2` to pause for two seconds, then set a character to "Z" and print it. For a more advanced exercise, create a 3-second loop with `GETCT`, `ADDCT1`, and `WAITCT1` that increments a counter each second before printing "#". Finally, use `POLLCT1` in a non-blocking loop to check CT1 every second for four seconds while incrementing a counter, then print "@", reinforcing both blocking and non-blocking timing techniques.

10.7 Summary

We now transition from general instruction control to hardware-specific functionality, where the Propeller 2's internal systems are directly engaged through specialized instructions.

Chapter Summary

- Timing is controlled using clock cycles
- `waitx` pauses execution for a duration
- `waitct` enables precise timing using counters
- `nop` provides minimal delay
- Accurate timing depends on the system clock

Chapter 11

Hardware Functions

The Propeller 2 microcontroller features a built-in hardware math engine called CORDIC (COordinate Rotation DIgital Computer), which performs complex arithmetic operations far faster than software routines. Using a shift-and-add algorithm, CORDIC efficiently computes trigonometric, vector, and other arithmetic functions, reducing code size by replacing long instruction sequences with a single hardware operation. Its deterministic timing makes it ideal for real-time applications, and it supports high-precision fractional math without requiring floating-point hardware, enabling efficient, compact, and predictable calculations on the Propeller 2.

11.1 QMUL — Unsigned Multiplication Using CORDIC

The QMUL instruction leverages the Propeller 2's CORDIC hardware to perform unsigned multiplication directly in hardware using a shift-and-add approach. The hardware shifts one operand and adds it conditionally to an accumulating sum, efficiently computing the product in just a few cycles. This method executes far faster than equivalent software routines, conserves clock cycles and program memory, and provides precise, deterministic results for unsigned values. QMUL also integrates seamlessly with other CORDIC operations, such as division or square root, enabling flexible and efficient arithmetic workflows on the Propeller 2.

To use QMUL, first load the two unsigned values into normal registers, for example: `MOV number1, ##3` and `MOV number2, ##2`. Then execute `QMUL source1, source2` to instruct the CORDIC hardware to perform the multiplication; note that the source registers themselves are not modified. Finally, retrieve the computed product from the CORDIC output register QX using `GETQX destination`, which stores the result in a general-purpose register or memory location.

11.1.1 Syntax

`QMUL source1, source2`

- `source1` = register or literal value

- `source2` = register or literal value

GETQX destination

- `destination` = register, variable, or memory location where the result is stored
- Retrieves the computed value from the CORDIC output queue

11.1.2 Program: 0034_qmul.spin2

Listing 11.1: Program: 0034_qmul.spin2

```

'qmul.spin2
CON
    _CLKFREQ          = 180_000_000
    BAUD_RATE         = 2_000_000
    TRANSMISSION_PIN  = 62
    BIT_PERIOD        = (_CLKFREQ / BAUD_RATE)
    BAUD_MODE         = (BIT_PERIOD << 16) | (8 - 1)

DAT
    org 0
    asmclk

    dir1    #TRANSMISSION_PIN
    wrpin   ##(P_ASYNC_TX | P_OE), #TRANSMISSION_PIN
    wxpin   ##BAUD_MODE, #TRANSMISSION_PIN
    dirh    #TRANSMISSION_PIN
    wypin   #1, #TRANSMISSION_PIN

    mov     number1, #3
    mov     number2, #2

    qmul     number1, number2
    getqx    number1

    add     number1, #"0"
    wypin   number1, #TRANSMISSION_PIN

endprog
    jmp     $$

number1 res 1
number2 res 1

```

Here we multiply 3 by 2 using the Propeller 2's **QMUL** instruction. First, the numbers are loaded into **number1** and **number2**. Executing **QMUL number1, number2** multiplies the two 32-bit signed values, storing the high 32 bits of the 64-bit product in the **QX** register, which is then retrieved into **number1** using **GETQX**. In this specific case, the numbers are small, so **QX** contains the exact product. The program converts the numeric result 6 to its ASCII equivalent by adding "0"—note that this approach works only for single-digit results—and transmits it via the transmission pin, producing the character "6" as output. The variables **number1** and **number2** each reserve a single byte of storage for this operation. This calculation and transmission occur immediately and non-blocking, unlike **WAITX** or **WAITCT1** delays.

11.2 QDIV — Unsigned Division Using CORDIC

The QDIV instruction uses the Propeller 2's CORDIC hardware to perform unsigned integer division efficiently. It computes `source1 ÷ source2` far faster than a software routine, eliminating the need for loops or long instruction sequences. Division is achieved using a shift-and-subtract approach, where the numerator is shifted and compared to the denominator in successive steps, with the quotient built incrementally. This hardware-based method ensures precise, deterministic results while saving program memory and clock cycles.

To use QDIV, load the numerator and denominator into registers (for example, `MOV number1, ##6` for the numerator and `MOV number2, ##3` for the denominator), then execute `QDIV source1, source2`. The CORDIC engine performs the division, and the quotient is retrieved using `GETQX dest`; the remainder can be fetched with `GETQY dest` if needed.

11.2.1 Syntax

```
QDIV source1, source2
    GETQX destination
    GETQY destination
```

- `source1` = register or literal value, used as the numerator
- `source2` = register or literal value, used as the denominator
- `destination` = register, variable, or memory location where the result is stored
- Use `GETQX destination` to retrieve the quotient
- Use `GETQY destination` to retrieve the remainder (optional)

11.2.2 Program: 0035_qdiv.spin2

Listing 11.2: Program: 0035_qdiv.spin2

```
'qdiv.spin2
CON
    _CLKFREQ          = 180_000_000
    BAUD_RATE         = 2_000_000
    TRANSMISSION_PIN  = 62
    BIT_PERIOD        = (_CLKFREQ / BAUD_RATE)
    BAUD_MODE         = (BIT_PERIOD << 16) | (8 - 1)

DAT
    org 0
    asmclk

    dir1    #TRANSMISSION_PIN
    wrpin   ##(P_ASYNC_TX | P_OE), #TRANSMISSION_PIN
    wxpin   ##BAUD_MODE, #TRANSMISSION_PIN
    dirh    #TRANSMISSION_PIN
    wypin   #1, #TRANSMISSION_PIN

    mov     number1, #6
    mov     number2, #3

    qdiv    number1, number2
    getqx   number1

    add     number1, #"0"
    wypin   number1, #TRANSMISSION_PIN

endprog
    jmp    #$

number1 res 1
number2 res 1
```

The QDIV instruction used here divides 6 by 3. The numerator and denominator are first loaded into `number1` and `number2`. Executing `QDIV number1, number2` directs the CORDIC hardware to perform the division, producing a 32-bit quotient stored in the QX register, which is then retrieved into `number1` using `GETQX`. In this case, the numbers divide evenly, so the quotient is exact. The program converts the numeric result 2 to its ASCII equivalent by adding "0"—note that this approach works correctly only for single-digit results—and transmits it via the transmission pin, producing the character "2" as output. The variables `number1` and `number2` each reserve a single byte of storage for this operation. This calculation and transmission occur immediately and non-blocking, unlike `WAITX` or `WAITCT1` delays.

11.3 Exercise

With this exercise, start by using `QMUL` to multiply 4×5 and print the result. Next, perform unsigned division with `QDIV` by dividing $8 \div 2$ and printing the quotient.

11.4 Summary

To support coordinated execution across the system, we now examine hub memory architecture and the mechanisms by which data is accessed and shared between processing units.

Chapter Summary

- Hardware math (CORDIC) accelerates operations
- `qmul` and `qdiv` perform fast calculations
- Results use special registers
- Hardware functions improve performance
- Useful for advanced math operations

Chapter 12

Hub Memory Data Access

Hub RAM is the Propeller 2's main shared memory, used to store larger programs and data accessible by all COGs. P2ASM provides instructions for reading and writing 8-bit, 16-bit, or 32-bit values, allowing you to manipulate data safely and efficiently. These instructions ensure precise memory access without accidentally modifying nearby values, making them especially useful for working with buffers, arrays, or tables.

Hub memory access also includes instructions designed to handle Hub RAM safely and precisely. They prevent errors that can occur if the wrong data size is read or written, which could otherwise corrupt adjacent values. These instructions are especially important when working with structured programs that use arrays, buffers, or other shared data.

12.1 Variables in P2ASM

Variables are placeholders in memory that store data your program can read or modify. Each variable has three important parts: a name, which is the label used to access it in code; a type, which defines the size and format of the data (bit, byte, word, or long); and an optional initial value, which sets the starting contents of the variable.

Memory alignment determines how data is arranged in memory for efficient access. Single bytes can be stored at any address, but words (2 bytes) should begin at even addresses, and longs (4 bytes) must begin at addresses divisible by 4. Following these alignment rules ensures correct and efficient CPU access. When referencing a specific memory location, using `##@label` ensures the address is treated as a full 32-bit value.

12.1.1 Example

Listing 12.1: Variable Definitions and Alignment Example

```
character byte "A", 0, 0, 0
number1   word 0, 0
number2   long 0
```

12.1.2 Program: 0036_variables.spin2

Listing 12.2: Program: 0036_variables.spin2

```
'variables.spin2
CON
    _CLKFREQ          = 180_000_000
    BAUD_RATE         = 2_000_000
    TRANSMISSION_PIN  = 62
    BIT_PERIOD        = (_CLKFREQ / BAUD_RATE)
    BAUD_MODE          = (BIT_PERIOD << 16) | (8 - 1)

DAT
    org 0
    asmclk

    dirl    #TRANSMISSION_PIN
    wrpin   ##(P_ASYNC_TX | P_OE), #TRANSMISSION_PIN
    wxpin   ##BAUD_MODE, #TRANSMISSION_PIN
    dirh    #TRANSMISSION_PIN
    wypin   #1, #TRANSMISSION_PIN

    wypin character, #TRANSMISSION_PIN
    waitx #511

    mov number1, #1
    add number1, #"0"
    wypin number1, #TRANSMISSION_PIN
    waitx #511

    mov number2, #2
    add number2, #"0"
    wypin number2, #TRANSMISSION_PIN

    endprog
    jmp    $$

    character byte "A", 0, 0, 0
    number1    word 0, 0
    number2    long 0
```

We send three characters—"A", "1", and "2"—out through the transmission pin using the WYPIN instruction. First, the program transmits the **character** variable, which holds "A" (not "0"). It then loads the numbers 1 and 2 into **number1** (a 2-byte word) and **number2** (a 4-byte long), converts them into their ASCII characters by adding "0", and sends them as well. Short delays (WAITX #511) separate each transmission; these delays are very short and produce small but noticeable spacing. The variable definitions at the end use BYTE

(`character`), `WORD` (`number1`), and `LONG` (`number2`) types, demonstrating different sizes, ensuring proper alignment, and allowing reliable memory access.

12.2 RDBYTE

`RDBYTE` reads a single byte (8 bits) from Hub RAM and copies it into a register so the CPU can manipulate it. Since the CPU cannot access Hub memory directly, this instruction is essential for working with data stored outside the COG's local memory. `RDBYTE` allows programs to safely inspect, modify, or use values such as characters in a string, entries in a byte buffer, or small numerical variables. By transferring a single byte at a time into a register, `RDBYTE` ensures precise memory access without affecting adjacent data, making it a fundamental tool for handling hub-stored data in P2ASM programs.

12.2.1 Syntax

`RDBYTE` *destination*, *address*

- *destination* — the COG register where the byte will be stored
- *address* — the Hub memory address of the byte to read
- Use `@label` to get the address of a label, and `##@label` for the full 32-bit address

12.2.2 Program: 0037_rdbyte.spin2

Listing 12.3: Program: 0037_rdbyte.spin2

```
'rdbyte.spin2
CON
    _CLKFREQ          = 180_000_000
    BAUD_RATE         = 2_000_000
    TRANSMISSION_PIN  = 62
    BIT_PERIOD        = (_CLKFREQ / BAUD_RATE)
    BAUD_MODE          = (BIT_PERIOD << 16) | (8 - 1)

DAT
    org 0
    asmclk

    dir1    #TRANSMISSION_PIN
    wrpin   ##(P_ASYNC_TX | P_OE), #TRANSMISSION_PIN
    wxpin   ##BAUD_MODE, #TRANSMISSION_PIN
    dirh    #TRANSMISSION_PIN
    wypin   #1, #TRANSMISSION_PIN

    rdbyte  data_byte, ##@character
    wypin   data_byte, #TRANSMISSION_PIN

    endprog
    jmp     $$

    character byte "A", 0, 0, 0
    data_byte      res 1
```

We read a single byte from the `character` variable in Hub RAM into the COG register `data_byte` using `RDBYTE`. The value stored is the character "A". The program then immediately transmits that byte via the transmission pin using `WYPIN`. The variable `character` is defined as a 4-byte sequence ("A", 0, 0, 0), but only the first byte is read and sent. `data_byte` is a one-byte COG register space reserved to hold the read value. This transfer from Hub RAM to the COG register occurs non-blocking, and the transmission happens immediately according to the UART timing. As a result, the character "A" is transmitted.

12.3 WRBYTE

`WRBYTE` writes a single byte (8 bits) from a register into Hub RAM, allowing the CPU to update shared memory safely. Since the CPU cannot modify Hub memory directly, `WRBYTE` is used to store processed values back into Hub RAM after calculations or modifications have been made in a register. It is essential for updating byte-sized data such as characters, flags, or entries in buffers and arrays.

WRBYTE works hand-in-hand with **RDBYTE**: first, a byte is read from Hub memory into a register, then processed, and finally written back using **WRBYTE**. This makes it the fundamental instruction for managing byte-level data in shared memory. By enabling precise, controlled updates, **WRBYTE** ensures that individual bytes can be modified without affecting neighboring memory, which is crucial when working with strings, buffers, or other shared data structures in P2ASM programs.

12.3.1 Syntax

WRBYTE *source*, *address*

- *source* — register holding the byte to write
- *address* — Hub memory address where the byte will be stored
- Use `@label` for labels and `##@label` for full 32-bit addresses

12.3.2 Program: 0038_wrbyte.spin2

Listing 12.4: Program: 0038_wrbyte.spin2

```

'wrbyte.spin2
CON
    _CLKFREQ          = 180_000_000
    BAUD_RATE         = 2_000_000
    TRANSMISSION_PIN  = 62
    BIT_PERIOD        = (_CLKFREQ / BAUD_RATE)
    BAUD_MODE          = (BIT_PERIOD << 16) | (8 - 1)

DAT
    org 0
    asmclk

    dirl    #TRANSMISSION_PIN
    wrpin   ##(P_ASYNC_TX | P_OE), #TRANSMISSION_PIN
    wxpin   ##BAUD_MODE, #TRANSMISSION_PIN
    dirh    #TRANSMISSION_PIN
    wypin   #1, #TRANSMISSION_PIN

    mov data_byte, #"B"
    wrbyte data_byte, ##@character

    rdbyte data_byte, ##@character
    wypin data_byte, #TRANSMISSION_PIN

endprog
    jmp    #$

character byte "A", 0, 0, 0
data_byte      res 1

```

The character "B" is written into the Hub-memory variable `character` using `WRBYTE`. First, the program loads "B" into the COG register `data_byte`, then writes that byte to the address of `character`, replacing the original "A". Next, the program reads the updated value back from Hub RAM into `data_byte` using `RDBYTE` and immediately transmits it via the transmission pin using `WYPIN`. The variable `character` is defined as a 4-byte sequence ("A", 0, 0, 0), but only the first byte is written and read. `data_byte` is a one-byte COG register space reserved for this operation. This hub-to-COG transfer and transmission occur non-blocking, and as a result, the transmitted character is "B".

12.4 RDBYTE Looping

RDBYTE reads a single byte (8 bits) from Hub RAM into a register, and although it operates on one byte at a time, it can be used within a loop to access multiple bytes in sequence. This makes it ideal for processing strings, buffers, arrays, or other collections of byte-sized data stored in Hub memory. By combining RDBYTE with a pointer and a loop, your program can efficiently step through each memory address, reading bytes one after another, often stopping when a terminator such as 0 is encountered.

When paired with WRBYTE, this approach allows not only reading but also updating or transmitting data efficiently, making sequential Hub memory access both flexible and powerful. Hub memory is organized one byte at a time, but using a pointer or label like `##@label` ensures that each address is treated correctly. This combination of RDBYTE, pointers, and loops provides a robust method for handling strings, buffers, and other sequential byte-sized structures in Propeller 2 programs.

12.4.1 Program: 0039_multi_rdbyte.spin2

Listing 12.5: Program: 0039_multi_rdbyte.spin2

```
'multi_rdbyte.spin2
CON
    _CLKFREQ          = 180_000_000
    BAUD_RATE         = 2_000_000
    TRANSMISSION_PIN  = 62
    BIT_PERIOD        = (_CLKFREQ / BAUD_RATE)
    BAUD_MODE          = (BIT_PERIOD << 16) | (8 - 1)
    DELAY              = 180_000_000

DAT
    org 0
    asmclk

    dirl    #TRANSMISSION_PIN
    wrpin   ##(P_ASYNC_TX | P_OE), #TRANSMISSION_PIN
    wxpin   ##BAUD_MODE, #TRANSMISSION_PIN
    dirh    #TRANSMISSION_PIN
    wypin   #1, #TRANSMISSION_PIN

    mov char_ptr, ##@character

    read_loop
        rdbyte current_char, char_ptr
        cmp    current_char, #0 wz
        if_z   jmp #done
        wypin  current_char, #TRANSMISSION_PIN
        waitx  ##DELAY
        add    char_ptr, #1
        jmp    #read_loop
    done

    endprog
        jmp   #$

    character    byte "A", "B", "C", "D", 0
    char_count   res 1
    current_char res 1
    char_ptr     res 1
```

This program reads and transmits a string of characters stored in Hub memory. First, `char_ptr` is set to point to the start of the character array, which contains "A", "B", "C", "D", followed by a 0 terminator. Inside the loop, `RDBYTE` reads one byte at a time from Hub RAM into the COG register `current_char`. If the byte is 0, the loop ends. Otherwise,

the program immediately transmits the byte via the transmission pin using `WYPIN`, waits briefly (`WAITX ##DELAY`), and then increments `char_ptr` to point to the next byte. This process repeats until the 0 terminator is reached, transmitting the entire string in order. All Hub-to-COG transfers and transmissions occur non-blocking. The character array is stored as consecutive bytes in Hub RAM, while `char_ptr` and `current_char` are single-byte COG registers reserved for the pointer and read value, respectively.

12.5 RDBYTE for Strings

A string in Propeller 2 assembly is a sequence of characters stored in memory, with each character occupying 1 byte. Strings are typically null-terminated, ending with a 0, so the program can determine where the string ends. `RDBYTE` can be used to read each character from Hub RAM into a register, and by combining it with a pointer and a loop, your program can process the string one character at a time. The loop continues reading bytes until it encounters the null terminator, ensuring that only valid characters are processed. This method makes `RDBYTE` an essential tool for handling strings efficiently, allowing programs to inspect, modify, or output characters sequentially while working safely with Hub memory.

12.5.1 Program: 0040_rdbyte_strings.spin2

Listing 12.6: Program: 0040_rdbyte_strings.spin2

```
'rdbyte_strings.spin2
CON
    _CLKFREQ          = 180_000_000
    BAUD_RATE         = 2_000_000
    TRANSMISSION_PIN  = 62
    BIT_PERIOD        = (_CLKFREQ / BAUD_RATE)
    BAUD_MODE          = (BIT_PERIOD << 16) | (8 - 1)
    DELAY              = 180_000_000 / 100

DAT
    org 0
    asmclk

    dirl    #TRANSMISSION_PIN
    wrpin   ##(P_ASYNC_TX | P_OE), #TRANSMISSION_PIN
    wxpin   ##BAUD_MODE, #TRANSMISSION_PIN
    dirh    #TRANSMISSION_PIN
    wypin   #1, #TRANSMISSION_PIN

    mov char_ptr, ##@character

    read_loop
        rdbyte current_char, char_ptr
        cmp    current_char, #0 wz
        if_z   jmp #done
        wypin  current_char, #TRANSMISSION_PIN
        waitx  ##DELAY
        add    char_ptr, #1
        jmp    #read_loop
    done

    endprog
    jmp    #$

character      byte "Hello assembly!", 0
char_count    res 1
current_char   res 1
char_ptr       res 1
```

The program begins by loading the Hub address of the string `character` into the pointer `char_ptr`. It then reads one byte at a time from Hub RAM into the COG register `current_char` using `RDBYTE`. Each byte is checked against the null terminator (0); if it matches, the loop ends. Otherwise, the byte is immediately transmitted via the transmission pin using `WYPIN`,

and the program waits briefly (`WAITX ##DELAY`) before incrementing `char_ptr` to point to the next byte. This process repeats, reading and transmitting each character in order until the null terminator is reached. The character array is stored consecutively in Hub RAM, while `char_ptr` and `current_char` are single-byte COG registers reserved for the pointer and read value, respectively. As a result, the string "Hello assembly!" is transmitted character by character. All Hub-to-COG transfers and transmissions occur non-blocking, with precise timing dictated by the `WAITX` delay.

12.6 RDWORD

`RDWORD` reads a 16-bit word (2 bytes) from Hub RAM into a register, allowing the CPU to work with the value safely since it cannot access Hub memory directly. When using `RDWORD`, the Hub memory address must be word-aligned, pointing to the correct 2-byte boundary. You can reference the address with `@label` and use `##` to ensure full 32-bit encoding.

Once the 16-bit word is loaded into a register, it can be processed, transmitted, or written back to Hub memory using `WRWORD`. `RDWORD` is especially useful for reading numerical data stored in Hub RAM, such as word-sized variables in buffers, arrays, or other data structures. By providing safe, precise access to 16-bit values, `RDWORD` simplifies handling larger or structured data in Propeller 2 programs.

12.6.1 Syntax

`RDWORD destination, address`

- `destination` = COG register where the 16-bit word will be stored
- `address` = Hub memory address of the word
- Use `##@label` to reference a label's Hub address

12.6.2 Program: 0041_rdword.spin2

Listing 12.7: Program: 0041_rdword.spin2

```
'rdword.spin2
CON
    _CLKFREQ          = 180_000_000
    BAUD_RATE         = 2_000_000
    TRANSMISSION_PIN  = 62
    BIT_PERIOD        = (_CLKFREQ / BAUD_RATE)
    BAUD_MODE         = (BIT_PERIOD << 16) | (8 - 1)

DAT
    org 0
    asmclk

    dirl    #TRANSMISSION_PIN
    wrpin   ##(P_ASYNC_TX | P_OE), #TRANSMISSION_PIN
    wxpin   ##BAUD_MODE, #TRANSMISSION_PIN
    dirh    #TRANSMISSION_PIN
    wypin   #1, #TRANSMISSION_PIN

    rdword  number_value, ##@number1
    add     number_value, #"0"
    wypin   number_value, #TRANSMISSION_PIN

    endprog
    jmp     #$

    number1      word 6, 0
    number_value  res 1
```

Our program reads a 16-bit word from the Hub-memory variable `number1` into the COG register `number_value` using `RDWORD`. In this case, `number1` is defined as a 16-bit word containing the value 6. The program then converts the numeric value to its ASCII equivalent by adding "0" and immediately transmits it via the transmission pin using `WYPIN`. `number_value` is a one-byte COG register reserved to hold the read value. The `RDWORD` instruction transfers the 16-bit word from Hub RAM to the COG register non-blocking, and the transmission occurs immediately. As a result, the character "6" is output.

12.7 WRWORD

`WRWORD` writes a 16-bit word (2 bytes) from a register into Hub RAM, allowing the CPU to safely update memory since it cannot modify Hub RAM directly. When using `WRWORD`, the Hub memory address must be word-aligned, meaning it should be divisible by 2. Use `@label` to reference the Hub address and `##` to ensure full 32-bit encoding.

WRWORD is commonly paired with RDWORD to read, process, and write numerical data, buffers, or other word-sized values in Hub memory. By enabling safe, precise updates of 16-bit variables and structured data, WRWORD simplifies working with larger programs and shared memory in Propeller 2 assembly.

12.7.1 Syntax

WRWORD *source*, *address*

- *source* = register containing the 16-bit value to store
- *address* = Hub memory address where the word will be written
- Use `##@label` for full 32-bit Hub addresses

12.7.2 Program: 0042_wrword.spin2

Listing 12.8: Program: 0042_wrword.spin2

```
'wrword.spin2
CON
    _CLKFREQ      = 180_000_000
    BAUD_RATE     = 2_000_000
    TRANSMISSION_PIN = 62
    BIT_PERIOD    = (_CLKFREQ / BAUD_RATE)
    BAUD_MODE     = (BIT_PERIOD << 16) | (8 - 1)

DAT
    org 0
    asmclk

    dirl    #TRANSMISSION_PIN
    wrpin   ##(P_ASYNC_TX | P_OE), #TRANSMISSION_PIN
    wxpin   ##BAUD_MODE, #TRANSMISSION_PIN
    dirh    #TRANSMISSION_PIN
    wypin   #1, #TRANSMISSION_PIN

    mov     number_value, ##7
    wrword  number_value, ##@value

    rdword  number_value, ##@value
    add number_value, #"0"
    wypin   number_value, #TRANSMISSION_PIN

    endprog
    jmp    #$.

value      word 0
number_value res 1
```

We are writing a 16-bit number into the Hub-memory variable `value` using `WRWORD`. First, the number 7 is loaded into the COG register `number_value` and then written to the address of `value` in Hub RAM. The program then reads the updated 16-bit word back from Hub RAM into `number_value` using `RDWORD`, converts it to its ASCII equivalent by adding "0", and immediately transmits it via the transmission pin using `WYPIN`. The variable `value` is defined as a 16-bit word in Hub RAM, and `number_value` is a one-byte COG register reserved to hold the value. The Hub-to-COG transfers and transmission occur non-blocking. As a result, the character "7" is output.

12.8 RDLONG

RDLONG reads a 32-bit long value (4 bytes) from Hub RAM into a register, allowing the CPU to safely process data it cannot access directly in Hub memory. The Hub address must be long-aligned, meaning divisible by 4, and can be referenced using `@label` with `##` for full 32-bit encoding. **RDLONG** is part of the basic Hub memory transfer instructions, alongside **RDBYTE** and **RDWORD**.

This instruction is ideal for accessing 32-bit numerical data stored in buffers, arrays, tables, or other long-sized variables. Once transferred into a register, the program can safely manipulate, process, or transmit the value without directly touching Hub RAM. **RDLONG** is often paired with **WRLONG** to enable efficient reading and writing of 32-bit values, making it a key tool for handling larger or structured data in Propeller 2 programs.

12.8.1 Syntax

RDLONG *destination*, *address*

- *destination* = register to store the 32-bit value
- *address* = Hub memory address of the long
- Use `##@label` for full 32-bit Hub addresses

12.8.2 Program: 0043_rdlong.spin2

Listing 12.9: Program: 0043_rdlong.spin2

```
'rdlong.spin2
CON
    _CLKFREQ          = 180_000_000
    BAUD_RATE         = 2_000_000
    TRANSMISSION_PIN  = 62
    BIT_PERIOD        = (_CLKFREQ / BAUD_RATE)
    BAUD_MODE          = (BIT_PERIOD << 16) | (8 - 1)

DAT
    org 0
    asmclk

    dir1    #TRANSMISSION_PIN
    wrpin   ##(P_ASYNC_TX | P_OE), #TRANSMISSION_PIN
    wxpin   ##BAUD_MODE, #TRANSMISSION_PIN
    dirh    #TRANSMISSION_PIN
    wypin   #1, #TRANSMISSION_PIN

    rdlong  number_value, ##@number1
    add number_value, #"0"
    wypin  number_value, #TRANSMISSION_PIN

    endprog
    jmp    #$

    number1      long 3
    number_value res 1
```

Our code reads a 32-bit value from the Hub-memory variable `number1` into the COG register `number_value` using `RDLONG`. In this case, `number1` is defined as a 32-bit long containing the value 3. The program then converts the numeric value to its ASCII equivalent by adding "0" and immediately transmits it via the transmission pin using `WYPIN`. The `number_value` register is a one-byte COG register reserved to hold the read value. The transfer from Hub RAM to the COG register is non-blocking, and the transmission occurs immediately. As a result, the character "3" is output.

12.9 WRLONG

`WRLONG` writes a 32-bit long value (4 bytes) from a register into Hub RAM, allowing the CPU to safely store processed data since it cannot modify Hub memory directly. It is part of the basic Hub memory transfer instructions, along with `WRBYTE` and `WRWORD`, and is commonly used to update long-sized variables, buffers, or numerical data.

When using `WRLONG`, the Hub memory address must be long-aligned, meaning divisible by 4. Use `@label` to reference the location and `##` to ensure the assembler encodes the full 32-bit address. Following these rules guarantees that 32-bit values are stored correctly and can be read back safely with `RDLONG`.

`WRLONG` is ideal for saving large numerical variables, updating buffers, storing counters, configuration values, lookup tables, or any other 32-bit data that needs to persist in shared memory for later use.

12.9.1 Syntax

`WRLONG source, address`

- `source` = register containing the 32-bit value to store
- `address` = Hub memory address to write the value
- Use `##@label` for full 32-bit addresses

12.9.2 Program: 0044_wrlong.spin2

Listing 12.10: Program: 0044_wrlong.spin2

```
'wrlong.spin2
CON
    _CLKFREQ      = 180_000_000
    BAUD_RATE     = 2_000_000
    TRANSMISSION_PIN = 62
    BIT_PERIOD    = (_CLKFREQ / BAUD_RATE)
    BAUD_MODE     = (BIT_PERIOD << 16) | (8 - 1)

DAT
    org 0
    asmclk

    dirl    #TRANSMISSION_PIN
    wrpin   ##(P_ASYNC_TX | P_OE), #TRANSMISSION_PIN
    wxpin   ##BAUD_MODE, #TRANSMISSION_PIN
    dirh    #TRANSMISSION_PIN
    wypin   #1, #TRANSMISSION_PIN

    mov     number_value, #5
    wrlong  number_value, ##@number1

    rdlong  number_value, ##@number1
    add number_value, #"0"
    wypin   number_value, #TRANSMISSION_PIN

endprog
    jmp    #$

number1      long 0
number_value res 1
```

A 32-bit value is assigned into the Hub-memory variable `number1` using `WRLONG`. First, the number 5 is loaded into the COG register `number_value` and then written to the address of `number1` in Hub RAM. The program then reads the 32-bit value back from Hub RAM into `number_value` using `RDLONG`, converts it to its ASCII equivalent by adding "0", and immediately transmits it via the transmission pin using `WYPIN`. The variable `number1` is defined as a 32-bit long in Hub RAM, and `number_value` is a one-byte COG register reserved to hold the value. Both the Hub-to-COG transfer and transmission occur non-blocking. As a result, the character "5" is transmitted.

12.10 Exercise

Practice reading and writing data in Hub RAM with this exercise.

- Start by writing the value `0x55` into the third byte of a buffer using `WRBYTE`.
- Next, load a 16-bit word from Hub memory, add 5 to it, and store it back in the same location using `RDWORD` and `WRWORD`.
- Then, load a 32-bit long value from Hub RAM and transmit it through a smart pin, reinforcing the use of `RDLONG`.
- Finally, copy a 32-bit value from one long variable to another in Hub memory using `RDLONG` and `WRLONG`, giving practice with safe and efficient handling of byte-, word-, and long-sized data in Propeller 2 assembly.

12.11 Summary

Before advancing further into system-level control, we establish a formal understanding of numeric representation within Propeller 2 assembly, including signedness and binary interpretation.

Chapter Summary

- Hub memory is shared across all cogs
- `rdbyte`, `rdword`, and `rdlong` read memory
- `wrbyte`, `wrword`, and `wrlong` write memory
- Hub access is slower than cog memory
- Used for communication and data storage

Chapter 13

How Numbers Work in P2 Assembly

Numbers are stored in binary inside registers or hub memory. Before doing math or printing values, it is important to understand how these numbers are represented. The Propeller 2 works with both unsigned and signed integers, and it can store values in different sizes such as bits, bytes, words, and longs. Arithmetic instructions use these sizes and signed types to determine how the numbers are processed.

13.1 Binary Representation

Binary numbers work by using each bit to represent a power of two. For example, the 8-bit value 01010101 adds up to 85 in decimal. The Propeller 2 works entirely in binary internally, even though decimal is used for readability. Decimal values are only human-friendly representations, while actual computation operates on raw binary data.

When displaying numbers (for example over UART or an LCD), values must be converted from binary into ASCII characters so they can be printed correctly.

13.2 Signed Numbers and Two's Complement

For signed values, the Propeller 2 uses two's complement representation. This allows both positive and negative numbers to be stored.

To form a negative number:

- Invert all bits
- Add 1

For example, in 8-bit form, -1 becomes 11111111.

13.3 Arithmetic Instructions

ADD Instruction

The ADD instruction performs addition between two values and stores the result in a destination register. Overflow may occur if the result exceeds the data size.

Syntax

```
add destination, source
```

SUB Instruction

The SUB instruction subtracts one value from another. Unsigned results may wrap around if negative, while signed arithmetic produces correct negative values using two's complement.

Syntax

```
sub destination, source
```

QDIV Instruction

QDIV uses the Propeller 2 CORDIC hardware to perform fast division. The quotient is stored in QX and the remainder in QY.

Syntax

```
qdiv source1, source2  
getqx destination  
getqy destination
```

13.4 Memory Alignment

Different data sizes must be correctly aligned in hub memory:

- Bytes: can be stored anywhere
- Words: must start at even addresses
- Longs: must start at addresses divisible by 4

Incorrect alignment may cause invalid reads or writes.

13.5 Printing Numbers in P2 Assembly

Before printing, numbers are stored in registers, converted into decimal digits using SUB, DIV, or QDIV, and then transmitted as ASCII characters.

Numbers can also be printed using repeated subtraction instead of division. This method counts how many times powers of ten fit into a value and is useful for small numbers or lightweight routines.

Flow

- Load number into register
- Convert to digits using arithmetic
- Add "0" to convert to ASCII
- Send via transmission pin

13.5.1 Program: 0045_print_number_sub.spin2

Listing 13.1: Program: 0045_print_number_sub.spin2

```

'print_number_sub.spin2
CON
    _CLKFREQ          = 180_000_000
    BAUD_RATE         = 2_000_000
    TRANSMISSION_PIN  = 62
    BIT_PERIOD        = (_CLKFREQ / BAUD_RATE)
    BAUD_MODE          = (BIT_PERIOD << 16) | (8 - 1)

DAT
    org 0
    asmclk

    dir1    #TRANSMISSION_PIN
    wrpin   ##(P_ASYNC_TX | P_OE), #TRANSMISSION_PIN
    wxpin   ##BAUD_MODE, #TRANSMISSION_PIN
    dirh    #TRANSMISSION_PIN
    wypin   #1, #TRANSMISSION_PIN

    mov     number_store, #245
    wrlong  number_store, ##@number1
    rdlong  number_store, ##@number1
    mov     value, number_store

    mov     digit, #0
hundreds_loop
    cmp     value, #100    wc
    if_nc   sub     value, #100
    if_nc   add     digit, #1
    if_nc   jmp     #hundreds_loop

    add     digit, #"0"
    wypin   digit, #TRANSMISSION_PIN
    waitx   ##1000

    mov     digit, #0
tens_loop
    cmp     value, #10     wc
    if_nc   sub     value, #10
    if_nc   add     digit, #1
    if_nc   jmp     #tens_loop

    add     digit, #"0"
    wypin   digit, #TRANSMISSION_PIN
    waitx   ##1000

```

```
mov     digit, value
add     digit, #"0"
wypin   digit, #TRANSMISSION_PIN

endprog
  jmp   #$.

number1      long 0
number_store res 1
value        res 1
digit        res 1
```

This code prints the number 245 digit by digit through the `TRANSMISSION_PIN`. First, the number 245 is stored in the COG register `number_store` and written into the hub-memory variable `number1` using `WRLONG`. It is then read back into `number_store` with `RDLONG`, and copied into `value` for digit extraction. The program computes the hundreds digit by repeatedly subtracting 100 from `value` and incrementing `digit` until `value` is less than 100, then converts `digit` to its ASCII equivalent by adding "0" and transmits it using `WYPIN`, followed by a brief `WAITX` delay. The tens digit is extracted similarly using repeated subtraction of 10, converted to ASCII, and transmitted with another short delay. Finally, the remaining ones digit is converted to ASCII and transmitted. All digits are sent as ASCII characters, with each step non-blocking except for the short `WAITX` pauses. As a result, the digits "2", "4", and "5" are transmitted sequentially, accurately representing the original number 245.

13.6 Printing Numbers Using `QDIV.spin2`

Here is a faster method to extract decimal digits using the `QDIV` instruction, which performs integer division in hardware. Unlike repeated subtraction, `QDIV` gives both the quotient and the remainder in a single operation. This makes it much more efficient for breaking a number into digits, especially when printing values in loops or working with large numbers.

13.6.1 Program: 0046_print_number_qdiv.spin2

Listing 13.2: Program: 0046_print_number_qdiv.spin2

```

'print_number_qdiv.spin2
CON
    _CLKFREQ          = 180_000_000
    BAUD_RATE         = 2_000_000
    TRANSMISSION_PIN  = 62
    BIT_PERIOD        = (_CLKFREQ / BAUD_RATE)
    BAUD_MODE          = (BIT_PERIOD << 16) | (8 - 1)

DAT
    org 0
    asmclk

    dir1    #TRANSMISSION_PIN
    wrpin   ##(P_ASYNC_TX | P_OE), #TRANSMISSION_PIN
    wxpin   ##BAUD_MODE, #TRANSMISSION_PIN
    dirh    #TRANSMISSION_PIN
    wypin   #1, #TRANSMISSION_PIN

    mov     number_store, #245
    wrlong  number_store, ##@number1
    rdlong  number_store, ##@number1
    mov     value, number_store

    qdiv    value, #100
    getqx   digit
    add     digit, #"0"
    wypin   digit, #TRANSMISSION_PIN

    getqy   value
    waitx   ##1000

    qdiv    value, #10
    getqx   digit
    add     digit, #"0"
    wypin   digit, #TRANSMISSION_PIN

    getqy   value
    waitx   ##1000

    mov     digit, value
    add     digit, #"0"
    wypin   digit, #TRANSMISSION_PIN

    endprog

```

```
    jmp    $$

number1      long 0
number_store res 1
value        res 1
digit        res 1
```

We will print the number 245 digit by digit through the `TRANSMISSION_PIN` using the `QDIV` instruction. First, the number 245 is stored in the COG register `number_store` and written into the hub-memory variable `number1` using `WRLONG`. It is then read back into `number_store` with `RDLONG` and copied into `value` for digit extraction. The hundreds digit is obtained by dividing `value` by 100 with `QDIV`; the quotient is retrieved from `QX` into `digit`, converted to its ASCII equivalent by adding "0", and transmitted via `WYPIN`. The remainder after the division is retrieved from `QY` into `value` for the next step. The tens digit is similarly extracted by dividing `value` by 10 using `QDIV`, converting the quotient to ASCII, and transmitting it. Finally, the remaining ones digit in `value` is converted to ASCII and transmitted. Short `WAITX` delays are used between digits to ensure sequential output. All hub-to-COG transfers and transmissions occur non-blocking. As a result, the digits "2", "4", and "5" are transmitted sequentially, accurately representing the original number stored in `number_store`.

13.7 Exercises

Practice working with numbers, arithmetic, and memory in Propeller 2 assembly with this exercise.

- Begin by converting given decimal numbers into 8-bit binary to understand their bit patterns.
- Next, use `ADD` and `SUB` on both signed and unsigned numbers to observe sums, differences, and overflow behavior.
- Then, pick a number in a register and use shifts or masks to isolate specific bits, recording their values in binary to see how individual bits can be manipulated.
- Finally, store bytes, words, and longs at different hub memory addresses and read them back to explore what happens when data is misaligned, reinforcing safe and precise memory access techniques.

13.8 Summary

Building upon our understanding of hub memory, we now examine the control mechanisms that govern coordinated access and interaction between processing cores.

Chapter Summary

- Numbers are stored in binary form
- Signed values use two's complement
- Arithmetic depends on bit representation
- Memory alignment affects access
- Converting numbers to readable output requires extra steps

Chapter 14

Cog Control

14.1 What is a cog?

A cog in the Propeller 2 is like a small, separate processor inside the chip that can run its own program at the same time as the others. This allows the Propeller 2 to handle multiple tasks in parallel, such as reading sensors, updating a display, or generating sound, without slowing each other down. Learning how to control cogs is important because it allows you to start and stop tasks, assign responsibilities, and safely share data through hub RAM.

The Propeller 2 contains 8 cogs (numbered 0 through 7). Each cog has its own local workspace, including registers and a stack, while also having access to shared hub RAM. Because each cog executes independently, the system can perform multiple operations simultaneously, such as handling I/O, timing, or communication, without blocking other processes.

Key features of cog control include the ability to start and stop cogs dynamically and assign them routines stored in hub memory. Instructions such as `COGINIT`, `COGSTOP`, and `COGID` are used to manage these operations. Each cog maintains its own program counter and execution state, allowing it to run independently without interfering with other cogs.

14.1.1 Syntax

```
coginit cogID, ##@routine
cogid destination_register
cogstop cogID
```

- `cogID` = The ID of the cog (0–7) or `#newcog` to use the next free cog.
- `routine` = The hub memory address where the cog’s program begins.
- `destination_register` = A cog register used to store the cog’s own ID.

In the Propeller 2, each cog operates as an independent processor core capable of running concurrently with others. The `COGINIT` instruction starts a cog and assigns it a routine stored in hub memory, enabling parallel execution of tasks. The `COGSTOP` instruction halts a cog when it is no longer needed, freeing system resources. The `COGID` instruction allows a cog to determine its own ID, which is useful for coordination between multiple running cogs.

Together, these instructions provide essential control over concurrent execution, allowing programs to efficiently manage parallel tasks while maintaining clear organization and predictable behavior.

14.1.2 Program: 0047_cog_control.spin2

Listing 14.1: Program: 0047_cog_control.spin2

```

'cog_control.spin2
CON
    _CLKFREQ          = 180_000_000
    BAUD_RATE         = 2_000_000
    TRANSMISSION_PIN  = 62
    BIT_PERIOD        = (_CLKFREQ / BAUD_RATE)
    BAUD_MODE          = (BIT_PERIOD << 16) | (8 - 1)
    DELAY              = 180_000_000 / 2

DAT
    org 0
    asmclk

    dirl    #TRANSMISSION_PIN
    wrpin   ##(P_ASYNC_TX | P_OE), #TRANSMISSION_PIN
    wxpin   ##BAUD_MODE, #TRANSMISSION_PIN
    dirh    #TRANSMISSION_PIN
    wypin   #1, #TRANSMISSION_PIN

    start
        call #print_cogid
        coginit #newcog, ##@print_cogid
        waitx ##DELAY
        cogstop #1

    jmp #endprog

print_cogid
    cogid cog_number
    add cog_number, #"0"
    wypin cog_number, #TRANSMISSION_PIN
    waitx ##DELAY
    ret

endprog
    jmp #$$

cog_number res 1

```

The main cog executes the `print_cogid` routine first and outputs its ID (0) to the transmission pin using `WYPIN`. Next, a second cog is started with `COGINIT` to run the same routine independently, which outputs its own ID (1). Both cogs operate concurrently and independently, each using its own cog register space without interfering with the other. Short

`WAITX ##DELAY` pauses ensure the outputs are clearly separated in time. After the delay, the main cog stops the second cog using `COGSTOP #1`. The serial output displays the characters “0” followed by “1”, demonstrating correct cog identification as well as parallel execution, cog initialization, and basic cog control. The variable `cog_number` is a cog register used for temporary storage, holding each cog’s ID for ASCII conversion and transmission.

The Propeller 2 allows multiple cogs to run programs simultaneously, enabling true parallel processing. By starting multiple cogs using `COGINIT`, each cog can execute its own routine independently—for example, one cog handling input while another generates output. Using `COGID`, each cog can identify itself and adapt its behavior accordingly, which is useful for coordinating shared resources or preventing conflicts. When a cog completes its task, `COGSTOP` can be used to halt it and free the core for other operations. Running multiple cogs efficiently allows programs to perform concurrent tasks, improve responsiveness, and fully utilize the processing power of the Propeller 2.

14.1.3 Program: 0048_cog_addition.spin2

Listing 14.2: Program: 0048_cog_addition.spin2

```
'cog_addition.spin2
CON
    _CLKFREQ          = 180_000_000
    BAUD_RATE         = 2_000_000
    TRANSMISSION_PIN  = 62
    BIT_PERIOD         = (_CLKFREQ / BAUD_RATE)
    BAUD_MODE          = (BIT_PERIOD << 16) | (8 - 1)
    DELAY              = 180_000_000 / 2

DAT
    org 0
    asmclk

    dirl    #TRANSMISSION_PIN
    wrpin   ##(P_ASYNC_TX | P_OE), #TRANSMISSION_PIN
    wxpin   ##BAUD_MODE, #TRANSMISSION_PIN
    dirh    #TRANSMISSION_PIN
    wypin   #1, #TRANSMISSION_PIN

    main
        coginit #1, ##@addition

        waitx   ##DELAY

        rdword  calculation_result, ##@calculation_result_hub
        add     calculation_result, #"0"
        wypin   calculation_result, #TRANSMISSION_PIN

        jmp #endprog

    addition
        mov     calculation_result, #4
        add     calculation_result, #2
        wrword  calculation_result, ##@calculation_result_hub
        cogstop #1
        ret

    endprog
        jmp    #$

    calculation_result_hub    word 0
    calculation_result        res 1
```

The main cog starts a second cog using `COGINIT #1` to execute the addition routine inde-

pendently. The second cog calculates the sum of $4 + 2$, stores the 16-bit result in hub memory (`calculation_result_hub`) using `WRWORD`, and then stops itself with `COGSTOP #1`. After waiting long enough with `WAITX ##DELAY` for the second cog to finish, the main cog reads the value from hub memory into the cog register `calculation_result` using `RDWORD`, converts it to its ASCII equivalent by adding "0", and immediately transmits it via the transmission pin using `WYPIN`. The variables `calculation_result_hub` (hub RAM) and `calculation_result` (cog register) are used to store the calculation result and facilitate hub-to-cog transfer. As a result, the serial output displays the character "6", demonstrating that the addition was performed in a separate cog and the main cog successfully retrieved and transmitted the result.

Using multiple cogs in the Propeller 2 allows a program to divide work across independent cores, improving efficiency and responsiveness. For example, one cog can handle real-time input, such as reading a sensor or keyboard, while another cog simultaneously updates a display or controls LEDs. By starting each cog with `COGINIT` and using `COGID` to identify which cog is executing, routines can be tailored to specific tasks without interfering with each other. Once a cog finishes its assigned work, `COGSTOP` can safely halt it, freeing resources and preventing unnecessary CPU usage. This approach demonstrates how multi-cog programming can simplify complex tasks and enable parallel, synchronized operations within a single Propeller 2 system.

14.1.4 Program: 0049_cog_multicog.spin2

Listing 14.3: Program: 0049_cog_multicog.spin2

```
'cog_multicog.spin2
CON
    _CLKFREQ          = 180_000_000
    BAUD_RATE         = 2_000_000
    TRANSMISSION_PIN  = 62
    BIT_PERIOD        = (_CLKFREQ / BAUD_RATE)
    BAUD_MODE          = (BIT_PERIOD << 16) | (8 - 1)
    DELAY              = 180_000_000 / 2

DAT
    org 0
    asmclk

    dirl    #TRANSMISSION_PIN
    wrpin   ##(P_ASYNC_TX | P_OE), #TRANSMISSION_PIN
    wxpin   ##BAUD_MODE, #TRANSMISSION_PIN
    dirh    #TRANSMISSION_PIN
    wypin   #1, #TRANSMISSION_PIN

    mov     working_value, #0
    wrlong  working_value, ##@shared_counter

    start
        coginit #1, ##@incrementer
        coginit #2, ##@reporter

    jmp     #endprog

incrementer
    .loop
        rdlong  working_value, ##@shared_counter
        cmp     working_value, #10 wz
        if_z    jmp #stop_cog

        add     working_value, #1
        wrlong  working_value, ##@shared_counter
        waitx   ##DELAY
        jmp     #.loop

reporter
    .loop
        rdlong  working_value, ##@shared_counter
```

```

    cmp      working_value, #10 wz
    if_z     jmp #stop_cog

    add      working_value, #"0"
    wypin    working_value, #TRANSMISSION_PIN
    waitx    ##DELAY
    jmp      #.loop

stop_cog
    cogid     working_value
    cogstop   working_value

endprog
    jmp      #$

shared_counter    long 0
working_value     res 1

```

Cog 1 (incrementer) repeatedly reads the 32-bit hub-memory variable `shared_counter` into the cog register `working_value`, increments it by 1, and writes it back to hub memory using `RDLONG` and `WRLONG`. At the same time, Cog 2 (reporter) reads the same counter from hub memory, converts it to its ASCII equivalent by adding "0", and immediately transmits it via the transmission pin using `WYPIN`. Both cogs monitor the counter and automatically stop themselves with `COGSTOP` once the value reaches 10. Short `WAITX ##DELAY` pauses ensure that increments and transmissions occur at visible intervals. The `shared_counter` variable in hub RAM allows inter-cog communication, while `working_value` is a one-byte cog register used for temporary storage. As a result, the serial output displays the numbers “0” through “9” sequentially, demonstrating parallel execution, safe sharing of hub memory between cogs, and proper self-stopping to prevent unnecessary execution.

A cog in the Propeller 2 can be dedicated to flipping or toggling memory values in hub RAM, allowing a program to perform continuous or repetitive tasks independently. By starting a cog with `COGINIT`, you can run a routine that repeatedly reads a value from hub memory, modifies it (for example, inverting bits or toggling a flag), and writes it back. Using `COGID`, the cog can identify itself to coordinate memory access if multiple cogs are involved, preventing conflicts. Once the flipping task is no longer needed, `COGSTOP` can safely halt the cog. This approach uses a separate core to handle ongoing memory operations without blocking the main program, demonstrating efficient use of parallel processing for dynamic data manipulation.

14.1.5 Program: 0050_cog_memoryflip.spin2

Listing 14.4: Program: 0050_cog_memoryflip.spin2

```
'cog_memoryflip.spin2
CON
    _CLKFREQ          = 180_000_000
    BAUD_RATE         = 2_000_000
    TRANSMISSION_PIN  = 62
    BIT_PERIOD        = (_CLKFREQ / BAUD_RATE)
    BAUD_MODE          = (BIT_PERIOD << 16) | (8 - 1)
    DELAY              = 180_000_000 / 2

DAT
    org 0
    asmclk

    dirl    #TRANSMISSION_PIN
    wrpin   ##(P_ASYNC_TX | P_OE), #TRANSMISSION_PIN
    wxpin   ##BAUD_MODE, #TRANSMISSION_PIN
    dirh    #TRANSMISSION_PIN
    wypin   #1, #TRANSMISSION_PIN

    main
        wrbyte #"+", #shared_char

        coginit #1, ##@swap_char
        waitx ##DELAY

        rdbyte working_value, ##shared_char
        wypin working_value, ##TRANSMISSION_PIN

        jmp #endprog

    swap_char
        rdbyte working_value, ##shared_char
        cmp working_value, #"-" wz
        if_z mov working_value, #"+
        if_nz mov working_value, #"-"
        wrbyte working_value, ##shared_char
        cogstop #1

    endprog
        jmp #

    shared_char    byte 0
    working_value  res 1
```

The main cog first writes the character “+” into the shared hub-memory variable `shared_char` using `WRBYTE`. A second cog is then started with `COGINIT` to execute the `swap_char` routine independently. This routine reads the value from `shared_char` into the cog register `working_value` using `RDBYTE`, checks whether it is “-”: if so, it changes it to “+”, otherwise it changes it to “-”, and writes the updated character back to hub memory using `WRBYTE`. After a short delay (`WAITX ##DELAY`), the main cog reads the modified value from `shared_char` into `working_value` and immediately transmits it via the transmission pin using `WYPIN`. Both hub-to-cog transfers and transmissions occur non-blocking. The variable `shared_char` resides in hub RAM to allow inter-cog communication, while `working_value` is a one-byte cog register used for temporary storage. As a result, the serial output displays the character “-“, demonstrating that a separate cog can independently modify a shared hub-memory variable and that the main cog can successfully retrieve and use the updated value.

14.2 Cog Process Control

Process control in the Propeller 2 is about managing what each cog does and when it does it. You can start a cog running a specific task, let it run independently, and then stop it when it is finished. This allows the main program or other cogs to focus on different work without interference. By controlling cogs carefully, you can divide a program into smaller tasks, assign them to different cogs, and coordinate their actions through shared memory safely.

You can safely share hub memory between cogs using locks. `LOCKNEW` reserves a lock for use before any cog accesses shared data. Each cog attempts to acquire the lock using `LOCKTRY`, which ensures that only one cog can modify shared memory at a time. After finishing, the cog releases the lock with `LOCKREL`. This allows multiple cogs to run independently while preventing conflicts, keeping the program predictable and correct.

14.2.1 Syntax

```
locknew destination_register
locktry destination_register wc
lockrel destination_register
```

- `destination_register` = A cog register holding the lock ID (0–15)
- Use `LOCKNEW` once to reserve a lock before any cog accesses shared memory
- Use `LOCKTRY` in each cog to safely acquire the lock before modifying shared data
- Use `LOCKREL` after finishing access to shared memory so other cogs can proceed

14.2.2 Program: 0051_cog_lock.spin2

Listing 14.5: Program: 0051_cog_lock.spin2

```

'cog_lock.spin2
CON
    _CLKFREQ          = 180_000_000
    BAUD_RATE         = 2_000_000
    TRANSMISSION_PIN  = 62
    BIT_PERIOD        = (_CLKFREQ / BAUD_RATE)
    BAUD_MODE         = (BIT_PERIOD << 16) | (8 - 1)

DAT
    org 0
    asmclk

    dir1    #TRANSMISSION_PIN
    wrpin   ##(P_ASYNC_TX | P_OE), #TRANSMISSION_PIN
    wxpin   ##BAUD_MODE, #TRANSMISSION_PIN
    dirh    #TRANSMISSION_PIN
    wypin   #1, #TRANSMISSION_PIN

    mov     working_value, #0
    wrword  working_value, ##@shared_calculation

    mov     working_value, #0
    wrbyte  working_value, ##@shared_done_counter

    locknew working_value
    wrbyte  working_value, ##@shared_lock

start
    coginit #1, ##@addition
    coginit #2, ##@addition

    wait_for_cogs
        rdbyte working_value, ##@shared_done_counter
        cmp working_value, #2 wz
        if_nz jmp #wait_for_cogs

    rdword  working_value, ##@shared_calculation
    add working_value, #"0"
    wypin  working_value, #TRANSMISSION_PIN

    jmp #endprog

addition
    .try_lock

```



```

    rdbyte working_value, ##@shared_lock
    locktry working_value wc
    if_nc jmp #.try_lock

    rdword working_value, ##@shared_calculation
    add working_value, #2
    add working_value, #2
    wrword working_value, ##@shared_calculation

    rdbyte working_value, ##@shared_done_counter
    add working_value, #1
    wrbyte working_value, ##@shared_done_counter

    rdbyte working_value, ##@shared_lock
    lockrel working_value

    cogid working_value
    cogstop working_value
    ret

endprog
    jmp #$.

shared_calculation    word 0, 0
shared_done_counter    byte 0
shared_lock            res 1
working_value          res 1

```

The main cog first creates a shared lock in hub memory using **LOCKNEW**, which ensures that only one cog can access the shared data at a time. Then two cogs are launched to perform an addition operation in parallel. Each cog repeatedly tries to acquire the lock using **LOCKTRY** before reading and updating the 16-bit variable **shared_calculation**. Once a cog successfully acquires the lock, it adds $2 + 2$ to the shared value, writes the result back to hub memory, increments the **shared_done_counter** to indicate completion, releases the lock with **LOCKREL**, and stops itself using **COGSTOP**. Meanwhile, the main cog waits until both cogs have finished by monitoring the done counter, then reads the final result from **shared_calculation** and transmits it as an ASCII character through the transmission pin using **WYPIN**. The **working_value** register serves as temporary storage for reading, writing, and lock operations, while the hub variables coordinate safe parallel execution. This demonstrates how **LOCKNEW**, **LOCKTRY**, and **LOCKREL** work together to prevent conflicts when multiple cogs share memory on the Propeller 2.

14.3 Exercise

Write a short program where the main cog starts a second cog using **COGINIT**. The second cog should write the value 5 into a hub memory variable and then stop itself using **COGSTOP**.

The main cog should wait briefly, read the value from hub memory, convert it to ASCII by adding "0", and transmit it through the serial pin. When the program runs, the serial output should display "5", showing that one cog performed a task independently and shared the result through hub memory.

14.4 Summary

With memory coordination established, we now turn to string manipulation, where sequences of characters are processed, transformed, and interpreted as structured data within the system.

Chapter Summary

- Cogs can be started and controlled
- Multiple cogs can run simultaneously
- Shared resources require coordination
- Locks prevent conflicts between cogs
- Enables parallel processing

Chapter 15

String Manipulation

A string is simply a sequence of characters stored in memory, typically ending with a null byte (0). The Propeller 2 Assembly language provides no built-in string functions, so all operations must be performed manually using byte-level instructions such as `RDBYTE` and `WRBYTE`. Strings are commonly used for serial communication (UART text output), constructing messages, and comparing or modifying character sequences.

15.1 Print a String

Printing a string in P2 assembly involves reading characters sequentially from hub RAM until a null terminator (0) is found. Because the Propeller 2 has no built-in string functions, each character must be fetched manually using `RDBYTE` and then sent to an output device such as a UART SmartPin. The process continues one byte at a time until the null terminator is reached, signaling the end of the string. This method is fundamental for displaying messages or sending text to serial consoles.

15.2 String Length

Finding the length of a string requires iterating through the bytes stored in memory until the null terminator is encountered. A counter is incremented for each character processed, resulting in the total number of visible characters in the string. Since the P2 does not track string sizes automatically, this manual traversal ensures precision and allows later routines—such as formatting, validation, or buffer management—to rely on the correct character count.

15.2.1 Program: 0052_string_length.spin2

Listing 15.1: Program: 0052_string_length..spin2

```
'string_length.spin2
CON
    _CLKFREQ          = 180_000_000
    BAUD_RATE         = 2_000_000
    TRANSMISSION_PIN  = 62
    BIT_PERIOD        = (_CLKFREQ / BAUD_RATE)
    BAUD_MODE          = (BIT_PERIOD << 16) | (8 - 1)

DAT
    org 0
    asmclk

    dir1    #TRANSMISSION_PIN
    wrpin   ##(P_ASYNC_TX | P_OE), #TRANSMISSION_PIN
    wxpin   ##BAUD_MODE, #TRANSMISSION_PIN
    dirh    #TRANSMISSION_PIN
    wypin   #1, #TRANSMISSION_PIN

    mov stringPointer, ##@mystring
    call #get_string_length

    add string_length, #"0"
    wypin string_length, #TRANSMISSION_PIN

    jmp      #endprog

get_string_length
    mov      string_length, #0
    .loop
        rdbyte currentChar, stringPointer
        cmp    currentChar, #0 wz
        if_z   ret
        add    string_length, #1
        add    stringPointer, #1
    jmp #.loop

endprog
    jmp  #$.

mystring          byte "Hello!",0
string_length     res 1
currentChar       res 1
stringPointer     res 1
```

We demonstrate how to calculate the length of a null-terminated string in Propeller 2 assembly. The COG register `stringPointer` is initialized to the hub-memory address of the string `mystring`. The subroutine `get_string_length` repeatedly reads each byte from hub RAM into the COG register `currentChar` using `RDBYTE`, checks for the null terminator (0), and increments the counter `string_length` for each character processed. The loop continues until the null terminator is encountered. After the subroutine returns, `string_length` contains the number of characters in the string (excluding the null terminator). The program then converts `string_length` to its ASCII equivalent by adding "0" and transmits it via the transmission pin using `WYPIN`. The variables `stringPointer` and `currentChar` are one-byte COG registers used for pointer tracking and temporary storage, while `string_length` is a COG register counter. The string `mystring` resides in hub RAM. This method is fundamental in P2 programming, as strings do not carry inherent length information and must be manually processed.

15.3 String Number Length

Sometimes you may want to limit the maximum length of a string when counting characters. This prevents errors if the string is longer than expected. In this example, we count up to a specific number of characters, stopping either at the null terminator or the limit.

15.3.1 Program: 0053_string_number_length.spin2

Listing 15.2: Program: 0053_string_number_length.spin2

```
'string_number_length.spin2
CON
    _CLKFREQ          = 180_000_000
    BAUD_RATE         = 2_000_000
    TRANSMISSION_PIN  = 62
    BIT_PERIOD         = (_CLKFREQ / BAUD_RATE)
    BAUD_MODE          = (BIT_PERIOD << 16) | (8 - 1)

DAT
    org 0
    asmclk

    dirl    #TRANSMISSION_PIN
    wrpin   ##(P_ASYNC_TX | P_OE), #TRANSMISSION_PIN
    wxpin   ##BAUD_MODE, #TRANSMISSION_PIN
    dirh    #TRANSMISSION_PIN
    wypin   #1, #TRANSMISSION_PIN

main
    mov stringPointer, ##@mystring
    call #get_string_length

    add string_length, #"0"
    wypin string_length, #TRANSMISSION_PIN

    jmp     #endprog

get_string_length
    mov string_length, #0
    mov string_limit, #9
    .loop
        rdbyte    currentChar, stringPointer
        cmp       currentChar, #0 wz
        if_z      ret
        cmp       string_limit, string_length wz
        if_z      ret
        add       string_length, #1
        add       stringPointer, #1
    jmp          #.loop

endprog
    jmp  #$.

mystring          byte "Hello!",0
```

<code>string_length</code>	<code>res 1</code>
<code>string_limit</code>	<code>res 1</code>
<code>currentChar</code>	<code>res 1</code>
<code>stringPointer</code>	<code>res 1</code>

We calculate the length of a null-terminated string in Propeller 2 assembly. Our program calculates the length of a null-terminated string in Propeller 2 assembly with an enforced maximum limit. The COG register `stringPointer` is initialized to the hub-memory address of the string `mystring`. The subroutine `get_string_length` repeatedly reads each byte from hub RAM into the COG register `currentChar` using `RDBYTE`, checks for either the null terminator (0) or that the counter `string_length` has reached the maximum `string_limit`, and increments `string_length` for each character processed. The loop exits when either condition is met. After returning from the subroutine, `string_length` contains the number of characters in the string (up to the limit). The program then converts `string_length` to its ASCII equivalent by adding "0" and transmits it via the transmission pin using `WYPIN`. The variables `stringPointer` and `currentChar` are one-byte COG registers used for pointer tracking and temporary storage, while `string_length` and `string_limit` are COG registers for counting and limiting. The string `mystring` resides in hub RAM. In this example, the string "Hello!" produces a length of 6, transmitted as the ASCII character "6". This routine demonstrates safe and fundamental string handling in P2 assembly, where strings do not carry inherent length information and must be manually processed.

15.4 String Compare

String comparison is a byte-by-byte process where each character from the first string is matched with the corresponding character from the second string. The comparison continues until a mismatch is found or both strings reach the null terminator at the same position. If all characters match up to the terminator, the strings are considered equal. This method is the basis for implementing searching, sorting, or command-parsing routines in low-level programs.

15.4.1 Program: 0054_string_compare.spin2

Listing 15.3: Program: 0054_string_compare.spin2

```
'string_compare.spin2
CON
    _CLKFREQ          = 180_000_000
    BAUD_RATE         = 2_000_000
    TRANSMISSION_PIN  = 62
    BIT_PERIOD        = (_CLKFREQ / BAUD_RATE)
    BAUD_MODE          = (BIT_PERIOD << 16) | (8 - 1)

DAT
    org 0
    asmclk

    dir1    #TRANSMISSION_PIN
    wrpin   ##(P_ASYNC_TX | P_OE), #TRANSMISSION_PIN
    wxpin   ##BAUD_MODE, #TRANSMISSION_PIN
    dirh    #TRANSMISSION_PIN
    wypin   #1, #TRANSMISSION_PIN

main
    mov     stringPointer1, ##@mystring1
    mov     stringPointer2, ##@mystring2
    call    #string_compare

    add     string_result, #"0"
    wypin   string_result, #TRANSMISSION_PIN

    jmp     #endprog

string_compare
    .loop
        rdbyte currentChar1, stringPointer1
        rdbyte currentChar2, stringPointer2
        cmp   currentChar1, currentChar2 wz
        if_nz jmp     #.different
        cmp   currentChar1, #0 wz
        if_z  jmp     #.same
        add   stringPointer1, #1
        add   stringPointer2, #1
        jmp   #.loop

    .same
        mov   string_result, #0
ret
```



```

        .different
        mov     string_result, #1
ret

endprog
    jmp     #$

mystring1    byte "Hello!", 0, 0
mystring2    byte "Hello!", 0
currentChar1    res 1
currentChar2    res 1
stringPointer1    res 1
stringPointer2    res 1
string_result    res 1

```

This compares two null-terminated strings in Propeller 2 assembly. The COG registers `stringPointer1` and `stringPointer2` are initialized to the hub-memory addresses of `mystring1` and `mystring2`. The subroutine `string_compare` reads each byte from the strings into the one-byte COG registers `currentChar1` and `currentChar2` using `RDBYTE`, and compares them. If a mismatch is found at any position, the subroutine immediately sets `string_result` to 1, indicating the strings are different, and returns. If both strings reach their null terminators simultaneously without a mismatch, `string_result` is set to 0, indicating the strings are identical. After returning from the subroutine, `string_result` is converted to its ASCII equivalent by adding "0" and transmitted via the transmission pin using `WYPIN`. The variables `currentChar1` and `currentChar2` serve as temporary COG registers for byte comparison, while `string_result` holds the comparison outcome. The strings `mystring1` and `mystring2` reside in hub RAM. This routine provides a reliable, byte-by-byte comparison of any pair of null-terminated strings and outputs "0" for identical strings or "1" for different strings.

15.5 String Number Compare

Sometimes you only want to compare the first N characters of two strings. This is useful when you want a prefix match, when the strings may have different lengths, or when you want to limit processing to a specific number of characters. In such a comparison, you check each character up to the specified length or stop sooner if a null terminator is reached. If all compared characters match, the strings are considered equal within that range; if a mismatch is found, they are considered different. This approach allows efficient checking of string prefixes without needing to process the entire string.

15.5.1 Program: 0055_string_number_compare.spin2

Listing 15.4: Program: 0055_string_number_compare.spin2

```
'string_number_compare.spin2
CON
    _CLKFREQ          = 180_000_000
    BAUD_RATE         = 2_000_000
    TRANSMISSION_PIN  = 62
    BIT_PERIOD        = (_CLKFREQ / BAUD_RATE)
    BAUD_MODE          = (BIT_PERIOD << 16) | (8 - 1)

DAT
    org 0
    asmclk

    dir1    #TRANSMISSION_PIN
    wrpin   ##(P_ASYNC_TX | P_OE), #TRANSMISSION_PIN
    wxpin   ##BAUD_MODE, #TRANSMISSION_PIN
    dirh    #TRANSMISSION_PIN
    wypin   #1, #TRANSMISSION_PIN

main
    mov stringPointer1, ##@mystring1
    mov stringPointer2, ##@mystring2
    mov compareLength, #6
    call #string_n_compare

    add stringResult, #"0"
    wypin stringResult, #TRANSMISSION_PIN

    jmp      #endprog

string_n_compare
    .loop
        cmp      compareLength, #0 wz
        if_z     jmp      #.same

        rdbyte   currentChar1, stringPointer1
        rdbyte   currentChar2, stringPointer2
        cmp      currentChar1, currentChar2 wz
        if_nz    jmp      #.different
        cmp      currentChar1, #0 wz
        if_z     jmp      #.same

        add      stringPointer1, #1
        add      stringPointer2, #1
        sub      compareLength, #1
```

```

        jmp      #.loop

    .same
        mov      stringResult, #0
ret

    .different
        mov      stringResult, #1
ret

endprog
    jmp    #$

mystring1      byte "Hello!",0, 0
mystring2      byte "Hello?",0

currentChar1    res 1
currentChar2    res 1
stringPointer1  res 1
stringPointer2  res 1
stringResult    res 1
compareLength   res 1

```

We are going to compare up to a specified number of characters from two null-terminated strings in Propeller 2 assembly. The COG registers `stringPointer1` and `stringPointer2` are initialized to the hub-memory addresses of `mystring1` and `mystring2`, and `compareLength` is set to the maximum number of characters to compare. The subroutine `string_n_compare` reads one byte at a time from hub RAM into the COG registers `currentChar1` and `currentChar2` using `RDBYTE`. Each pair of bytes is compared: if a mismatch is found, `stringResult` is immediately set to 1 and the subroutine returns. The loop also exits if a null terminator (0) is encountered in either string or if `compareLength` reaches zero, in which case `stringResult` is set to 0, indicating the strings match up to the specified length. After returning, `stringResult` is converted to its ASCII equivalent by adding "0" and transmitted via the transmission pin using `WYPIN`. The temporary COG registers `currentChar1` and `currentChar2` hold each byte for comparison, while `stringResult` stores the final outcome. The strings reside in hub RAM. In this example, `mystring1` is "Hello!" and `mystring2` is "Hello?"; the program compares the first six characters and outputs "0" if they match within that limit, or "1" if a difference is found. This routine is useful for prefix matching, safely limiting string comparisons, and handling strings of differing lengths.

15.6 String Copy

Copying a string is done by reading each byte from a source address and writing it to a destination address, repeating the process until the null terminator is copied as well. The terminator ensures the destination string is properly closed and safe for later operations.

This operation is essential when building dynamic messages, managing multiple buffers, or preparing data for output, because the P2 provides no direct memory-copy instructions for strings.

15.6.1 Program: 0056_string_copy.spin2

Listing 15.5: Program: 0056_string_copy.spin2

```
'string_copy.spin2
CON
    _CLKFREQ          = 180_000_000
    BAUD_RATE         = 2_000_000
    TRANSMISSION_PIN  = 62
    BIT_PERIOD        = (_CLKFREQ / BAUD_RATE)
    BAUD_MODE          = (BIT_PERIOD << 16) | (8 - 1)
    DELAY              = _CLKFREQ

DAT
    org 0
    asmclk

    dirl    #TRANSMISSION_PIN
    wrpin   ##(P_ASYNC_TX | P_OE), #TRANSMISSION_PIN
    wxpin   ##BAUD_MODE, #TRANSMISSION_PIN
    dirh    #TRANSMISSION_PIN
    wypin   #1, #TRANSMISSION_PIN

main
    mov     sourcePointer, #@mystring1
    mov     destinationPointer, #@mystring2
    call    #string_copy

    mov     destinationPointer, #@mystring2
    .loop_transmit
        rdbyte currentChar, destinationPointer
        cmp    currentChar, #0 wz
        if_z   jmp #endprog
        wypin  currentChar, #TRANSMISSION_PIN
        waitx  ##DELAY
        add    destinationPointer, #1
        jmp    #.loop_transmit

    jmp    #endprog

string_copy
    .loop
        rdbyte currentChar, sourcePointer
        wrbyte currentChar, destinationPointer
        add    sourcePointer, #1
        add    destinationPointer, #1
        cmp    currentChar, #0 wz
        if_z   ret
```

```

        jmp #.loop

endprog
        jmp  #$.

mystring1      byte "Hello!",0, 0
mystring2      byte 0,0,0,0,0,0      ' enough space for copy

currentChar    res 1
sourcePointer  res 1
destinationPointer res 1

```

Our program demonstrates how to copy a null-terminated string from one hub memory location to another in Propeller 2 assembly and then transmit it via the transmission pin. The COG registers `sourcePointer` and `destinationPointer` are initialized to the addresses of `mystring1` and `mystring2`, respectively. The subroutine `string_copy` reads one byte at a time from the source using `RDBYTE`, writes it to the destination using `WRBYTE`, and increments both pointers. The loop terminates when a null terminator (0) is encountered, ensuring the full string is copied. After returning, the main program resets `destinationPointer` to the start of `mystring2` and iterates through the copied string: each byte is read into the temporary COG register `currentChar`, transmitted via `WYPIN`, delayed briefly with `WAITX`, and the pointer incremented. This guarantees the copied string is output character by character in order. The variables `sourcePointer`, `destinationPointer`, and `currentChar` clearly indicate their roles, and the space allocated for `mystring2` ensures it can fully accommodate the copied string. The original string remains unchanged, while the copy is safely stored in hub memory and transmitted sequentially.

15.7 String Number Copy

Copying a string of characters from one location to another is common. The process involves taking each character from a source string and placing it into a destination, one by one, until the end of the string is reached. If the string is shorter than the space available, the remaining locations can be filled with zeros. This method is useful for creating exact copies of messages, preparing data for output, or organizing strings in memory, giving a beginner a clear understanding of how string data can be moved and managed in a program.

15.7.1 Program: 0057_string_number_copy.spin2

Listing 15.6: Program: 0057_string_number_copy.spin2

```
'string_number_copy.spin2
CON
    _CLKFREQ          = 180_000_000
    BAUD_RATE         = 2_000_000
    TRANSMISSION_PIN  = 62
    BIT_PERIOD        = (_CLKFREQ / BAUD_RATE)
    BAUD_MODE          = (BIT_PERIOD << 16) | (8 - 1)
    DELAY              = 180_000_000

DAT
    org 0
    asmclk

    dirl    #TRANSMISSION_PIN
    wrpin   ##(P_ASYNC_TX | P_OE), #TRANSMISSION_PIN
    wxpin   ##BAUD_MODE, #TRANSMISSION_PIN
    dirh    #TRANSMISSION_PIN
    wypin   #1, #TRANSMISSION_PIN

main
    mov sourcePointer, #@mystring1
    mov destinationPointer, #@mystring2
    mov maxCopyBytes, #2
    call #string_number_copy

    mov     destinationPointer, #@mystring2
.loop_transmit
    rdbyte  currentChar, destinationPointer
    cmp     currentChar, #0 wz
    if_z    jmp #endprog
    wypin   currentChar, #TRANSMISSION_PIN
    waitx   ##DELAY
    add     destinationPointer, #1
    jmp     #.loop_transmit

    jmp     #endprog

string_number_copy
.loop
    cmp     maxCopyBytes, #0 wz
    if_z    ret

    rdbyte  currentChar, sourcePointer
    cmp     currentChar, #0 wz
```

```

        if_z      jmp #.fill_zero

        wrbyte    currentChar, destinationPointer
        add       sourcePointer, #1
        add       destinationPointer, #1
        sub       maxCopyBytes, #1
        jmp       #.loop

.fill_zero
        mov       currentChar, #0
.fill_loop
        wrbyte    currentChar, destinationPointer
        add       destinationPointer, #1
        sub       maxCopyBytes, #1
        cmp       maxCopyBytes, #0 wz
        if_z      ret
        jmp       #.fill_loop

endprog
        jmp      #$

currentChar      res 1
sourcePointer    res 1
destinationPointer res 1
maxCopyBytes     res 1

mystring1        byte "Hello!",0, 0
mystring2        byte 0,0,0,0,0,0,0

```

When this program runs, it copies a limited number of characters from one null-terminated string to another in Propeller 2 assembly. The main program initializes `sourcePointer` and `destinationPointer` to the addresses of `mystring1` and `mystring2`, respectively, and sets `maxCopyBytes` to 2, limiting the copy to two characters. The subroutine `string_number_copy` reads one byte at a time from the source using `RDBYTE`, writes it to the destination using `WRBYTE`, and increments both pointers while decrementing `maxCopyBytes`. If the source string ends before the limit is reached, the routine fills the remaining space in the destination with zeros to ensure proper null-termination. After returning, the main program resets `destinationPointer` to the start of `mystring2` and transmits each byte sequentially through `WYPIN`, with a full-second delay (`WAITX ##DELAY`) between transmissions. The loop stops when a null terminator (0) is encountered. This ensures only the intended number of characters is copied and transmitted—in this example, the characters “H” and “e”—demonstrating safe, length-limited string copying in hub memory. The variables `currentChar`, `sourcePointer`, `destinationPointer`, and `maxCopyBytes` clearly indicate their roles in controlling the copy process and temporary storage.

15.8 String Concatenate

String concatenation is performed by manually copying two null-terminated strings into a single buffer. The process begins by copying the first string byte-by-byte into the destination until its null terminator is reached. Then, instead of stopping, the routine continues by reading each character of the second string and writing it immediately after the first string's final byte. After the second string's null terminator is copied, the combined buffer contains a seamless concatenation of both strings. This method is essential for building messages, merging input fragments, constructing command lines, or creating dynamic text output in environments where no built-in string library exists.

15.8.1 Program: 0058_string_concatenate.spin2

Listing 15.7: Program: 0058_string_concatenate.spin2

```
'string_concatenate.spin2
CON
    _CLKFREQ          = 180_000_000
    BAUD_RATE         = 2_000_000
    TRANSMISSION_PIN  = 62
    BIT_PERIOD        = (_CLKFREQ / BAUD_RATE)
    BAUD_MODE          = (BIT_PERIOD << 16) | (8 - 1)
    DELAY              = 180_000_000

DAT
    org 0
    asmclk

    dirl    #TRANSMISSION_PIN
    wrpin   ##(P_ASYNC_TX | P_OE), #TRANSMISSION_PIN
    wxpin   ##BAUD_MODE, #TRANSMISSION_PIN
    dirh    #TRANSMISSION_PIN
    wypin   #1, #TRANSMISSION_PIN

main
    mov sourcePointer1, #@mystring2
    mov destinationPointer, #@mystring3
    call #string_copy

    mov sourcePointer2, #@mystring1
    mov destinationPointer, #@mystring3
    call #string_concat

    mov transmitPointer, #@mystring3
.loop_transmit
    rdbyte currentChar, transmitPointer
    cmp    currentChar, #0 wz
    if_z   jmp #endprog
    wypin  currentChar, #TRANSMISSION_PIN
    waitx  ##DELAY
    add    transmitPointer, #1
    jmp    #.loop_transmit

    jmp    #endprog

string_copy
.loop_copy
    rdbyte currentChar, sourcePointer1
    wrbyte currentChar, destinationPointer
```

```

        add     sourcePointer1, #1
        add     destinationPointer, #1
        cmp     currentChar, #0 wz
        if_z    ret
        jmp     #.loop_copy

string_concat
.loop_find_end
        rdbyte   currentChar, destinationPointer
        cmp     currentChar, #0 wz
        if_z    jmp     #.append_start
        add     destinationPointer, #1
        jmp     #.loop_find_end

.append_start
.loop_append
        rdbyte   currentChar, sourcePointer2
        wrbyte   currentChar, destinationPointer
        add     sourcePointer2, #1
        add     destinationPointer, #1
        cmp     currentChar, #0 wz
        if_z    ret
        jmp     #.loop_append

endprog
        jmp     #$.

currentChar      res 1
sourcePointer1    res 1
sourcePointer2    res 1
destinationPointer res 1
transmitPointer   res 1

mystring1         byte "World!",0, 0
mystring2         byte "Hello,",0,0,0,0,0,0
mystring3         byte 256

```

Let's demonstrate how to combine two strings into a single buffer and transmit the result character by character. First, the string "Hello," stored in `mystring2` is copied into the buffer `mystring3` using the `string_copy` routine, which reads each byte from the source with `RDBYTE` and writes it to the destination with `WRBYTE`, stopping at the null terminator. Next, the string "World!" from `mystring1` is appended to the end of `mystring3` using the `string_concat` routine, which first locates the null terminator in the destination buffer and then sequentially writes each byte of the second string, including its null terminator, to the buffer. After concatenation, the main program sets `transmitPointer` to the start of `mystring3` and transmits each character one by one through the `TRANSMISSION_PIN` using `WYPIN`, with a full-second delay (`WAITX ##DELAY`) between characters, stopping at the null

terminator. The final result, “Hello,World!”, appears sequentially on the serial output. Temporary registers `currentChar`, `sourcePointer1`, `sourcePointer2`, `destinationPointer`, and `transmitPointer` clearly indicate the roles of each value in reading, writing, appending, and transmitting the string. This routine illustrates safe string copy, concatenation, and sequential transmission in hub memory while preserving proper null-termination.

15.9 String Number Concatenate

String concatenation can be done by first copying a source string into a destination buffer and then appending a specific number of characters from another string. A null terminator is added at the end to maintain a valid string. This approach is useful for working with fixed-size buffers, ensuring that memory is not overwritten while combining strings.

15.9.1 Program: 0059_string_number_concatenate.spin2

Listing 15.8: Program: 0059_string_number_concatenate.spin2

```

'string_number_concatenate.spin2
CON
    _CLKFREQ          = 180_000_000
    BAUD_RATE         = 2_000_000
    TRANSMISSION_PIN  = 62
    BIT_PERIOD        = (_CLKFREQ / BAUD_RATE)
    BAUD_MODE          = (BIT_PERIOD << 16) | (8 - 1)
    DELAY              = 180_000_000

DAT
    org 0
    asmclk

    dirl    #TRANSMISSION_PIN
    wrpin   ##(P_ASYNC_TX | P_OE), #TRANSMISSION_PIN
    wxpin   ##BAUD_MODE, #TRANSMISSION_PIN
    dirh    #TRANSMISSION_PIN
    wypin   #1, #TRANSMISSION_PIN

    main
        mov sourcePointer2, #@mystring2
        mov destinationPointer, #@mystring3
        call #string_copy

        mov sourcePointer1, #@mystring1
        mov destinationPointer, #@mystring3
        mov maxAppendCount, #2
        call #string_n_concat

        mov transmitPointer, #@mystring3
        .loop_transmit
            rdbyte currentChar, transmitPointer
            cmp     currentChar, #0 wz
            if_z    jmp #endprog
            wypin   currentChar, #TRANSMISSION_PIN
            waitx   ##DELAY
            add     transmitPointer, #1
            jmp     #.loop_transmit

        jmp #endprog

    string_copy
        .loop_copy
            rdbyte currentChar, sourcePointer2

```

```

    wrbyte currentChar, destinationPointer
    add sourcePointer2, #1
    add destinationPointer, #1
    cmp currentChar, #0 wz
    if_z ret
    jmp #.loop_copy

string_n_concat
.loop_find_end
    rdbyte currentChar, destinationPointer
    cmp currentChar, #0 wz
    if_z jmp #.append_start
    add destinationPointer, #1
    jmp #.loop_find_end

.append_start
.loop_append
    cmp maxAppendCount, #0 wz
    if_z jmp #.null_terminate

    rdbyte currentChar, sourcePointer1
    cmp currentChar, #0 wz
    if_z jmp #.null_terminate
    wrbyte currentChar, destinationPointer
    add sourcePointer1, #1
    add destinationPointer, #1
    sub maxAppendCount, #1
    jmp #.loop_append

.null_terminate
    mov     currentChar, #0
    wrbyte  currentChar, destinationPointer
ret

endprog
    jmp    #$.

mystring1      byte "World!",0, 0
mystring2      byte "Hello,",0,0,0,0,0,0
mystring3      byte 256

currentChar     res 1
sourcePointer1  res 1
sourcePointer2  res 1
destinationPointer res 1
transmitPointer res 1
maxAppendCount  res 1

```

Here we demonstrate how to copy a string and append a limited number of characters from another string in Propeller 2 assembly, then transmit the final result through a smart pin. First, the string “Hello,” in `mystring2` is copied into the buffer `mystring3` using the `string_copy` routine. This routine reads each byte from the source with `RDBYTE` and writes it to the destination with `WRBYTE`, stopping at the null terminator. Next, the `string_n_concat` routine appends up to `maxAppendCount` characters from `mystring1` (“World!”) to the end of `mystring3`. It first finds the null terminator of the destination buffer, then sequentially writes each byte from the source until either the maximum number of characters is appended or the source string’s null terminator is reached. After appending, the routine ensures the resulting string is properly null-terminated. Finally, the main program sets `transmitPointer` to the start of `mystring3` and transmits each character one by one over the `TRANSMISSION_PIN` using `WYPIN`, with a full-second delay (`WAITX ##DELAY`) between characters, stopping at the null terminator. Temporary registers—`currentChar`, `sourcePointer1`, `sourcePointer2`, `destinationPointer`, `transmitPointer`, and `maxAppendCount`—clearly indicate the role of each value in reading, writing, appending, and transmitting the string. The output will display the characters sequentially: H, e, l, l, o, ,, W, o, representing the concatenated string “Hello,Wo”. This example illustrates safe string copying, controlled-length concatenation, proper null-termination, and sequential character transmission in hub memory.

15.10 String to Decimal

Converting a numeric string to an integer involves reading ASCII characters from a null-terminated string and translating them into their numeric values. Each character is processed by subtracting the ASCII code for ‘0’ (0x30), which yields the corresponding decimal digit. For single-digit strings, this operation is equivalent to the standard `atoi` function in high-level languages. The resulting integer can then be used in arithmetic operations, comparisons, or transmitted over serial interfaces. Extending this approach with loops allows multi-digit numbers to be fully converted into usable integers.

15.10.1 Program: 0060_string_to_decimal.spin2

Listing 15.9: Program: 0060_string_to_decimal.spin2

```
'string_to_decimal.spin2
CON
    _CLKFREQ          = 180_000_000
    BAUD_RATE         = 2_000_000
    TRANSMISSION_PIN  = 62
    BIT_PERIOD        = (_CLKFREQ / BAUD_RATE)
    BAUD_MODE          = (BIT_PERIOD << 16) | (8 - 1)
    DELAY              = 180_000_000

DAT
    org 0
    asmclk

    dir1    #TRANSMISSION_PIN
    wrpin   ##(P_ASYNC_TX | P_OE), #TRANSMISSION_PIN
    wxpin   ##BAUD_MODE, #TRANSMISSION_PIN
    dirh    #TRANSMISSION_PIN
    wypin   #1, #TRANSMISSION_PIN

    main
        mov stringPointer, ##@testStr
        call #strToDec

        add numericValue, #"0"
        wypin    numericValue, #TRANSMISSION_PIN
        waitx    ##DELAY

        jmp      #endprog

    strToDec
        rdbYTE    currentChar, stringPointer
        sub       currentChar, #"0"
        mov       numericValue, currentChar
        ret

    endprog
        jmp      #$

    testStr byte "3", 0

    numericValue    res 1
    currentChar     res 1
    stringPointer    res 1
```


Here we are converting a single-digit string into its numeric value. The string “3” is stored in `testStr` as a null-terminated string. The subroutine `strToDec` reads the first byte of the string using `RDBYTE` into the temporary register `currentChar`. Since ASCII digits start at 0x30, the routine subtracts “0” from the character to obtain the numeric integer value and stores it in `numericValue`. This effectively converts the ASCII character ‘3’ into the integer 3.

In the main program, `numericValue` is then converted back to its ASCII character by adding “0” and transmitted through `TRANSMISSION_PIN` using `WYPIN`. A brief delay (`WAITX ##DELAY`) ensures proper spacing on the serial line. The temporary registers `currentChar`, `stringPointer`, and `numericValue` clearly indicate their roles: reading the input character, pointing to the source string, and storing the numeric conversion result. The output will transmit the character ‘3’, demonstrating that the program correctly converts a string digit into a number and then transmits it.

This approach is useful for processing numeric input in string form while keeping the value ready for display or further arithmetic operations. It can be extended to handle multi-digit strings by iterating through the string and accumulating the total numeric value.

15.11 Decimal to Character

Converting a single-digit integer into its ASCII representation involves adding the ASCII code for ‘0’ (0x30) to the numeric value. The routine `DecToStr` takes an integer between 0 and 9 and stores the resulting ASCII character in a buffer or variable. This character can then be transmitted over serial, displayed, or combined with other strings. This process is essentially the inverse of the `strToDec` routine and is useful for generating text output from numeric calculations in low-level P2 programs.

15.11.1 Program: 0061_decimal_to_character.spin2

Listing 15.10: Program: 0061_decimal_to_character..spin2

```
'decimal_to_character.spin2
CON
    _CLKFREQ          = 180_000_000
    BAUD_RATE         = 2_000_000
    TRANSMISSION_PIN  = 62
    BIT_PERIOD        = (_CLKFREQ / BAUD_RATE)
    BAUD_MODE          = (BIT_PERIOD << 16) | (8 - 1)
    DELAY              = 180_000_000

DAT
    org 0
    asmclk

    dirl    #TRANSMISSION_PIN
    wrpin   ##(P_ASYNC_TX | P_OE), #TRANSMISSION_PIN
    wxpin   ##BAUD_MODE, #TRANSMISSION_PIN
    dirh    #TRANSMISSION_PIN
    wypin   #1, #TRANSMISSION_PIN

    main
        mov number, #4
        call #DecToStr

        add asciiChar, #"0"
        wypin asciiChar, #TRANSMISSION_PIN
        waitx ##DELAY

        jmp #endprog

    DecToStr
        mov asciiChar, number
        ret

    endprog
        jmp #$$

    asciiChar    res 1
    number        res 1
```

The result of this program is that the single-digit integer stored in **number** (in this case 4) is converted to its corresponding ASCII character and transmitted over the **TRANSMISSION_PIN**. The integer value 4 is stored in the register **number**, and the **TRANSMISSION_PIN** is configured for asynchronous output using **WRPIN** and **WXPIN**, with **DIRH** setting the pin as output. The

subroutine `DecToStr` moves the integer from `number` into the variable `asciiChar`, and the main program adds the ASCII value of “0” to convert the integer into its ASCII representation. For example, $4 + \text{“0”}$ results in `0x34`, which corresponds to the character ‘4’. This character is then sent out through the transmission pin using `WYPIN`, with a short `WAITX` delay to ensure proper spacing for serial reception. The program’s variables include `number`, which holds the original integer, and `asciiChar`, which holds the converted character ready for output. When run, the program transmits the character ‘4’ on the serial line, demonstrating how a numeric value can be converted to a character for display or further serial communication. The approach works for any single-digit integer and the program enters an infinite loop after transmission, allowing repeated inspection if needed. This routine is a fundamental building block for displaying numeric values as characters and can be extended to handle multi-digit numbers for serial output.

15.12 Decimal to String

Converting a multi-digit decimal string into an integer requires processing each character sequentially. Each ASCII digit is read from the null-terminated string and converted to its numeric value by subtracting ‘0’ (`0x30`). The integer is accumulated using the formula $result = result \times 10 + digit$, effectively building the number as each new digit is added. Once the full string has been processed, the resulting integer can be used for calculations, comparisons, or output. For display or serial transmission, the integer can be printed one digit at a time, using a routine such as `DecToStr` to convert each digit back into an ASCII character.

15.12.1 Program: 0062_decimal_to_string.spin2

Listing 15.11: Program: 0062_decimal_to_string..spin2

```
'decimal_to_string.spin2
CON
    _CLKFREQ          = 180_000_000
    BAUD_RATE         = 2_000_000
    TRANSMISSION_PIN  = 62
    BIT_PERIOD        = (_CLKFREQ / BAUD_RATE)
    BAUD_MODE          = (BIT_PERIOD << 16) | (8 - 1)
    DELAY              = 180_000_000

DAT
    org 0
    asmclk

    dirl    #TRANSMISSION_PIN
    wrpin   ##(P_ASYNC_TX | P_OE), #TRANSMISSION_PIN
    wxpin   ##BAUD_MODE, #TRANSMISSION_PIN
    dirh    #TRANSMISSION_PIN
    wypin   #1, #TRANSMISSION_PIN

    main
        call #stringToInteger
        call #printInteger
        jmp #endprog

    stringToInteger
        mov stringPointer, ##@testString
        mov convertedInteger, #0
        .loop
            rdbYTE currentCharacter, stringPointer
            cmp currentCharacter, #0 wz
            if_z ret
            sub currentCharacter, #"0"
            qmul convertedInteger, #10
            getqX convertedInteger
            add convertedInteger, currentCharacter
            add stringPointer, #1
            jmp #.loop

    printInteger
        mov divisor, ##1_000_000_000
        mov leadingZeroFlag, #0
        mov tempInteger, convertedInteger
        .loop
            mov      currentDigit, #0
```

```

    qdiv    tempInteger, divisor
    getqx   currentDigit
    getqy   tempInteger
    cmp     currentDigit, #0 wz
    if_z cmp leadingZeroFlag, #0 wz
    if_z jmp #.skipPrint
    if_nz mov leadingZeroFlag, #1
    add     currentDigit, #"0"
    wypin   currentDigit, #TRANSMISSION_PIN
    waitx   ##DELAY
.skipPrint
    qdiv    divisor, #10
    getqx   divisor
    cmp     divisor, #0 wz
    if_nz jmp #.loop
    cmp     leadingZeroFlag, #0 wz
    if_z jmp #.loop
    ret

endprog
    jmp    $$

testString byte "451", 0

currentDigit res 1
divisor res 1
tempInteger res 1
leadingZeroFlag res 1

convertedInteger res 1
currentCharacter res 1
stringPointer res 1

```

This code takes a string of ASCII digits and converts it into an actual integer, then prints that integer as a sequence of ASCII characters. The `stringToInteger` routine begins by initializing a pointer to the null-terminated input string `testString` and a register `convertedInteger` to zero. It reads each character one by one, subtracts the ASCII value of “0” to obtain its numeric value, multiplies the accumulated integer by 10 using `QMUL`, and adds the new digit. This process continues until the null terminator is encountered, resulting in the full numeric value stored in `convertedInteger`.

The `printInteger` routine then extracts each decimal digit by dividing the integer by decreasing powers of ten using `QDIV`, retrieves both the quotient and remainder using `GETQX` and `GETQY`, converts each digit to its ASCII representation by adding “0”, and transmits it sequentially through `TRANSMISSION_PIN` with short `WAITX` delays. A leading-zero flag ensures that only significant digits are printed, skipping unnecessary zeros. For the example string “451”, the program transmits the characters ‘4’, ‘5’, and ‘1’ in order, accurately representing the numeric value of the original string. This approach demonstrates a complete round-trip

conversion between string and numeric forms in P2 assembly, suitable for processing numeric input and producing formatted output.

15.13 Decimal to string revisited

Converting a multi-digit integer into a null-terminated ASCII string involves first breaking the number into individual digits. The routine `DecToStr` stores the integer in a variable such as `result` and converts each digit into its ASCII equivalent, temporarily writing them in reverse order to a buffer. After all digits are processed, the string is reversed in-place to produce the correct sequence. The resulting null-terminated string can then be transmitted over a SmartPin using `printStr` or used elsewhere in the program. This routine functions like a classic `itoa`, handling integers up to 32 bits and enabling formatted numeric output in low-level P2 programs.

To make things simple we introduce the register `ptrA`. In Propeller 2 assembly, `ptrA` is a pointer register used to reference memory addresses for reading and writing data. It allows the program to access RAM dynamically, moving through memory locations sequentially without hardcoding addresses.

The register `ptrA` works by holding the address of the current byte in memory. Instructions like `WRBYTE` and `RDBYTE` automatically operate on the memory location pointed to by `ptrA`. Incrementing or decrementing `ptrA` moves the pointer forward or backward, enabling loops over memory for tasks such as string writing, string reversal, or sequential data access.

Because it points directly to RAM, `ptrA` makes routines flexible and reusable: the same subroutine can write to any buffer simply by setting `ptrA` to a different starting address. This eliminates hardcoded addresses and allows efficient handling of null-terminated strings.

15.13.1 Program: 0063_decimal_to_string2.spin2

Listing 15.12: Program: 0063_decimal_to_string2.spin2

```

'decimal_to_string2.spin2
CON
    _CLKFREQ          = 180_000_000
    BAUD_RATE         = 2_000_000
    TRANSMISSION_PIN  = 62
    BIT_PERIOD        = (_CLKFREQ / BAUD_RATE)
    BAUD_MODE          = (BIT_PERIOD << 16) | (8 - 1)
    DELAY              = 180_000_000

DAT
    org 0
    asmclk

    dirl    #TRANSMISSION_PIN
    wrpin   ##(P_ASYNC_TX | P_OE), #TRANSMISSION_PIN
    wxpin   ##BAUD_MODE, #TRANSMISSION_PIN
    dirh    #TRANSMISSION_PIN
    wypin   #1, #TRANSMISSION_PIN

    main
        mov result, ##994421
        mov ptra, ##@testStr

        call #decToStr
        mov ptra, ##@testStr
        call #printStr

        jmp #endprog

    decToStr
        cmp result, #0 wz
        if_z jmp #.zero_case

        mov string_start_pointer, ptra
        mov quotient, result

        .digit_loop
            qdiv quotient, #10
            getqx quotient
            getqy current_char
            add current_char, #"0"
            wrbyte current_char, ptra
            add ptra, #1
            cmp quotient, #0 wz

```

```

    if_nz jmp #.digit_loop
    wrbyte #0, ptra
    mov string_left_index, string_start_pointer
    sub ptra, #1
.reverse_loop
    cmp string_left_index, ptra wc
    if_nc ret
    rdbyte current_char, string_left_index
    rdbyte string_right_index, ptra
    wrbyte string_right_index, string_left_index
    wrbyte current_char, ptra
    add string_left_index, #1
    sub ptra, #1
    jmp #.reverse_loop
.zero_case
    mov ptra, string_start_pointer
    mov current_char, #"0"
    wrbyte current_char, ptra
    add ptra, #1
    wrbyte #0, ptra
ret

printStr
.loop
    rdbyte current_char, ptra
    cmp current_char, #0 wz
    if_z ret
    call #tx_char
    add ptra, #1
    jmp #.loop

tx_char
    wypin current_char, #TRANSMISSION_PIN
    waitx ##DELAY
    ret

endprog
    jmp #$.

testStr byte 0
digit res 1
divisor res 1
tmp res 1
number res 1
leading_zero res 1
result res 1
current_char res 1

```



```
string_start_pointer res 1
quotient res 1
string_left_index res 1
string_right_index res 1
```

This decimal to string program converts a decimal integer into a string and transmits it character by character over a serial output pin. The integer to be converted is stored in the variable `result`, and the resulting string is written into the buffer `testStr`. The main program initializes the integer and the string pointer, then calls the `decToStr` subroutine to perform the conversion and `printStr` to transmit each character sequentially through the `TRANSMISSION_PIN`.

In `decToStr`, the integer is repeatedly divided by 10 using `QDIV` to extract each digit starting from the least significant digit. Each digit is converted to its ASCII representation and stored in the string buffer. Because the digits are generated in reverse order, the subroutine subsequently reverses the string in place to obtain the correct sequence. A special case handles the number zero by directly writing the character “0” to the buffer.

The `printStr` subroutine reads each character from the null-terminated string and calls `tx_char` to transmit it with a short `WAITX` delay between characters, ensuring proper serial output timing. As a result, an integer like 994421 is converted to the string “994421” and transmitted sequentially as individual ASCII characters, producing an accurate textual representation of the original number.

15.14 Exercise

For this exercise, you will write a Propeller 2 assembly program that counts the number of vowels in a null-terminated string stored in hub memory. The program should initialize a pointer to the start of the string and then loop through each character using `RDBYTE` until it reaches the null terminator. Each character is checked to see if it is a vowel (A, E, I, O, U, both uppercase and lowercase), and a counter is incremented whenever a vowel is found. After processing the entire string, the program converts the count to its ASCII equivalent by adding “0” and transmits the result via a UART SmartPin using `WYPIN`. For example, the string “Hello Propeller 2!” should produce an output of 5. This exercise helps reinforce key string manipulation concepts such as byte-level traversal, conditional checks, counting, and serial output in Propeller 2 assembly.

15.15 Summary

We now shift focus to external interfacing, beginning with the configuration and initialization of smart pins for hardware-level input and output control.

Chapter Summary

- Strings are sequences of characters in memory
- Operations include copy, compare, and concatenate
- Loops process strings character by character
- Numeric strings require conversion
- String handling is manual and low-level

Chapter 16

SmartPin for UART)

16.1 About SmartPins

The Propeller 2’s SmartPin system simplifies handling advanced I/O tasks by offloading them from the COG processors to dedicated hardware. This allows features like UART communication, PWM signal generation, and counting to operate independently and efficiently. By leveraging SmartPins, developers can perform precise, high-speed I/O operations without using extensive processor cycles. In this chapter, we explore how to use SmartPins for asynchronous UART transmission, enabling reliable serial data transfer at a configurable baud rate while keeping the COGs free for other tasks.

On the Propeller 2, any I/O pin can be turned into a SmartPin by configuring its registers with the `WRPIN`, `WXPIN`, and `WYPIN` instructions. When a SmartPin is set to asynchronous transmit mode (`P_ASYNC_TX`), it automatically manages all aspects of UART timing and framing. This means you only need to write the data bytes to the pin, and the SmartPin takes care of sending the start bit, data bits, and stop bit—simplifying serial communication.

To receive asynchronous UART data on the Propeller 2, the SmartPin system provides dedicated instructions that simplify handling incoming signals. `TESTP` allows the program to efficiently poll the reception pin, detecting when a new byte is available without halting the COG. Once data arrives, `RDPIN` reads the byte from the SmartPin’s FIFO buffer, and bit shifting and masking isolate the meaningful 8-bit value. `AKPIN` then acknowledges that the byte has been processed, clearing the reception flag so the SmartPin can handle the next incoming byte. Using these instructions in combination, developers can implement continuous, high-speed data reception while relying on the SmartPin hardware to manage all UART timing, framing, and serialization, leaving the COG free for other operations.

The Propeller 2 instruction set also includes `RQPIN`, which can request SmartPin status or data and allows software to check for completion through status flags. However, for the purposes of this manual, the simpler `TESTP` → `RDPIN` → `AKPIN` workflow provides a clear and reliable method for UART reception, so `RQPIN` will not be used in the examples presented here.

16.1.1 Syntax

TESTP

- **destination** = where the result goes
- **source** = value or register to test
- Uses optional **#** for literals
- Sets the Carry flag based on tested bits
- Does not modify source data
- Commonly used for polling SmartPin status

RDPIN

- **destination** = register to store value
- **source** = SmartPin or register
- Reads data from SmartPin FIFO/input
- May require masking or shifting (e.g., UART 8-bit extraction)

RQPIN

- **destination** = register to store status
- **source** = SmartPin ID
- Queries SmartPin state or completion status
- Can be used with Carry flag for polling

AKPIN

- Acknowledges received SmartPin data
- Clears receive status flag
- Used after RDPIN
- Does not modify the received value

16.2 Basic UART Echo with SmartPins

We demonstrate a basic UART echo program using the Propeller 2's SmartPin system. The program receives serial data on one pin and immediately sends it back on another, providing a simple way to verify both receive (RX) and transmit (TX) functionality.

16.2.1 Configuring a SmartPin for Reception

The `WRPIN` instruction sets the role of a Propeller 2 pin, telling it whether to send signals, receive signals, or remain idle. When configured for reception, `WRPIN` prepares the pin to accept incoming data from an external source.

For reception, the pin is placed into asynchronous receive mode using `P_ASYNC_RX`, allowing it to read serial data (like UART). The `P_HIGH_15K` option enables a weak pull-up resistor, which keeps the line in a stable high state when no data is being transmitted. This prevents the pin from floating and picking up unwanted noise.

16.2.2 Syntax

```
WRPIN ##(P_ASYNC_RX | P_HIGH_15K), #RECEPTION_PIN
```

Meaning:

- `P_ASYNC_RX` = UART receive mode
- `P_HIGH_15K` = weak pull-up resistor
- `|` = combine configuration flags
- `#RECEPTION_PIN` = physical pin number

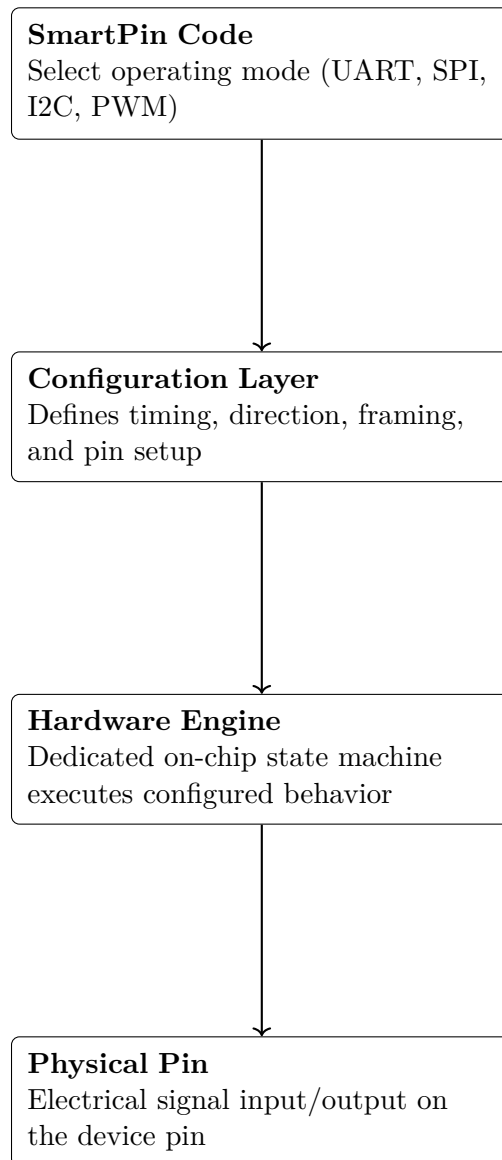


Figure 16.1: SmartPin execution flow from software configuration through hardware execution to physical I/O behavior

16.2.3 Program: 0064_echo.spin2

Listing 16.1: Program: 0064_echo.spin2

```

'echo.spin2
CON
    _CLKFREQ          = 180_000_000
    BAUD_RATE         = 2_000_000
    TRANSMISSION_PIN  = 62
    RECEPTION_PIN     = 63
    BIT_PERIOD        = (_CLKFREQ / BAUD_RATE)
    BAUD_MODE         = (BIT_PERIOD << 16) | (8 - 1)

DAT
    org 0
    asmclk

    dirl    #RECEPTION_PIN
    wrpin   ##(P_ASYNC_RX | P_HIGH_15K), #RECEPTION_PIN
    wxpin   ##BAUD_MODE, #RECEPTION_PIN
    dirh    #RECEPTION_PIN

    dirl    #TRANSMISSION_PIN
    wrpin   ##(P_ASYNC_TX | P_OE), #TRANSMISSION_PIN
    wxpin   ##BAUD_MODE, #TRANSMISSION_PIN
    dirh    #TRANSMISSION_PIN
    wypin   #1, #TRANSMISSION_PIN

main
.echo_loop
    testp   #RECEPTION_PIN wc
    if_nc   jmp     #.echo_loop
    rdpin   reception_byte, #RECEPTION_PIN
    shr     reception_byte, #24
    and     reception_byte, #255
    akpin   #RECEPTION_PIN
    wypin   reception_byte, #TRANSMISSION_PIN
    jmp     #.echo_loop

endprog
    jmp     #$

reception_byte    res 1

```

The echo program sets up the Propeller 2 to perform a continuous UART echo using SmartPin hardware for both reception and transmission. It configures one SmartPin for asynchronous RX input and another SmartPin for asynchronous TX output, setting the baud rate and timing parameters appropriately. In the main loop, the program continuously

monitors the RX pin for incoming data using `TESTP`, and when a character is detected, it reads the byte from the SmartPin FIFO, shifts and masks it to isolate the 8-bit value, acknowledges the reception with `AKPIN`, and immediately sends the same byte out through the TX pin using `WYPIN`.

This means any character sent to the RX pin is instantly echoed back; for example, sending “A” results in “A” being transmitted back, and a sequence like “1, 2, 3” is returned in the same order. This behavior confirms that the SmartPins are correctly handling UART framing and serialization, the baud rate is correctly configured, and the echo loop operates continuously without requiring the COG to manage low-level bit timing. As a result, the system provides a reliable and real-time serial interface, suitable for terminal I/O, debugging output, or interactive command-based applications.

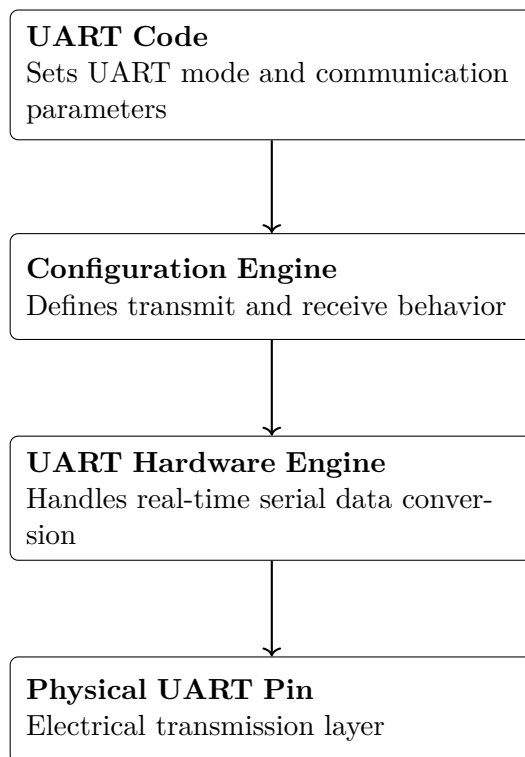


Figure 16.2: UART communication flow from software configuration to physical signal transmission

16.3 Single Digit Input and Check (0–9)

Building on the previous UART echo program, this version adds input validation. This example introduces key programming concepts such as character testing, conditional branching, and string output using SmartPins, demonstrating how the Propeller 2 can make decisions and communicate results over a serial interface.

16.3.1 Program: 0065_get_digit.spin2

Listing 16.2: Program: 0065_get_digit.spin2

```

'get_digit.spin2
CON
    _CLKFREQ          = 180_000_000
    BAUD_RATE         = 2_000_000
    TRANSMISSION_PIN  = 62
    RECEPTION_PIN     = 63
    BIT_PERIOD        = (_CLKFREQ / BAUD_RATE)
    BAUD_MODE         = (BIT_PERIOD << 16) | (8 - 1)

dat
    org 0
    asmclk

    dir1    #RECEPTION_PIN
    wrpin   ##(P_ASYNC_RX | P_HIGH_15K), #RECEPTION_PIN
    wxpin   ##BAUD_MODE, #RECEPTION_PIN
    dirh    #RECEPTION_PIN

    dir1    #TRANSMISSION_PIN
    wrpin   ##(P_ASYNC_TX | P_OE), #TRANSMISSION_PIN
    wxpin   ##BAUD_MODE, #TRANSMISSION_PIN
    dirh    #TRANSMISSION_PIN
    wypin   #1, #TRANSMISSION_PIN

main
    .get_input
    testp   #RECEPTION_PIN wc
    if_nc   jmp      #.get_input

    rdpin   input_character, #RECEPTION_PIN
    shr     input_character, #24
    and     input_character, #$FF
    akpin   #RECEPTION_PIN

    wypin   input_character, #TRANSMISSION_PIN
    call    #.flush_transmission
    cmp     input_character, #"0" wc
    if_c     jmp      #.not_number
    cmp     input_character, #"9" wcz
    if_a     jmp      #.not_number

    mov     string_pointer, ##@msg_number
    call    #.print_string
    jmp     #.done

```

```

    .not_number
        mov     string_pointer, ##@msg_notnum
        call    #.print_string
    .done
    jmp     #.get_input

.print_string
.nextch
    rdbyte     current_byte, string_pointer
    cmp        current_byte, #0 wz
    if_z       ret
    wypin      current_byte, #TRANSMISSION_PIN
    call       #.flush_transmission
    add        string_pointer, #1
    jmp        #.nextch

.flush_transmission
    rdpin      transmission_busy, #TRANSMISSION_PIN wc
    if_c       jmp     #.flush_transmission
    ret

endprog
    jmp     $$

msg_number      byte 13,10,"It is a number from 0 to 9",13,10,0,0
msg_notnum      byte 13,10,"Not a number!",13,10,0
input_character  res 1
current_byte     res 1
transmission_busy res 1
string_pointer   res 1

```

The program `get digit` enables the Propeller 2 to read characters from a terminal via UART, identify whether they are numeric digits, and provide appropriate feedback over the same serial interface. In the main loop, the program waits for a character to arrive on the RX pin, reads it from the SmartPin FIFO, isolates the 8-bit value, and immediately echoes it back for confirmation. It then checks whether the character falls between '0' and '9'. If so, it prints the message "It is a number from 0 to 9"; otherwise, it prints "Not a number!". The message printing routine reads each character from the message string and transmits it sequentially, using a flush subroutine to ensure that the transmission pin is ready before sending the next character. This cycle repeats indefinitely, demonstrating that the Propeller 2 can reliably capture user input, perform ASCII-based conditional logic, and output messages over UART in real time.

16.4 Multi-character input

Let's extend the previous serial input examples to handle multi-character input and simple string comparison. This example introduces concepts such as string buffering, looped character reading, and character-by-character comparison, which are essential foundations for building command interpreters and text-based interfaces.

16.4.1 Program: 0066_input_match.spin2

Listing 16.3: Program: 0066_input_match.spin2

```
'input_match.spin2
CON
    _CLKFREQ          = 180_000_000
    BAUD_RATE         = 2_000_000
    TRANSMISSION_PIN  = 62
    RECEPTION_PIN     = 63
    BIT_PERIOD        = (_CLKFREQ / BAUD_RATE)
    BAUD_MODE         = (BIT_PERIOD << 16) | (8 - 1)

dat
    org              0
    asmclk

    dir1    #RECEPTION_PIN
    wrpin   ##(P_ASYNC_RX | P_HIGH_15K), #RECEPTION_PIN
    wxpin   ##BAUD_MODE, #RECEPTION_PIN
    dirh    #RECEPTION_PIN

    dir1    #TRANSMISSION_PIN
    wrpin   ##(P_ASYNC_TX | P_OE), #TRANSMISSION_PIN
    wxpin   ##BAUD_MODE, #TRANSMISSION_PIN
    dirh    #TRANSMISSION_PIN
    wypin   #1, #TRANSMISSION_PIN

main
    mov buffer_pointer, #input_buffer_array

    .read_char
        testp    #RECEPTION_PIN wc
        if_nc    jmp #.read_char

        rdpin input_character, #RECEPTION_PIN
        shr input_character, #24
        and input_character, #$FF
        akpin #RECEPTION_PIN

        wypin input_character, #TRANSMISSION_PIN
        call #.flush_tx

        wrbyte input_character, buffer_pointer
        add buffer_pointer, #1

        cmp input_character, ##$0D wz
        if_nz    jmp #.read_char
```

```

    wrbyte #0, buffer_pointer
    mov command_pointer, #@cmd_hello
    mov buffer_pointer, #input_buffer_array
    call #.check_match

    jmp #main

.check_match
    rdbyte input_character, buffer_pointer
    rdbyte compare_character, command_pointer
    cmp compare_character, #0 wz
    if_z jmp #.print_world
    cmp input_character, compare_character wz
    if_nz ret
    add buffer_pointer, #1
    add command_pointer, #1
    jmp #.check_match

.print_world
    mov message_pointer, #@msg_world
    call #.print_string
    ret

.print_string
.nextch
    rdbyte compare_character, message_pointer
    cmp compare_character, #0 wz
    if_z ret
    wypin compare_character, #TRANSMISSION_PIN
    call #.flush_tx
    add message_pointer, #1
    jmp #.nextch

.flush_tx
    rdpin compare_character, #TRANSMISSION_PIN wc
    if_c jmp #.flush_tx
    ret

endprog
    jmp #$$

cmd_hello    byte "hello",0,0,0
msg_world    byte "p2asm",13,10,0
input_character    res 1
compare_character    res 1
buffer_pointer    res 1

```

<code>command_pointer</code>	<code>res 1</code>
<code>message_pointer</code>	<code>res 1</code>
<code>input_buffer_array</code>	<code>res 10</code>

The program reads a string typed from the terminal, ending when Enter is pressed. If the entered string exactly matches “hello”, it responds by printing “world” on the terminal. Any other input is ignored, and the program waits for the next string. This demonstrates that the Propeller 2 can handle multi-character input, store it in a buffer, perform character-by-character comparison with a target string, and output messages via UART using SmartPins.

Users can type different strings to verify that only the exact match triggers the response.

16.5 Line Editing with CRLF

Continuing to build a user input system, this version adds support for typing text on a terminal, correcting mistakes with the Backspace key, completing a line by pressing Enter (CR), and sending a carriage return and line feed (CRLF) sequence for clean terminal output. These enhancements are an important step toward creating interactive serial interfaces, such as command prompts or user input menus, allowing users to enter, edit, and submit text efficiently.

16.5.1 Program: 0067_line_edit.spin2

Listing 16.4: Program: 0067_line_edit.spin2

```
'line_edit.spin2
CON
    _CLKFREQ          = 180_000_000
    BAUD_RATE         = 2_000_000
    TRANSMISSION_PIN  = 62
    RECEPTION_PIN     = 63
    BIT_PERIOD        = (_CLKFREQ / BAUD_RATE)
    BAUD_MODE         = (BIT_PERIOD << 16) | (8 - 1)

DAT
    org 0
    asmclk

    dir1    #RECEPTION_PIN
    wrpin   ##(P_ASYNC_RX | P_HIGH_15K), #RECEPTION_PIN
    wxpin   ##BAUD_MODE, #RECEPTION_PIN
    dirh    #RECEPTION_PIN

    dir1    #TRANSMISSION_PIN
    wrpin   ##(P_ASYNC_TX | P_OE), #TRANSMISSION_PIN
    wxpin   ##BAUD_MODE, #TRANSMISSION_PIN
    dirh    #TRANSMISSION_PIN
    wypin   #1, #TRANSMISSION_PIN

main
    mov     buffer_pointer, #input_buffer_array

    .read_char
        testp    #RECEPTION_PIN wc
        if_nc    jmp     #.read_char

        rdpin    input_character, #RECEPTION_PIN
        akpin    #RECEPTION_PIN

        shr     input_character, #24
        and     input_character, #$FF

        cmp     input_character, ##$0D wz
        if_z    jmp     #.line_done

        cmp     input_character, ##$08 wz
        if_z    jmp     #.handle_backspace

        cmp     input_character, ##$7F wz
```

```

    if_z      jmp      #.handle_backspace

    wypin     input_character, #TRANSMISSION_PIN
    call      #flush_tx

    wrbyte    input_character, buffer_pointer
    add       buffer_pointer, #1

    jmp       #.read_char

.handle_backspace
    cmp       buffer_pointer, #input_buffer_array wz
    if_z      jmp      #.read_char

    sub       buffer_pointer, #1

    wypin     ##$08, #TRANSMISSION_PIN
    call      #flush_tx
    wypin     ##32, #TRANSMISSION_PIN
    call      #flush_tx
    wypin     ##$08, #TRANSMISSION_PIN
    call      #flush_tx

    jmp       #.read_char

.line_done
    wrbyte    #0, buffer_pointer

    wypin     ##$0D, #TRANSMISSION_PIN
    call      #flush_tx
    wypin     ##$0A, #TRANSMISSION_PIN
    call      #flush_tx

    jmp       #main

flush_tx
    rdpin     transmission_busy, #TRANSMISSION_PIN wc
    if_c      jmp      #flush_tx
    ret

endprog
    jmp      #$.

input_character      res 1
transmission_busy    res 1
buffer_pointer       res 1
input_buffer_array   res 32

```


Users can type text into the terminal, with each character immediately echoed back. If a mistake is made, pressing Backspace removes the last character both visually on the terminal and from the internal input buffer. Pressing Enter (CR) completes the line, sending a carriage return and line feed (CRLF) to move to a new line cleanly. The typed line is null-terminated in memory, ready for further processing. This demonstrates real-time character handling, line editing, and proper terminal formatting, forming the foundation for interactive command prompts, menus, or other text-based serial interfaces.

16.6 Read Line

This read line example demonstrates a complete UART input routine for the Propeller 2, capable of receiving text from a terminal, echoing typed characters, handling Backspace editing, ending input when Enter (CR) is pressed, and outputting a proper Carriage Return + Line Feed (CRLF) to format the text correctly on the terminal. By combining SmartPin setup, character echoing, and simple text editing, this chapter establishes the foundation for building more advanced interactive serial programs.

16.6.1 Program: 0068_read_line.spin2

Listing 16.5: Program: 0068_read_line.spin2

```

' read_line.spin2
CON
    _CLKFREQ          = 180_000_000
    BAUD_RATE         = 2_000_000
    TRANSMISSION_PIN  = 62
    RECEPTION_PIN     = 63
    BIT_PERIOD        = (_CLKFREQ / BAUD_RATE)
    BAUD_MODE         = (BIT_PERIOD << 16) | (8 - 1)

DAT
    org 0
    asmclk

    dir1    #RECEPTION_PIN
    wrpin   ##(P_ASYNC_RX | P_HIGH_15K), #RECEPTION_PIN
    wxpin   ##BAUD_MODE, #RECEPTION_PIN
    dirh    #RECEPTION_PIN

    dir1    #TRANSMISSION_PIN
    wrpin   ##(P_ASYNC_TX | P_OE), #TRANSMISSION_PIN
    wxpin   ##BAUD_MODE, #TRANSMISSION_PIN
    dirh    #TRANSMISSION_PIN
    wypin   #1, #TRANSMISSION_PIN

main
    mov input_buffer_arg, #input_buffer_array
    mov max_buffer_length, #32
    call #read_line
    jmp #main

read_line
    mov buffer_pointer, input_buffer_arg
    mov character_count, #0
    .read_char
        testp    #RECEPTION_PIN wc
        if_nc    jmp #.read_char

        rdpin    input_character, #RECEPTION_PIN
        akpin    #RECEPTION_PIN

        shr      input_character, #24
        and      input_character, #$FF

        cmp      input_character, ##$0D wz

```

```

    if_z      jmp #.line_done

    cmp       input_character, ##$08 wz
    if_z      jmp #.handle_backspace

    cmp       input_character, ##$7F wz
    if_z      jmp #.handle_backspace

    cmp       character_count, max_buffer_length wz
    if_z      jmp #.read_char

    wypin     input_character, #TRANSMISSION_PIN
    call      #flush_tx

    wrbyte    input_character, buffer_pointer
    add       buffer_pointer, #1
    add       character_count, #1

    jmp       #.read_char

.handle_backspace
    cmp       character_count, #0 wz
    if_z      jmp #.read_char

    sub       buffer_pointer, #1
    sub       character_count, #1

    wypin     ##$08, #TRANSMISSION_PIN
    call      #flush_tx
    wypin     ##32, #TRANSMISSION_PIN
    call      #flush_tx
    wypin     ##$08, #TRANSMISSION_PIN
    call      #flush_tx

    jmp       #.read_char

.line_done
    wrbyte    #0, buffer_pointer

    wypin     ##$0D, #TRANSMISSION_PIN
    call      #flush_tx
    wypin     ##$0A, #TRANSMISSION_PIN
    call      #flush_tx
ret

flush_tx
    rdpin     transmission_busy, #TRANSMISSION_PIN wc

```

```
    if_c      jmp #flush_tx
    ret

endprog
    jmp  $$

input_character      res 1
transmission_busy    res 1
buffer_pointer       res 1
character_count      res 1
input_buffer_arg     res 1
max_buffer_length    res 1
input_buffer_array   res 32
```

Users can type text into the terminal, with each character immediately echoed back. If a mistake is made, pressing Backspace removes the last character both from the display and the internal input buffer. Pressing Enter (CR) completes the line, automatically sending a Carriage Return + Line Feed (CRLF) to move to a new line on the terminal. The typed line is stored in memory as a null-terminated string, ready for further processing. This demonstrates real-time character handling, line editing, and proper terminal formatting, forming the foundation for interactive command prompts, menus, or other text-based serial interfaces on the Propeller 2.

16.7 Send String

Here we learn how to use the Propeller 2 SmartPins for full UART communication, including reading input strings from a terminal, comparing text, and sending replies. This example demonstrates reading user input using a subroutine `read_line`, echoing typed characters, sending strings back via the subroutine `send_string`, and comparing the input to a predefined command.

16.7.1 Program: 0069_send_string.spin2

Listing 16.6: Program: 0069_send_string.spin2

```
'send_string.spin2
CON
    _CLKFREQ          = 180_000_000
    BAUD_RATE         = 2_000_000
    TRANSMISSION_PIN  = 62
    RECEPTION_PIN     = 63
    BIT_PERIOD        = (_CLKFREQ / BAUD_RATE)
    BAUD_MODE         = (BIT_PERIOD << 16) | (8 - 1)

DAT
    org 0
    asmclk

    dirl    #RECEPTION_PIN
    wrpin   ##(P_ASYNC_RX | P_HIGH_15K), #RECEPTION_PIN
    wxpin   ##BAUD_MODE, #RECEPTION_PIN
    dirh    #RECEPTION_PIN

    dirl    #TRANSMISSION_PIN
    wrpin   ##(P_ASYNC_TX | P_OE), #TRANSMISSION_PIN
    wxpin   ##BAUD_MODE, #TRANSMISSION_PIN
    dirh    #TRANSMISSION_PIN
    wypin   #1, #TRANSMISSION_PIN

main
    mov input_buffer_arg, #input_buffer
    mov max_buffer_length, #32
    call #read_line

    mov compare_pointer, #@cmd_hello
    mov buffer_pointer, #input_buffer

check_loop
    rdbYTE input_character, buffer_pointer
    rdbYTE compare_character, compare_pointer
    cmp input_character, compare_character wz
    if_nz jmp #skip_response
    cmp input_character, #0 wz
    if_z  jmp #send_world
    add buffer_pointer, #1
    add compare_pointer, #1
    jmp #check_loop

send_world
```

```

    mov input_buffer_arg, #@msg_world
    call #send_string
    jmp #main

skip_response
    jmp #main

read_line
    mov buffer_pointer, input_buffer_arg
    mov character_count, #0

read_char
    testp #RECEPTION_PIN wc
    if_nc jmp #read_char
    rdpin input_character, #RECEPTION_PIN
    shr input_character, #24
    and input_character, #$FF
    akpin #RECEPTION_PIN

    cmp input_character, ##$0D wz
    if_z jmp #line_done
    cmp input_character, ##$08 wz
    if_z jmp #handle_backspace

    wypin input_character, #TRANSMISSION_PIN
    call #flush_transmission

    cmp character_count, max_buffer_length wz
    if_z jmp #read_char
    wrbyte input_character, buffer_pointer
    add buffer_pointer, #1
    add character_count, #1
    jmp #read_char

handle_backspace
    cmp character_count, #0 wz
    if_z jmp #read_char
    sub buffer_pointer, #1
    sub character_count, #1
    wypin ##$08, #TRANSMISSION_PIN
    wypin ##32, #TRANSMISSION_PIN
    wypin ##$08, #TRANSMISSION_PIN
    call #flush_transmission
    jmp #read_char

line_done
    wrbyte #0, buffer_pointer

```

```

    wypin ##$0D, #TRANSMISSION_PIN
    wypin ##$0A, #TRANSMISSION_PIN
    call #flush_transmission
    ret

send_string
    mov string_pointer, input_buffer_arg
loop_send
    rdbYTE output_character, string_pointer
    cmp output_character, #0 wz
    if_z jmp #done_send
    wypin output_character, #TRANSMISSION_PIN
    call #flush_transmission
    add string_pointer, #1
    jmp #loop_send
done_send
    ret

flush_transmission
    rdpin transmission_busy, #TRANSMISSION_PIN wc
    if_c jmp #flush_transmission
    ret

endprog
    jmp #$$

cmd_hello byte "hello", 0, 0, 0
msg_world byte "p2 assembly!", 0

input_character      res 1
compare_character    res 1
output_character      res 1
buffer_pointer       res 1
character_count      res 1
input_buffer_arg     res 1
max_buffer_length    res 1
string_pointer       res 1
compare_pointer      res 1
transmission_busy    res 1
input_buffer         res 32

```

As the user types, each character is immediately echoed back. Mistakes can be corrected using Backspace, which removes the character both visually and from the input buffer. Pressing Enter (CR) completes the input line, sending a proper Carriage Return + Line Feed (CRLF) to the terminal. The program then compares the typed line to the predefined command string "hello". If the input matches, the system responds by sending "world" back to the terminal; otherwise, it ignores the input and waits for a new line. All communi-

cation occurs at 2,000,000 baud, demonstrating real-time input handling, text comparison, and string output over UART using the Propeller 2 SmartPin system.

16.8 Tokenized Echo Command

We extend the UART functionality by introducing string tokenization, allowing the Propeller 2 to parse an input line into individual words or parameters. Users can type simple commands. By recognizing the first token as a command, the system can respond appropriately—in this case, sending the second token back to the terminal. This approach lays the groundwork for more advanced command-line interfaces and interactive serial control on the Propeller 2.

16.8.1 Program: 0070_token_echo.spin2

Listing 16.7: Program: 0070_token_echo.spin2

```
'token_echo.spin2
CON
    _CLKFREQ          = 180_000_000
    BAUD_RATE         = 2_000_000
    TRANSMISSION_PIN  = 62
    RECEPTION_PIN     = 63
    BIT_PERIOD        = (_CLKFREQ / BAUD_RATE)
    BAUD_MODE         = (BIT_PERIOD << 16) | (8 - 1)

DAT
    org 0
    asmclk

    dir1    #RECEPTION_PIN
    wrpin   ##(P_ASYNC_RX | P_HIGH_15K), #RECEPTION_PIN
    wxpin   ##BAUD_MODE, #RECEPTION_PIN
    dirh    #RECEPTION_PIN

    dir1    #TRANSMISSION_PIN
    wrpin   ##(P_ASYNC_TX | P_OE), #TRANSMISSION_PIN
    wxpin   ##BAUD_MODE, #TRANSMISSION_PIN
    dirh    #TRANSMISSION_PIN
    wy-pin  #1, #TRANSMISSION_PIN

main
    mov input_buffer_arg, #input_buffer
    mov max_length, #32
    call #read_line

    mov buffer_pointer, #input_buffer
    call #parse_two_tokens

    mov compare_ptr, #cmd_echo
    mov string_pointer, token1_ptr

.check_token1
    rdbYTE input_character, string_pointer
    rdbYTE compare_character, compare_ptr
    cmp input_character, compare_character wz
    if_nz jmp #main
    cmp input_character, #0 wz
    if_z jmp #.send_token2
    add string_pointer, #1
    add compare_ptr, #1
```

```

    jmp #.check_token1

.send_token2
    mov input_buffer_arg, token2_ptr
    call #send_string
    jmp #main

parse_two_tokens
    mov token1_ptr, buffer_pointer
    mov token2_ptr, #0

.loop1
    rdbyte input_character, buffer_pointer
    cmp input_character, #0 wz
    if_z jmp #.done_tokens
    cmp input_character, ##32 wz
    if_z jmp #.found_space1
    add buffer_pointer, #1
    jmp #.loop1

.found_space1
    wrbyte #0, buffer_pointer
    add buffer_pointer, #1
    mov token2_ptr, buffer_pointer

.skip_spaces
    rdbyte input_character, buffer_pointer
    cmp input_character, ##32 wz
    if_z add buffer_pointer, #1
    if_nz jmp #.done_tokens
    jmp #.skip_spaces

.done_tokens
ret

read_line
    mov buffer_pointer, input_buffer_arg
    mov character_count, #0

.read_char
    testp #RECEPTION_PIN wc
    if_nc jmp #.read_char

    rdpin input_character, #RECEPTION_PIN
    akpin #RECEPTION_PIN

    shr input_character, #24

```

```

    and input_character, #$FF

    cmp input_character, ##$0D wz
    if_z jmp #.line_done

    cmp input_character, ##$08 wz
    if_z jmp #.handle_backspace

    cmp input_character, ##$7F wz
    if_z jmp #.handle_backspace

    cmp character_count, max_length wz
    if_z jmp #.read_char

    wypin input_character, #TRANSMISSION_PIN
    call #flush_tx

    wrbyte input_character, buffer_pointer
    add buffer_pointer, #1
    add character_count, #1
    jmp #.read_char

.handle_backspace
    cmp character_count, #0 wz
    if_z jmp #.read_char

    sub buffer_pointer, #1
    sub character_count, #1

    wypin ##$08, #TRANSMISSION_PIN
    call #flush_tx
    wypin ##32, #TRANSMISSION_PIN
    call #flush_tx
    wypin ##$08, #TRANSMISSION_PIN
    call #flush_tx

    jmp #.read_char

.line_done
    wrbyte #0, buffer_pointer
    wypin ##$0D, #TRANSMISSION_PIN
    call #flush_tx
    wypin ##$0A, #TRANSMISSION_PIN
    call #flush_tx
ret

send_string

```

```

    mov string_pointer, input_buffer_arg
.loop_send
    rdbYTE input_character, string_pointer
    cmp input_character, #0 wz
    if_z jmp #.done_send
    wypin input_character, #TRANSMISSION_PIN
    call #flush_tx
    add string_pointer, #1
    jmp #.loop_send
.done_send
ret

flush_tx
    rdpin transmission_busy, #TRANSMISSION_PIN wc
    if_c jmp #flush_tx
    ret

endprog
    jmp #$.

cmd_echo byte "echo", 0

input_character      res 1
compare_character    res 1
transmission_busy    res 1
buffer_pointer       res 1
character_count      res 1
input_buffer_arg     res 1
max_length           res 1
string_pointer       res 1
token1_ptr           res 1
token2_ptr           res 1
compare_ptr          res 1
input_buffer         res 32

```

This code demonstrates a basic Propeller 2 command-line interface over UART. It reads a line of text from the terminal, echoing each character as it is typed and supporting backspace for corrections. When Enter is pressed, the input is split into two tokens. The first token is compared to the predefined command "echo", and if it matches, the second token is sent back to the terminal. Lines that do not match are ignored, and the program loops to wait for new input. This example showcases interactive text input, simple string parsing, command recognition, and UART output — forming the foundation for flexible serial command interfaces.

16.9 Exercise

Read a line of text from a terminal via UART using SmartPins and count how many times a specific character appears in the input. Users type a line of text, and each character is echoed back in real time. The program scans the input buffer character by character, comparing each one to a target character (for example, the letter “A”) and incrementing a counter whenever a match is found. Once the user presses Enter, the program prints the total count to the terminal. This exercise teaches how to implement looped character processing, comparison, accumulation, and output formatting while using SmartPins for UART communication, building on earlier concepts of reading, echoing, and handling text input.

16.10 Summary

Finally, we extend our study of serial communication by introducing SPI, a high-speed synchronous protocol used for efficient peripheral interfacing.

Chapter Summary

- SmartPins handle complex I/O automatically
- Setup requires: `wrpin` → `wxpin` → `dirh` → `wypin`
- UART depends on correct timing configuration
- Input and output must be handled explicitly
- Enables serial communication without bit-banging

Chapter 17

Introduction to SPI (Serial Peripheral Interface)

17.1 What is SPI?

SPI, or Serial Peripheral Interface, is one of the most widely used communication protocols in microcontrollers. It provides a fast, full-duplex connection between a master device—typically the microcontroller—and one or more slave devices, such as EEPROMs, sensors, displays, flash chips, or DACs. Unlike UART, which transmits data asynchronously, SPI operates synchronously, meaning that data transmission is coordinated by a shared clock signal. This synchronization enables higher speeds and precise timing control. On the Pro- peller 2, SmartPins can manage SPI in hardware, making communication both efficient and flexible.

SPI communication relies on four main signals. SCLK is the clock line generated by the master, which synchronizes data transfer. MOSI (Master Out, Slave In) carries data sent from the master to the slave, while MISO (Master In, Slave Out) carries data from the slave back to the master. CS (Chip Select) is used by the master to enable communication with a specific slave device. Only one slave's CS line is active (low) at a time; when a slave's CS line is high, it ignores all SPI bus activity. Together, these four lines enable fast, full-duplex, and precisely timed data exchange between master and slave devices.

In SPI communication, the clock signal acts like a steady beat or metronome that the master device creates to keep everything in sync. Each tick of the clock determines when a bit is sampled or shifted. The timing is defined by two settings: CPOL (Clock Polarity), which determines whether the clock idles high or low, and CPHA (Clock Phase), which determines whether data is captured on the leading or trailing edge of the clock. Together, these define four SPI modes: Mode 0 (idle low, sample on rising edge), Mode 1 (idle low, sample on falling edge), Mode 2 (idle high, sample on falling edge), and Mode 3 (idle high, sample on rising edge). Both master and slave must use the same mode for correct communication.

In SPI, data transfer begins when the master selects a slave by pulling its CS line low. The master then sends data bits through MOSI while simultaneously receiving data through MISO. Each clock cycle shifts one bit in both directions. After the transfer completes, the master sets CS high to deselect the slave. SPI is full-duplex, meaning transmission and

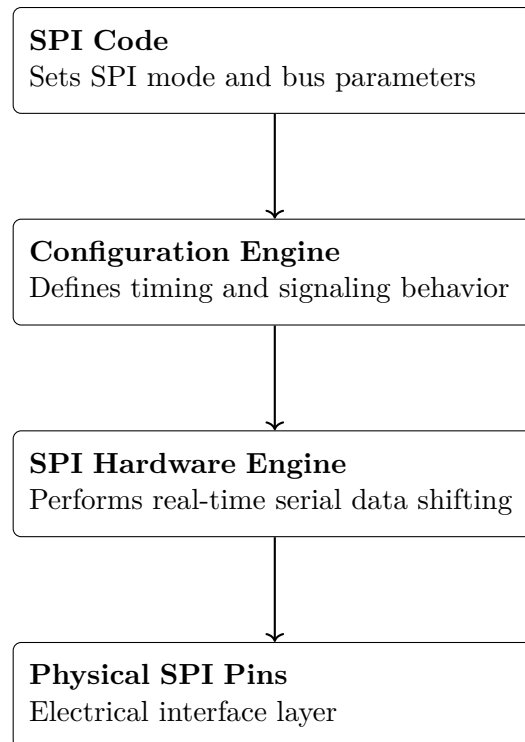


Figure 17.1: SPI smartpin execution flow from configuration to physical signaling

reception occur at the same time. Even when only sending data, the master still receives incoming bits from the slave, often treated as dummy data.

Learning SPI is valuable because it provides fast, versatile, and precise communication between the Propeller 2 and a wide range of peripherals. SPI supports high-speed full-duplex transfers, often reaching tens of megahertz, making it ideal for time-sensitive tasks like streaming graphics to displays or reading high-speed sensor data. Its simple hardware design uses basic logic-level pins without complex addressing, allowing devices to be controlled directly through shifting and toggling or by using the Propeller 2's SmartPins for hardware-timed communication. SPI is widely supported across industry-standard devices, including flash memory, EEPROMs, ADCs, DACs, displays, real-time clocks, and various sensors, making it a universal embedded communication standard. Mastering SPI also lays the foundation for more advanced protocols such as QSPI, SD card SPI mode, and HyperBus memory interfaces.

17.1.1 Example

Here is the general flow of SPI operations in assembly (pseudocode style):

```

set CS low          ; select device
repeat N bits
  output bit on MOSI
  toggle clock
  input bit from MISO

```

```
set CS high          ; deselect device
```

In practice, this can be implemented using SmartPins for hardware-driven SPI timing.

17.2 Reading the JEDEC ID from SPI Flash

Flash memory is a non-volatile storage medium, meaning it retains data even when power is removed. It is widely used in embedded systems for boot memory, firmware updates, data logging, and configuration storage. On the Propeller 2, SPI Flash typically holds the program image for booting, but user code can also access it by reading and writing bytes through SPI commands. A common first step in working with SPI Flash is reading the JEDEC ID, a standardized three-byte identifier that provides the manufacturer ID, memory type, and capacity code. This confirms that the SPI bus is functioning and ensures correct communication with the Flash device.

Most SPI Flash chips support the Read JEDEC ID command (0x9F). When the Propeller 2 sends this command, the Flash responds with three bytes: the first byte is the Manufacturer ID (for example, 0xEF for Winbond or 0xC2 for Macronix), the second byte is the Memory Type ID, and the third byte is the Capacity Code, which indicates the chip's size (for example, 0x18 = 128 Mbit = 16 Mbit). Reading the JEDEC ID is useful for hardware verification, identifying the exact Flash device for bootloader purposes, and preparing for memory operations such as reading, writing, or erasing sectors. This method works with nearly all SPI Flash chips, making it a reliable first step in SPI Flash programming.

17.2.1 Program: 0071_spi_display_jedec.spin2

Listing 17.1: Program: 0071_spi_display_jedec.spin2

```
'spi_display_jedec.spin2
CON
    _CLKFREQ          = 180_000_000
    BAUD_RATE         = 2_000_000
    TRANSMISSION_PIN  = 62
    RECEPTION_PIN     = 63
    BIT_PERIOD        = (_CLKFREQ / BAUD_RATE)
    BAUD_MODE         = (BIT_PERIOD << 16) | (8 - 1)

    SPI_CS            = 61
    SPI_CLK            = 60
    SPI_MOSI           = 59
    SPI_MISO           = 58

    SPI_CLK_MODE       = P_TRANSITION + P_OE
    SPI_MOSI_MODE      = P_SYNC_TX + P_OE | ((SPI_CLK - SPI_MOSI) & %111)
                        << 24
    SPI_MISO_MODE      = P_SYNC_RX | ((SPI_CLK - SPI_MISO) & %111) << 24
    SPI_CLK_DIV        = (_CLKFREQ / 100_000_000) #> 4

    SPI_8BIT_CONTROL   = $27

DAT
    org 0
    asmclk

    dirl    #TRANSMISSION_PIN
    wrpin   ##(P_ASYNC_TX | P_OE), #TRANSMISSION_PIN
    wxpin   ##BAUD_MODE, #TRANSMISSION_PIN
    dirh    #TRANSMISSION_PIN
    wy-pin  #1, #TRANSMISSION_PIN

    wrpin   ##SPI_CLK_MODE, #SPI_CLK
    wrpin   ##SPI_MOSI_MODE, #SPI_MOSI
    wrpin   ##SPI_MISO_MODE, #SPI_MISO
    wxpin   ##SPI_CLK_DIV, #SPI_CLK
    drvl    #SPI_CLK

    wxpin   ##SPI_8BIT_CONTROL, #SPI_MOSI
    wxpin   ##SPI_8BIT_CONTROL, #SPI_MISO

    call    #print_jedec

    jmp     #endprog
```

```

print_jedec
    drvl    #SPI_CS

    mov spi_command, #$9F
    call #spi_transmit

    call #spi_receive
    mov    pr0, spi_reception
    call   #print_hex_byte
    mov    uart_transmission, #" "
    call   #uart_transmit_char

    call #spi_receive
    mov    pr0, spi_reception
    call   #print_hex_byte
    mov    uart_transmission, #" "
    call   #uart_transmit_char

    call #spi_receive
    mov    pr0, spi_reception
    call   #print_hex_byte
    mov    uart_transmission, #" "
    call   #uart_transmit_char

    drvh    #SPI_CS

    mov    uart_transmission, #13
    call   #uart_transmit_char
    mov    uart_transmission, #10
    call   #uart_transmit_char
ret

spi_transmit
    shl spi_command, #24
    rev spi_command

    wypin spi_command, #SPI_MOSI
    drvl #SPI_MOSI

    wypin #16, #SPI_CLK

    .waitRdy
        testp #SPI_CLK wz
        if_nz jmp #.waitRdy
ret

```

```

spi_receive
    drvl    #SPI_MISO
    fltl    #SPI_MOSI
    wypin   #16,#SPI_CLK
.waitDone
    testp   #SPI_MISO wz
    if_nz   jmp #.waitDone
    rdpin   spi_reception,#SPI_MISO
    rev     spi_reception
    zerox   spi_reception,#7
    ret

print_hex_byte
    mov hexadecimal_copy, pr0
    shr pr0, #4
    and pr0, #$F
    call #print_nibble
    mov pr0, hexadecimal_copy
    and pr0, #$F
    call #print_nibble
    ret

print_nibble
    cmp pr0, #10 wc
    if_c add pr0, #"0"
    if_nc add pr0, #"A"-10
    mov uart_transmission, pr0
    call #uart_transmit_char
    ret

uart_transmit_char
    wypin   uart_transmission, #TRANSMISSION_PIN
    .txwait
    rdpin   pr2, #TRANSMISSION_PIN wc
    if_c    jmp #.txwait
    ret

endprog
    jmp    $$

spi_command          res 1
spi_reception        res 1
uart_transmission    res 1
hexadecimal_copy     res 1

```

The program `spi_display_jedec.spin2` initializes the Propeller 2 for both UART and

SPI communication to read the JEDEC ID from an SPI flash device and display it over a serial terminal. The `CON` section defines system parameters, including the system clock `_CLKFREQ`, UART baud rate `BAUD_RATE`, and pin assignments for UART (`TRANSMISSION_PIN`, `RECEPTION_PIN`) and SPI (`SPI_CS`, `SPI_CLK`, `SPI_MOSI`, `SPI_MISO`). It also configures smartpin modes for asynchronous UART and synchronous SPI, sets the SPI clock divider (`SPI_CLK_DIV`) to control the SPI speed, and defines 8-bit SPI transfers. The `DAT` section contains the main program, which initializes the UART and SPI smartpins, sets the SPI pins low or high as needed, and ensures proper pin directions.

The `print_jedec` routine selects the SPI flash by pulling CS low, sends the JEDEC ID command (0x9F) via `spi_transmit`, and receives three bytes using `spi_receive`. Each received byte is reversed to account for MSB-first SPI convention, masked to extract valid bits, and printed as hexadecimal using `print_hex_byte` and `print_nibble`. Between bytes, spaces are transmitted over UART for readability. After all three bytes are sent, CS is set high with `drvh`, and newline characters are transmitted to finish the output. The `spi_transmit` and `spi_receive` routines handle full-duplex SPI communication at the hardware level using smartpins, including clock pulses and bit reversal. The `print_hex_byte` and `print_nibble` routines convert numeric SPI data into ASCII hexadecimal characters for terminal display, and `uart_transmit_char` ensures each character is fully transmitted before continuing.

As a result, when executed with a typical SPI flash such as a Winbond 16 MB device, the program outputs the JEDEC ID in hexadecimal format (for example, `EF 40 18`), where the first byte identifies the manufacturer, the second indicates memory type, and the third specifies capacity. This demonstrates how the Propeller 2 can read SPI flash identification data and display it over UART using smartpin hardware for efficient asynchronous and synchronous serial communication.

17.3 Reading from SPI Flash

The next step is to focus on reading a single byte from an SPI Flash chip at a specific address. This operation introduces the basic SPI communication commands needed to access flash memory and serves as the foundation for more advanced tasks, such as reading larger memory blocks, erasing sectors, or writing new data. By mastering byte-level reads, you gain the skills necessary to interact with flash memory for firmware updates, configuration storage, or other embedded system applications. Understanding this process is essential for safely and efficiently accessing and managing non-volatile memory.

To read a byte from an SPI Flash chip, the microcontroller first pulls the chip select (CS) line low to select the device. It then sends the 0x03 READ command over SPI, followed by the 24-bit address of the memory location to access. The flash chip responds by sending the requested byte back on the MISO line. After the byte is received, the CS line is set high to deselect the chip, and the data can be printed over UART to verify the operation. This sequence establishes the core procedure for all SPI Flash interactions, forming the basis for reading larger blocks, updating memory, or performing other flash operations.

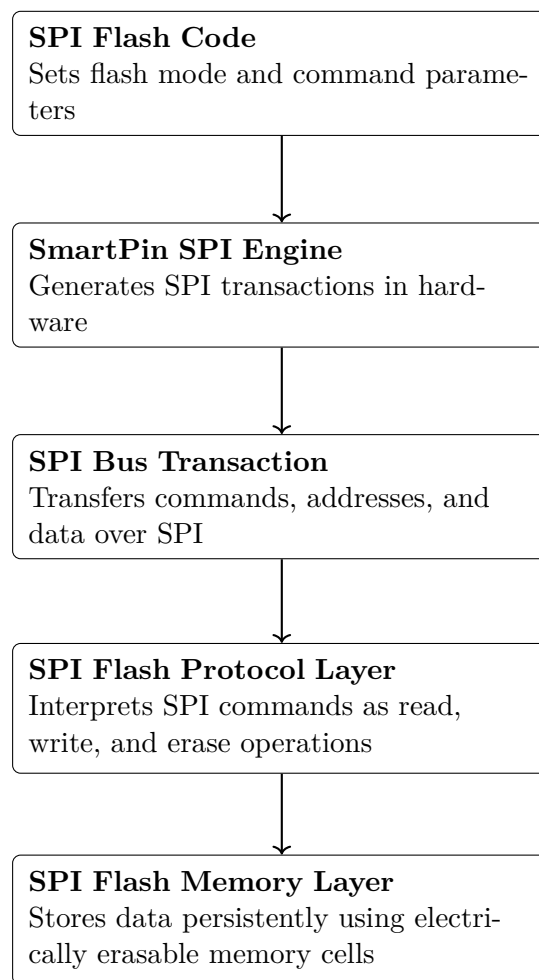


Figure 17.2: SPI flash communication and storage abstraction flow from software to physical memory

17.3.1 Program: 0072_spi_read_byte.spin2

Listing 17.2: Program: 0072_spi_read_byte.spin2

```

'spi_read_byte.spin2
CON
    _CLKFREQ          = 180_000_000
    BAUD_RATE         = 2_000_000
    TRANSMISSION_PIN  = 62
    RECEPTION_PIN     = 63
    BIT_PERIOD        = (_CLKFREQ / BAUD_RATE)
    BAUD_MODE         = (BIT_PERIOD << 16) | (8 - 1)

    SPI_CS            = 61
    SPI_CLK           = 60
    SPI_MOSI          = 59
    SPI_MISO          = 58

    SPI_CLK_MODE      = P_TRANSITION + P_OE
    SPI_MOSI_MODE     = P_SYNC_TX + P_OE | ((SPI_CLK - SPI_MOSI) & %111)
                        << 24
    SPI_MISO_MODE     = P_SYNC_RX | ((SPI_CLK - SPI_MISO) & %111) << 24
    SPI_CLK_DIV       = (_CLKFREQ / 100_000_000) #> 4
    SPI_8BIT_CONTROL  = $27

DAT
    org 0
    asmclk

    dir1    #TRANSMISSION_PIN
    wrpin   ##(P_ASYNC_TX | P_OE), #TRANSMISSION_PIN
    wxpin   ##BAUD_MODE, #TRANSMISSION_PIN
    dirh    #TRANSMISSION_PIN
    wypin   #1, #TRANSMISSION_PIN

    wrpin   ##SPI_CLK_MODE, #SPI_CLK
    wrpin   ##SPI_MOSI_MODE, #SPI_MOSI
    wrpin   ##SPI_MISO_MODE, #SPI_MISO
    wxpin   ##SPI_CLK_DIV, #SPI_CLK
    drvl    #SPI_CLK

    wxpin   ##SPI_8BIT_CONTROL, #SPI_MOSI
    wxpin   ##SPI_8BIT_CONTROL, #SPI_MISO

    mov flash_address, ##0
    mov page_size, ##255

    call #read_flash_256

```

```

jmp    #endprog

read_flash_256
    drvl #SPI_CS
    mov spi_command, #$03
    call #spi_transmit

    mov spi_command, flash_address
    shr spi_command, #16
    and spi_command, #$FF
    call #spi_transmit

    mov spi_command, flash_address
    shr spi_command, #8
    and spi_command, #$FF
    call #spi_transmit

    mov spi_command, flash_address
    and spi_command, #$FF
    call #spi_transmit

    mov byte_count, ##512

.read_loop
    call #spi_receive
    mov uart_transmission, spi_reception
    call #uart_transmit_char
    djnz byte_count, #.read_loop

    drvh #SPI_CS
    ret

spi_transmit
    shl spi_command, #24
    rev spi_command
    wypin spi_command, #SPI_MOSI
    drvl #SPI_MOSI
    wypin #16, #SPI_CLK
    waitx ##1
    .waitRdy
        testp    #SPI_CLK wz
        if_nz    jmp #.waitRdy
    ret

spi_receive
    drvl #SPI_MISO

```

```

    flt1 #SPI_MOSI
    wypin #16,#SPI_CLK
    waitx ##1
    .waitDone
        testp #SPI_CLK wz
        if_nz jmp #.waitDone
        rdpin spi_reception,#SPI_MISO
        rev spi_reception
        zerox spi_reception,#7
    ret

uart_transmit_char
    wypin uart_transmission, #TRANSMISSION_PIN
    .txwait
        rdpin pr2, #TRANSMISSION_PIN wc
        if_c jmp #.txwait
    ret

endprog
    jmp #\$

spi_command      res 1
flash_address    res 1
byte_count       res 1
spi_reception     res 1
uart_transmission res 1
page_size        res 1

```

The program begins by initializing `flash_address` to 0x000000, pointing to the first byte of flash memory, and `page_size` to 255, which sets up the block size for reading. The main routine calls `read_flash_256` to handle the SPI read, then jumps to `endprog` when complete. This structure separates setup from the actual memory reading, making the code modular and reusable for different addresses or page sizes.

In `read_flash_256`, CS is pulled low to select the flash chip, and the 0x03 READ command is sent to initiate reading. The 24-bit address is transmitted in three bytes (MSB first) to indicate where to start reading. A loop then reads 512 bytes sequentially via `spi_receive`, storing each byte in `spi_reception` and sending it over UART with `uart_transmit_char`. Finally, CS is pulled high to end the transaction.

The `spi_transmit` subroutine sends a byte over SPI by shifting and reversing it, writing it to MOSI, and pulsing the clock until the transfer completes. Conversely, `spi_receive` reads a byte from MISO, reversing it to ensure correct interpretation. `uart_transmit_char` sends a byte over UART, waiting for the previous transmission to finish, ensuring clean output.

When run on a Propeller 2 connected to SPI Flash, the program reads 512 consecutive bytes starting from address 0x000000 and prints them over UART. If the flash contains ASCII text, readable characters appear; otherwise, the output may look like binary data,

confirming that SPI communication and memory reading work correctly.

17.4 Writing to SPI Flash

Flash memory often includes hardware protection features to prevent accidental modification of stored data. Most SPI Flash chips provide a Status Register (accessed via the `RDSR` command) that indicates whether writing is allowed and which memory sectors are protected. Key fields include the WEL (Write Enable Latch) bit, which must be set using the `0x06` Write Enable command before any write or erase operation; the BPx bits, which define block-level protection for specific sectors; and the SRWD (Status Register Write Protect) bit, which locks the status register itself. Before performing any write or erase operations, it is essential to check and configure these bits to ensure the flash memory is in a writable state.

Enabling writes to SPI Flash is straightforward but critical for safe memory management. First, the CS line is pulled low to select the chip, then the Write Enable command (`0x06`) is sent via SPI to set the WEL bit. After the command, CS is pulled high, and the Status Register can optionally be read to verify that WEL is set. As long as WEL remains cleared, the chip will reject any write or erase commands, protecting firmware, configuration, and critical data from accidental modification. This controlled approach allows precise updates to specific blocks or sectors, ensuring safe firmware updates, bootloader execution, and configuration storage without risking corruption of the device.

Writing data to SPI Flash is a multi-step process because the chip cannot overwrite memory directly without preparation. First, you must enable writing by sending the Write Enable (`0x06`) command, which sets the WEL (Write Enable Latch) bit in the Status Register. Next, if necessary, you erase a sector (typically 4KB) using the Sector Erase command (`0x20`), which resets all bytes in that sector to `0xFF`. Then, the Byte or Page Write command (`0x02`) is sent, including the target address and the data byte(s) to be written. After issuing a write, you must wait for the operation to complete by polling the Status Register until the BUSY bit clears. Optionally, you can verify the write by reading the byte(s) back to ensure the correct value was stored. This structured process ensures data integrity and prevents accidental overwrites.

Many SPI Flash chips organize memory into pages, usually 256 bytes in size. Writing an entire page at once is much more efficient than writing single bytes individually. Single-byte writes are slower and can cause more wear on the memory over time, whereas page writes allow you to send up to 256 consecutive bytes in a single SPI transaction, improving speed and reducing wear. This method is ideal for bulk data updates, such as firmware uploads or storing large configuration tables. Flash page writes wrap around at page boundary (256 bytes), so care must be taken not to cross pages unintentionally.

17.4.1 Program: 0073_spi_write_byte.spin2

Listing 17.3: Program: 0073_spi_write_byte.spin2

```

'spi_write_byte.spin2
CON
    _CLKFREQ          = 180_000_000
    BAUD_RATE         = 2_000_000
    TRANSMISSION_PIN  = 62
    RECEPTION_PIN     = 63
    BIT_PERIOD        = (_CLKFREQ / BAUD_RATE)
    BAUD_MODE         = (BIT_PERIOD << 16) | (8 - 1)

    SPI_CS            = 61
    SPI_CLK           = 60
    SPI_MOSI          = 59
    SPI_MISO          = 58

    SPI_CLK_MODE      = P_TRANSITION + P_OE
    SPI_MOSI_MODE     = P_SYNC_TX + P_OE | ((SPI_CLK - SPI_MOSI) & %111)
                        << 24
    SPI_MISO_MODE     = P_SYNC_RX | ((SPI_CLK - SPI_MISO) & %111) << 24
    SPI_CLK_DIV       = (_CLKFREQ / 100_000_000) #> 4
    SPI_8BIT_CONTROL  = $27

DAT
    org 0
    asmclk

    dirl    #TRANSMISSION_PIN
    wrpin   ##(P_ASYNC_TX | P_OE), #TRANSMISSION_PIN
    wxpin   ##BAUD_MODE, #TRANSMISSION_PIN
    dirh    #TRANSMISSION_PIN
    wy-pin  #1, #TRANSMISSION_PIN

    wrpin   ##SPI_CLK_MODE, #SPI_CLK
    wrpin   ##SPI_MOSI_MODE, #SPI_MOSI
    wrpin   ##SPI_MISO_MODE, #SPI_MISO
    wxpin   ##SPI_CLK_DIV, #SPI_CLK
    drvl    #SPI_CLK

    wxpin   ##SPI_8BIT_CONTROL, #SPI_MOSI
    wxpin   ##SPI_8BIT_CONTROL, #SPI_MISO

    mov flash_address, ##0
    mov page_size, ##255

    call #spi_write_enable

```

```
call #spi_erase_sector
call #spi_write_enable

mov flash_address, #$0
mov spi_reception, #"a"
call #spi_byte_write

jmp #endprog

spi_write_enable
    drvl #SPI_CS
    mov spi_command, #$06
    call #spi_transmit
    drvh #SPI_CS
    .wait_wel
        call #spi_read_status_register
        testb status_register, #2 wz
        if_z jmp #.wait_wel
    ret

spi_read_status_register
    drvl #SPI_CS
    mov spi_command, #$05
    call #spi_transmit
    call #spi_receive
    mov status_register, spi_reception
    drvh #SPI_CS
    ret

spi_erase_sector
    drvl #SPI_CS
    mov spi_command, #$20
    call #spi_transmit
    call #spi_send_address24
    drvh #SPI_CS
    .wait_busy
        call #spi_read_status_register
        testb status_register, #0 wz
        if_z jmp #.wait_busy
    ret

spi_send_address24
    mov spi_command, flash_address
    shr spi_command, #16
    and spi_command, #$FF
    call #spi_transmit
```

```

    mov spi_command, flash_address
    shr spi_command, #8
    and spi_command, #$FF
    call #spi_transmit

    mov spi_command, flash_address
    and spi_command, #$FF
    call #spi_transmit
    ret

spi_byte_write
    drvl #SPI_CS
    mov spi_command, #$02
    call #spi_transmit
    call #spi_send_address24
    mov byte_count, #256
    .page_loop
        mov spi_command, spi_reception
        call #spi_transmit
        djnz byte_count, #.page_loop
    drvh #SPI_CS
    ret

spi_transmit
    shl spi_command, #24
    rev spi_command
    wypin spi_command, #SPI_MOSI
    drvl #SPI_MOSI
    wypin #16, #SPI_CLK
    waitx ##1
    .waitRdy
        testp #SPI_CLK wz
        if_nz jmp #.waitRdy
    ret

spi_receive
    drvl #SPI_MISO
    fltl #SPI_MOSI
    wypin #16, #SPI_CLK
    waitx ##1
    .waitDone
        testp #SPI_CLK wz
        if_nz jmp #.waitDone
        rdpin spi_reception, #SPI_MISO
        rev spi_reception
        zerox spi_reception, #7
    ret

```

```

uart_transmit_char
    wypin uart_transmission, #TRANSMISSION_PIN
    .txwait
    rdpin pr2, #TRANSMISSION_PIN wc
    if_c    jmp #.txwait
    ret

endprog
    jmp    $$

spi_command        res 1
flash_address      res 1
byte_count         res 1
spi_reception      res 1
uart_transmission  res 1
page_size          res 1
status_register    res 1

```

The program starts by initializing `flash_address` to 0x000000, indicating the first memory location in the SPI Flash, and `page_size` to 255, defining the number of bytes in a flash page. This setup ensures the program knows where to begin writing and how many bytes each write operation will handle.

The main routine writes data to the SPI Flash in a safe sequence. It first calls `spi_write_enable` to allow write operations, then erases the sector containing the target address using `spi_erase_sector`. After enabling writing again, it writes 256 bytes of the character "e" to address 0x00 using `spi_byte_write`. This ensures the flash memory is cleared and ready for new data, preventing accidental overwrites.

The `spi_write_enable` subroutine sends the 0x06 Write Enable command to set the WEL (Write Enable Latch) in the status register. It repeatedly checks the WEL bit until it confirms the flash is ready to accept write commands. The `spi_read_status_register` routine sends the 0x05 command to retrieve the status register, which holds important flags like WEL and WIP (Write In Progress), indicating whether the chip is ready or busy.

`spi_erase_sector` pulls CS low, sends the 0x20 Sector Erase command, transmits the 24-bit sector address via `spi_send_address24`, and then pulls CS high to start the internal erase. It loops, polling the status register until the WIP bit clears, ensuring the sector is fully erased before writing. The `spi_send_address24` routine splits the 24-bit address into three bytes (MSB first) and sends them over SPI, required for any read, write, or erase operation.

The `spi_byte_write` routine programs a full page (256 bytes) starting at `flash_address`. It sends the Page Program command (0x02), the 24-bit address, and then loops to transmit all 256 bytes using the value in `spi_reception`. CS is pulled high at the end to complete the transaction, allowing efficient bulk writing rather than sending single bytes one at a time.

The `spi_transmit` routine shifts and reverses a byte, writes it to the MOSI smartpin, triggers clock pulses on `SPI_CLK`, and waits for completion, ensuring correct SPI timing. `spi_receive` reads a byte from MISO, floats MOSI, triggers clock pulses, and stores the reversed, aligned result in `spi_reception`. `uart_transmit_char` sends a byte over UART,

waiting for the previous transmission to finish to ensure reliable output.

When executed on a Propeller 2, this program erases the first sector of SPI Flash, enables writing, and writes a full page of "a" at address 0x00. The data is stored permanently until overwritten, and if connected to a UART terminal, it can display transmitted data for verification. This demonstrates safe, efficient SPI Flash programming.

17.5 Exercise

Create a Memory Inspector Lite that allows you to read and display single bytes from an SPI flash. The program connects to the flash via CS, MOSI, MISO, and SCLK pins and uses hardware SPI through SmartPins. The goal is to pull CS low, send the 0x03 READ command along with a 24-bit address, and read one byte at a time from the device, then pull CS high again. Each byte is sent over UART and displayed in hexadecimal format, allowing you to see exactly what is stored at each memory location. To make it interactive and beginner-friendly, the program can read and print the first eight bytes consecutively, each on a new line, giving immediate feedback without dealing with full pages or complex loops. This exercise reinforces basic SPI operations, proper timing with CS and SCLK, and converting binary data into human-readable hex, providing a hands-on way to explore memory contents safely.

17.6 Summary

This concludes our journey with SPI. Make sure to keep the following in mind when dealing with SPI:

Chapter Summary

- SPI is a synchronous communication protocol
- Uses clock, data, and chip select lines
- Data transfers bit-by-bit
- Requires precise timing and control
- Used for external devices like flash memory

Chapter 18

Practice Programs

These are programs you should implement on your own to practice and reinforce what you’ve learned about P2ASM.

- ASCII Table Printer – Display all 128 ASCII characters.
- Echo – Read characters from UART and echo them back.
- Sleep – Pause program execution for a set duration.
- Uptime – Show the time since the program started.
- Basic Calculator – Perform simple arithmetic operations from UART input.
- List COGs 0–7 – Display the status of all Propeller 2 cores.
- Editor – Simple text editor using UART.

Looking Ahead: From Learning Examples to Production-Ready Code

Throughout this text, you’ve explored fundamental concepts in Propeller 2 programming — from UART communication and string parsing to bitwise operations and shifting. The programs presented are designed for clarity, learning, and experimentation. They intentionally simplify hardware interactions, use fixed-size buffers, and rely on busy loops or direct manipulation to make the examples easy to follow.

In a production environment, however, these programs would require enhancements to ensure safety, efficiency, and maintainability. For instance, error handling would need to be robust: input values should be validated, buffers protected against overflow, and hardware responses monitored for unexpected states. Blocking loops, such as those waiting on UART RX or TX, would be replaced with interrupt-driven routines or hardware FIFOs, allowing the CPU to perform other tasks concurrently.

Buffer management would be more sophisticated, using circular buffers or dynamic allocation where appropriate, and all memory accesses would include rigorous bounds checking. Command parsing would scale to support multiple commands, optional parameters, and

case-insensitive comparison, with clear feedback for invalid input. Similarly, bitwise operations and shifting routines would be modularized, using descriptive constants for masks, flags, and bit positions to enhance readability and maintainability.

Finally, production code emphasizes modular design and separation of concerns. Low-level drivers handle hardware-specific tasks, utility routines manage data manipulation, and the main program coordinates the overall flow. This structure makes code easier to expand, debug, and maintain over time, whether controlling LEDs, reading sensors, or managing complex communication protocols.

In short, the programs you've written form the foundation of embedded programming skills. With careful attention to robustness, efficiency, and modularity, these foundations can be built into production-ready systems capable of reliable, real-world operation.

Appendix A

Integrating P2ASM with High-Level Programs

Throughout this book, we have written programs primarily in P2ASM so that we could see exactly how the Propeller 2 operates at the instruction level. In a larger application, however, it is not always necessary to write the entire program in assembly language.

A common approach is to write most of an application in a higher-level language such as Spin2 and use P2ASM only for small sections where additional speed, precise timing, or direct hardware control is useful.

This gives us the advantages of both approaches. High-level code can handle the overall program structure, while P2ASM handles the small portions of the program where instruction-level control is important.

Spin2 provides a convenient way to do this because P2ASM instructions can be placed directly inside a Spin2 method.

A.1 Inline P2ASM

An inline P2ASM section begins with **org** and ends with **end**.

A basic method can therefore have the following structure:

```
PUB example(value) : result

    result := value

    org
    ' P2ASM instructions
    end
```

The code outside the assembly block is Spin2. The instructions between **org** and **end** are P2ASM.

Spin2 method parameters, return values, and local variables can be used to exchange values with the inline P2ASM code. This makes small assembly routines convenient because the high-level program can prepare the data, allow P2ASM to process it, and then continue

using the result.

Conceptually, the program looks like this:

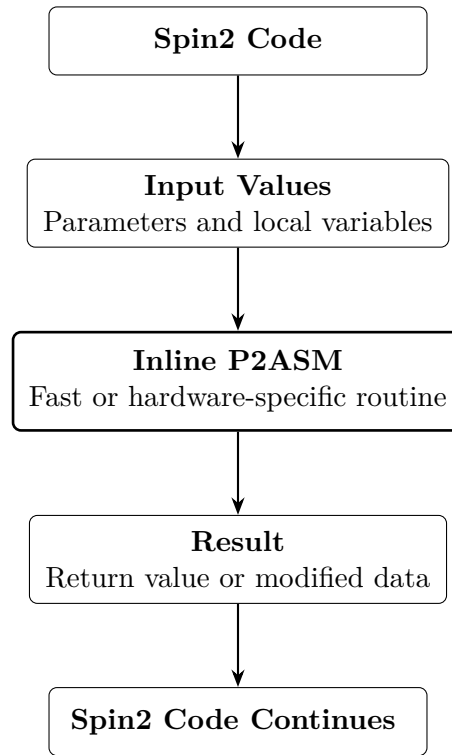


Figure A.1: Data flow between Spin2 and an inline P2ASM routine

Inline assembly is useful when only a small portion of a program requires assembly-language performance.

A.2 Example: Adding a Value

We will begin with a deliberately simple example.

The following Spin2 method receives a number, copies it into the return variable, and then uses P2ASM to add two.

```

PUB add_two(value) : result

    result := value

    org
    add     result, #2
    end
  
```

If we call:

```

answer := add_two(5)
  
```

the returned value is:

```
7
```

Most of the method is still written using the normal Spin2 method structure. Only the operation we want to perform in assembly appears between `org` and `end`.

The P2ASM instruction:

```
add result , #2
```

works just as the `add` instruction introduced earlier in this book.

The important difference is that `result` is also the return value of the surrounding Spin2 method.

The flow is therefore:

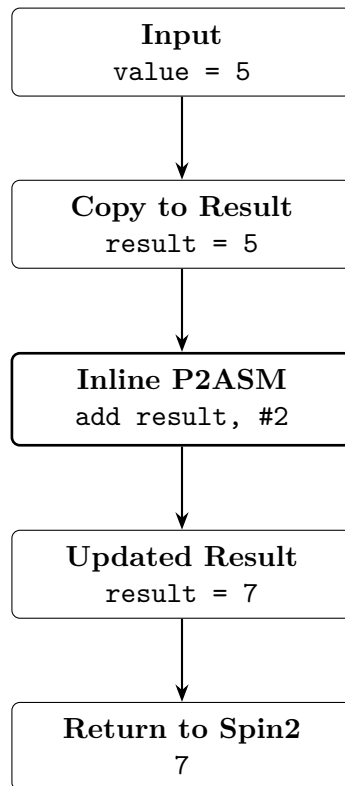


Figure A.2: Value flow through the inline P2ASM addition example

This tiny example does not need assembly for performance, but it demonstrates how easily a P2ASM operation can be inserted into a high-level program.

A.3 Example: Fast Multiplication by Shifting

Suppose a program needs to multiply a value by eight.

Earlier we learned that shifting a value left can efficiently multiply it by powers of two.

We can place that operation inside a Spin2 method:

```
PUB multiply_by_eight(value) : result

    result := value

    org
    shl     result, #3
    end
```

Calling:

```
answer := multiply_by_eight(4)
```

produces:

```
32
```

The operation performed by P2ASM is:

```
4 << 3 = 32
```

Again, the purpose of this example is not to suggest that every multiplication should be replaced with inline assembly. Spin2 is already capable of performing ordinary arithmetic.

Instead, it demonstrates an important design principle:

Move into P2ASM only the portion of the program that benefits from assembly-level control.

A larger program might perform user interaction, configuration, menus, calculations, or communication in Spin2 while using a short P2ASM section for a particularly performance-sensitive operation.

A.4 Example: Processing Several Values

Inline P2ASM becomes more interesting when we perform repeated operations.

The following method demonstrates the basic idea of processing a group of 32-bit values stored in hub memory.

```
PUB sum_values(address, count) : total | value

    total := 0

    org
    .loop
    rdlong value, address
    add     address, #4
```

```

add    total, value
djnz   count, #.loop
end

```

The method receives `address`, which points to the first value in hub memory, and `count`, which tells the routine how many values to process.

The instruction:

```
rdlong value, address
```

reads one 32-bit value from hub memory.

The instruction:

```
add address, #4
```

moves the address to the next long.

We then add the value to the running total:

```
add total, value
```

Finally:

```
djnz count, #.loop
```

decrements the counter and repeats the loop until every value has been processed.

Conceptually:

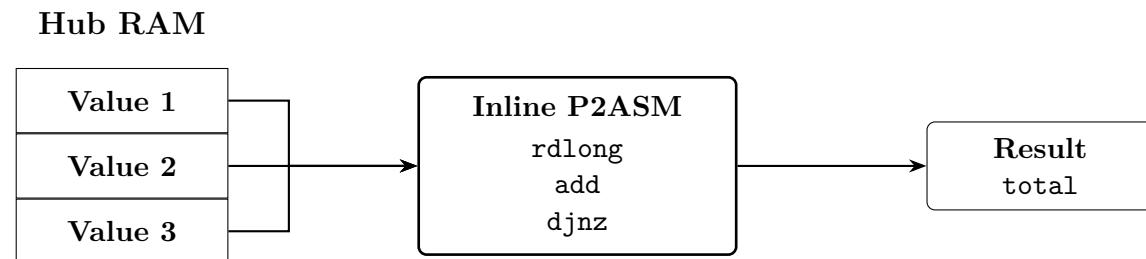


Figure A.3: Processing multiple values from Hub RAM using inline P2ASM

This is closer to the type of routine for which inline assembly may be useful.

A high-level program could manage the data and application logic while P2ASM performs the repeated low-level operation.

A.5 Example: Direct Pin Control

Another important reason to use P2ASM is precise hardware control.

Consider a method that creates a short pulse on an I/O pin:

```
PUB pulse(pin)
```

```

org
drvh    pin
nop
drvl    pin
end

```

The first instruction:

```
drvh pin
```

drives the selected pin high.

The `nop` instruction performs no operation but consumes an instruction period.

Finally:

```
drvl pin
```

drives the pin low again.

Conceptually:

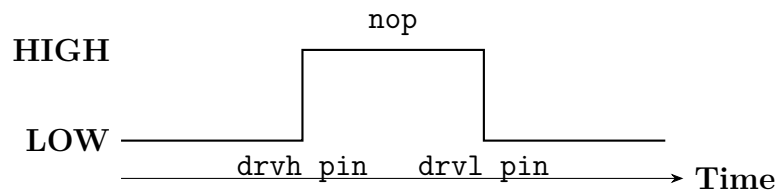


Figure A.4: Pin state during the inline P2ASM pulse example

A real hardware interface may require more instructions, delays, SmartPin configuration, or synchronization. However, this example demonstrates why assembly can be useful inside an otherwise high-level application.

High-level code can decide **when** an operation should happen.

P2ASM can precisely control **how** the hardware operation happens.

This separation is often useful in embedded programming.

A.6 Using Local Variables

Local variables can also be used as working values for an inline P2ASM routine.

For example:

```

PUB calculate(value) : result | temp

temp := value

org
shl    temp, #1

```

```

add      temp, #5
mov      result, temp
end

```

If:

```
value = 10
```

the P2ASM performs:

```

10 << 1 = 20
20 + 5  = 25

```

and places the final value into `result`.

The local variable `temp` acts as a working value for the calculation.

The major difference from the standalone P2ASM programs used throughout this book is that the variable belongs to the surrounding Spin2 method.

This provides a convenient interface between the high-level portion of the method and its assembly routine.

A.7 Persistent Values

There is an important distinction between local method variables and variables that must remain available between method calls.

A local variable belongs to the current invocation of a method. If we need to preserve a value between calls, it can be stored in the object's `VAR` section and moved into a method value before entering inline P2ASM.

For example:

```

VAR

long counter

```

We can then write:

```

PUB increment_counter() : result

result := counter

org
add     result, #1
end

counter := result

```

The flow is:

Suppose `counter` initially contains 10.

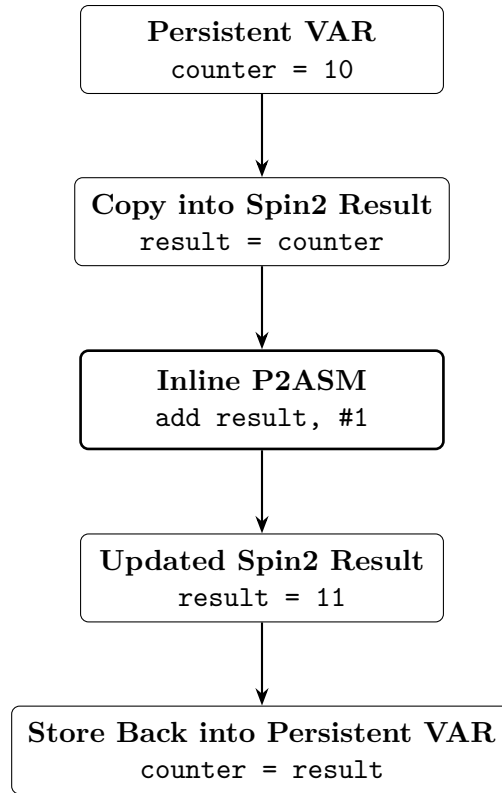


Figure A.5: Preserving a value between Spin2 method calls

The method copies 10 into `result`. P2ASM changes it to 11, and Spin2 then stores 11 back into `counter`.

The next time the method is called, the calculation begins with 11 instead of starting over.

This technique keeps persistent application state in high-level program storage while allowing P2ASM to perform the operation itself.

A.8 Inline P2ASM vs. a Separate Cog

Inline P2ASM should not be confused with starting an independent cog.

With inline assembly, the high-level program reaches the assembly block, executes it, and then continues with the high-level code.

Conceptually:

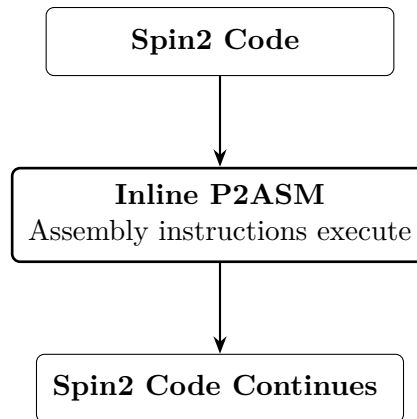


Figure A.6: Execution flow through an inline P2ASM section

A separate cog operates differently.

One cog might continue running the primary program while another cog performs an independent task:

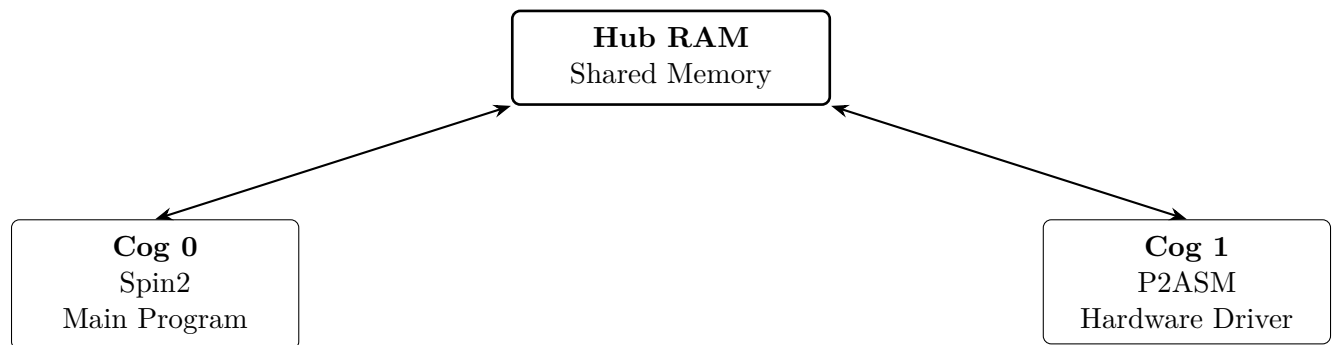


Figure A.7: Two cogs running independently while sharing Hub RAM

The Propeller 2 contains eight cogs, so larger applications can use both techniques.

For example, a program might use inline P2ASM for a short calculation while starting another cog to continuously operate a communications or hardware driver.

Inline P2ASM is generally attractive when the assembly operation is short, called when needed, and returns to the caller.

A separate cog is attractive when the operation must run independently, continuously, or concurrently with other code.

A.9 A Practical Design Pattern

One useful way to design Propeller 2 programs is to begin in the high-level language.

For example:

```
PUB main() | value
```

```

value := get_value()

value := fast_process(value)

display_value(value)

```

The application itself remains easy to understand.

The performance-sensitive routine can then contain P2ASM:

```

PUB fast_process(value) : result

result := value

org
shl     result, #2
add     result, #1
end

```

The program therefore becomes:

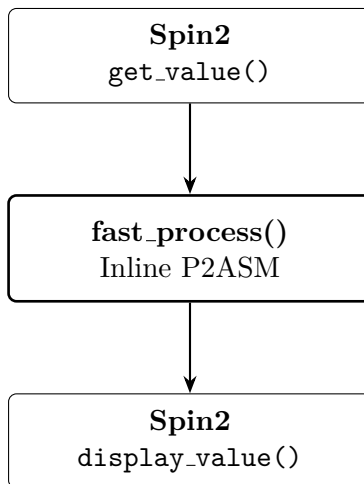


Figure A.8: Using inline P2ASM for one performance-sensitive part of a Spin2 program

This approach also allows an application to begin entirely in Spin2.

If testing later shows that one routine is too slow or requires tighter timing, that routine can become a candidate for P2ASM.

There is no need to optimize portions of the application that already perform adequately.

A.10 Start Small

If this is your first time combining P2ASM with a high-level language, begin with very small routines.

Good first candidates include:

- Incrementing a value
- Shifting a value
- Applying a bit mask
- Processing a short loop
- Reading or writing a pin
- Reading values from hub RAM

These operations use the same instructions introduced throughout this book.

The major new concept is simply that the P2ASM instructions are now operating as one small part of a larger program.

A useful progression is:

1. Write the high-level program
2. Identify one small operation that could benefit from P2ASM
3. Write that operation in P2ASM
4. Place it between `org` and `end`
5. Test the result
6. Expand the assembly portion only when necessary

Appendix B

Propeller 2 Assembly Reference

This appendix provides a categorized reference of Propeller 2 assembly instructions and directives included in this manual. It is not a complete listing of all instructions supported by the Propeller 2 architecture.

B.1 Hub / Cog Setup & Program Control

Name	Cycles	Description
org	0	Sets hub memory origin
asmclk	0	Synchronizes the COG clock to the system clock
coginit	2–9	Starts a COG running a specified hub address
cogid	2–9	Returns the ID of the current COG
cogstop	2–9	Stops the current COG
locknew	4–11	Allocates a hardware lock (returns lock ID)
locktry	2–9	Attempts to acquire lock without blocking
lockrel	2–9	Releases a previously acquired lock

B.2 Pin / SmartPin Instructions

Name	Cycles	Description
wrpin	2	Writes SmartPin mode/configuration
wxpin	2	Writes X parameter (timing/config value)
wypin	2	Writes Y parameter (data/compare value)
dirh	2	Sets pin direction to output
testp	2	Tests pin state
rdpin	2	Reads SmartPin input or FIFO value
rqpin	2	Requests SmartPin data (non-blocking)
akpin	2	Acknowledges SmartPin data
drvl	2	Drives pin output low
drvh	2	Drives pin output high
ftl	2	Sets pin to floating input (high-Z)
zerox	2	Clears SmartPin internal state

B.3 Arithmetic Instructions

Name	Cycles	Description
add	2	Adds source to destination
sub	2	Subtracts source from destination
mul	2	Multiplies two 16-bit values and stores 32-bit result
qmul	2–9	Hardware multiply using Q registers
qdiv	2–9	Hardware divide using Q registers

B.4 Bitwise / Shift Instructions

Name	Cycles	Description
and	2	Bitwise AND
or	2	Bitwise OR
xor	2	Bitwise XOR
not	2	Bitwise NOT
shl	2	Logical shift left
shr	2	Logical shift right (fills with 0)
sal	2	Shift arithmetic left (same as shl)
sar	2	Arithmetic shift right (preserves sign)
rev	2	Reverses bit order of a value

B.5 Flow Control

Name	Cycles	Description
jmp	4–20	Unconditional jump
djnz	2–20	Decrement and jump if not zero
tjnz	2–20	Jump if not zero
tjz	2–20	Jump if zero
call	13–20	Call subroutine
ret	13–20	Return from subroutine
waitx	2	Wait for system counter delay
nop	2	No operation

B.6 Compare & Conditional Execution

Name	Cycles	Description
cmp	2	Compare values and set flags
if_z	2–9	Execute next instruction if zero flag set
if_nz	2–9	Execute if zero flag not set
if_c	2–9	Execute if carry flag set
if_nc	2–9	Execute if carry flag not set
if_be	2–9	Execute if below or equal (unsigned)
if_a	2–9	Execute if above (unsigned)

B.7 Timing / Counter Instructions

Name	Cycles	Description
getct	2	Reads system counter
addct1	2	Adds value to CT1 compare register
waitct1	2+	Waits for CT1 match
pollct1	2	Non-blocking CT1 check

B.8 Data Movement / Memory Access

Name	Cycles	Description
mov	2	Copy value between registers
rdbyte	9–16	Read 8-bit value from hub memory
wrbyte	3–10	Write 8-bit value to hub memory
rdword	9–16	Read 16-bit value from hub memory
wrword	3–10	Write 16-bit value to hub memory
rdlong	9–16	Read 32-bit value from hub memory
wrlong	3–10	Write 32-bit value to hub memory
getqx	–	Reads result from QX after qmul/qdiv